UNIVERSIDAD TECNOLÓGICA METROPOLITANA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE ELECTRICIDAD
ESCUELA DE ELECTRÓNICA

NAVEGACIÓN Y EXPLORACIÓN AUTÓNOMA DE INTERIORES UTILIZANDO UN ROBOT MÓVIL DE TRACCIÓN DIFERENCIAL

TRABAJO DE TITULACIÓN PARA OPTAR AL TÍTULO DE INGENIERO CIVIL ELECTRÓNICO

AUTOR:

VALDERAS NECULQUEO, SEBASTIÁN

PROFESOR GUÍA:

GALARCE ACEVEDO, PATRICIO

SANTIAGO - CHILE

2026

Autorización para la Reproducción del Trabajo de Titulación

1. Identificación del trabajo de titulación

Nombre del alumno

RUT .....
Dirección .....
E-mail .....
Teléfono .....

Título de la tesis

Escuela .....
Carrera o programa .....
Título al que opta .....

2. Autorización de Reproducción (seleccione una opción)

a) Este trabajo de titulación no puede reproducirse o transmitirse bajo ninguna forma o por ningún medio o procedimiento, sin permiso escrito del autor, exceptuando la cita bibliográfica, resumen y metadatos que acreditan al trabajo y a su autor.

Fecha: _____ Firma: _____

b) Se autoriza la reproducción total o parcial de este trabajo de titulación, con fines académicos, por cualquier medio o procedimiento, incluyendo la cita bibliográfica que acredita al trabajo y a su autor.

En consideración a lo anterior, se autoriza su reproducción de forma (marque con una X):

	Inmediata
	A partir de la siguiente fecha: _____ (mes/año)

Fecha: _____ Firma: _____

Esta autorización se otorga en el marco de la ley N°17.336 sobre Propiedad Intelectual, con carácter gratuito y no exclusivo para la Institución.

NOTA OBTENIDA:

Firma y timbre de la
autoridad responsable

DEDICATORIA

A mi familia,
mi apoyo constante
y el lugar al que siempre puedo volver.

Al Grupo Scout de Don Orione,
a los lazos que permanecen
y a los valores que llevo conmigo.

Al Club de Robótica,
a nuestra curiosidad compartida
y al entusiasmo de construir juntos.

A mis compañeros de la generación 2016
de Ingeniería Civil en Electrónica,
a los recuerdos y las amistades
de una etapa que siempre será parte de mí.

A mis compañeros de PwC,
parte de mi presente
y del profesional en que me sigo convirtiendo.

A todos ustedes dedico este logro,
con el cariño de saber que cada uno tiene un lugar en esta historia.

«La máquina puede hacer el trabajo de cincuenta hombres corrientes. Ninguna máquina puede hacer el trabajo de un hombre extraordinario. »

Elbert Hubbard

«Traten de dejar este mundo en mejores condiciones de como lo encontraron. »

Robert Baden-Powell

AGRADECIMIENTOS

Al finalizar este trabajo, agradezco a quienes me acompañaron durante mi formación y contribuyeron, desde distintos espacios de mi vida, a que pudiera llegar hasta este momento.

A mi profesor guía, Patricio Galarce Acevedo, por su orientación, paciencia y disposición durante el desarrollo de este trabajo de titulación. Agradezco sus conocimientos, sus observaciones y el tiempo dedicado a acompañar este proceso, que me permitió aprender a abordar los desafíos con mayor rigor y perseverancia.

A mi familia, por el cariño y el apoyo que han sostenido mi formación. Gracias por confiar en mí, por escucharme y por acompañar tanto las alegrías como las dificultades de este recorrido. Este logro también guarda el valor de su presencia y de todos esos gestos cotidianos que, aunque no aparecen en estas páginas, hicieron una diferencia en mi camino.

Al Grupo Scout de Don Orione, por ser parte de mi formación humana y por las experiencias compartidas en torno al servicio, la fraternidad y el compromiso. A quienes me acompañaron en esa etapa, gracias por enseñarme el valor de colaborar, asumir responsabilidades y procurar que nuestras acciones aporten algo a los demás. Son aprendizajes que me acompañan más allá de cualquier espacio académico o profesional.

A mis compañeros del Club de Robótica, por compartir la curiosidad, el entusiasmo y las ganas de construir. Gracias por el intercambio de ideas, por aprender juntos y por hacer de la robótica un espacio de encuentro. En esa experiencia compartida encontré motivaciones que siguen presentes en mi manera de enfrentar nuevos proyectos.

A mis compañeros de la generación 2016 de Ingeniería Civil en Electrónica de la Universidad Tecnológica Metropolitana, por las jornadas de estudio, el inter-

cambio de conocimientos y el compañerismo durante nuestra formación. Agradezco haber compartido con ustedes una etapa que dejó aprendizajes, recuerdos y vínculos que forman parte de mi historia.

A mis compañeros de PwC, por la generosidad al compartir sus conocimientos, por las conversaciones y por las oportunidades de aprender en el trabajo cotidiano. Gracias por el compañerismo y por contribuir a mi crecimiento profesional, ampliando mi forma de comprender los problemas y de buscar soluciones junto a otros.

A los docentes que fueron parte de mi formación y a quienes revisaron este trabajo, por sus enseñanzas, su tiempo y las observaciones que contribuyeron a mejorarlo.

Finalmente, agradezco a todas las personas que, con una palabra de ánimo, una idea o su compañía, estuvieron presentes durante este proceso. Al cerrar esta etapa, me llevo también la gratitud de haberla compartido con ustedes.

TABLA DE CONTENIDO

Página

DEDICATORIA iv

AGRADECIMIENTOS v

RESUMEN xvi

ABSTRACT xvii

INTRODUCCIÓN 1
- Planteamiento del problema 1
- Justificación 1
- Objetivo general 2
- Objetivos específicos 2
- Alcances y limitaciones 3
- Metodología 4
- Organización del documento 4

CAPÍTULO 1 ESTADO DEL ARTE 5
- 1.1 Panorama general de la robótica móvil 5
- 1.2 Plataformas de *software* y *middleware* robótico 6
- 1.3 Percepción y mapeo del entorno 7
- 1.4 Navegación y exploración autónoma 9
  - 1.4.1 Plataformas para navegación autónoma 10
  - 1.4.2 Exploración autónoma 11
- 1.5 Simulación de robots móviles 11

CAPÍTULO 2 SOLUCIÓN IMPLEMENTADA 14
- 2.1 Selección de tecnologías 15

CAPÍTULO 3 MARCO TEÓRICO 17
- 3.1 El robot móvil: estructura, marcos de referencia y rotaciones 17
  - 3.1.1 Descripción del modelo mediante URDF 18
  - 3.1.2 Simulación robótica y Gazebo Classic 19
- 3.2 ROS 2: nodos, tópicos y comunicación 20
- 3.3 Principio de funcionamiento de sensores y actuadores 21
  - 3.3.1 Sensor láser LIDAR 2D 21
  - 3.3.2 *Encoders* incrementales de cuadratura 21
  - 3.3.3 Motores DC y *driver* de potencia 23

3.4 Cinemática diferencial y odometría 23
3.4.1 Estimación de estados de rueda a partir de *encoders* . 25
3.5 Identificación FOPDT 25
3.6 Control PID 26
3.7 Sintonización del controlador PID mediante PSO 27
3.7.1 Optimización por enjambre de partículas 28
3.8 SLAM basado en grafos 29
3.9 Localización en mapas conocidos 30
3.10 Planificación de rutas y control local 31
3.11 Exploración autónoma basada en fronteras 32

CAPÍTULO 4 IMPLEMENTACIÓN 34
4.1 Componentes y parámetros del prototipo 34
4.1.1 Configuración general del prototipo 34
4.1.2 Parámetros físicos y temporales de implementación . . 35
4.2 Gemelo digital del robot 36
4.2.1 Arquitectura del modelo digital 37
4.2.2 Estructura URDF/xacro y marcos de referencia 37
4.2.3 Selección del *backend* de control 38
4.2.4 Sensor láser simulado y criterios de aceptación 39
4.3 Construcción de *hardware* y comunicación con ROS 2 40
4.3.1 Arquitectura física y de datos 40
4.3.2 Integración con ROS 2 41
4.3.3 Control diferencial e interfaz de *hardware* 42
4.3.4 Flujo de datos del sistema 43
4.3.5 Teleoperación y multiplexación de comandos 45
4.4 *Firmware* de bajo nivel en Arduino Nano 45
4.4.1 Arquitectura modular y constantes físicas 46
4.4.2 Comunicación serial y lectura de *encoders* 46
4.4.3 Control de velocidad en el *firmware* 47
4.4.4 Bucle principal y autodetención 51
4.4.5 Sintonización del controlador asistida por PSO 52
4.5 Arquitectura de control y navegación: SLAM, Nav2 y exploración 54
4.5.1 Mapeo con `slam_toolbox` 54
4.5.2 Localización sobre un mapa conocido con AMCL 55
4.5.3 Navegación autónoma con Nav2 55
4.5.4 Mapas de costos 57
4.5.5 Árbol de comportamiento y recuperación 57
4.5.6 Exploración autónoma de fronteras 58
4.5.7 Transferencia simulación–*hardware* 59

CAPÍTULO 5 RESULTADOS EXPERIMENTALES 62
5.1 Evaluación de odometría y control diferencial 63
5.2 Evaluación del control de velocidad 66
5.3 Resultados de mapeo con `slam_toolbox` 70
5.4 Navegación autónoma y exploración 72
5.5 Coherencia entre simulación y *hardware* 77
5.6 Alcance estadístico de los ensayos 77
CONCLUSIONES 79
Limitaciones 82
Trabajo futuro 84
BIBLIOGRAFÍA 86
GLOSARIO 98
ANEXOS 99
ANEXO 1 Construcción, componentes y montaje 100
ANEXO 1.1 Costo de los materiales 100
ANEXO 1.2 Comparación con plataformas comerciales 101
ANEXO 1.3 Parámetros de impresión 3D 102
ANEXO 2 Gemelo digital y verificación estructural 103
ANEXO 2.1 Validación estructural del URDF 104
ANEXO 2.2 Criterios de aceptación del gemelo digital 105
ANEXO 3 *Hardware*, red e integración con ROS 2 105
ANEXO 3.1 Entorno de ejecución y *workspace* 105
ANEXO 3.2 Nodos del lanzamiento en *hardware* real 107
ANEXO 3.3 Asignación de pines y dispositivos 108
ANEXO 3.4 Nodo de adquisición del RPLidar A1 109
ANEXO 3.5 Temporalidad y ciclo de la interfaz `DiffDriveArduino` 109
ANEXO 3.6 Identificadores de código de los parámetros del cuerpo 110
ANEXO 4 *Firmware*, *encoders* y sintonización del controlador . . 111
ANEXO 4.1 *Ticks* por vuelta y ganancias finales 111
ANEXO 4.2 Comparación ilustrativa entre PSO y sintonización Lambda 112
ANEXO 4.3 Protocolo de comunicación serial 113
ANEXO 4.4 Ecuaciones y estados internos del PI incremental embebido 116
ANEXO 4.5 Origen de la ley de control y cambios respecto del *firmware* original 117
ANEXO 4.6 Procesamiento FOPDT usado por los *scripts* . . . 118

ANEXO 4.7 Planta utilizada durante la sintonización del controlador 119
ANEXO 4.8 Particularidades de la implementación PSO 119
ANEXO 4.9 Protocolos de captura, identificación y sintonización 121
ANEXO 4.10 Resultados de la ejecución PSO empleada . . . 122
ANEXO 4.11 Captura de la respuesta en lazo cerrado 123
ANEXO 4.12 Análisis de estabilidad del lazo de velocidad . . 124
ANEXO 5 Configuración complementaria de SLAM, Nav2 y exploración 127
ANEXO 5.1 Lanzamiento de la localización con AMCL 128
ANEXO 5.2 Geometría del *costmap* e inflación 128
ANEXO 5.3 Acciones de recuperación de Nav2 129
ANEXO 5.4 Parámetros principales de SLAM en *hardware* real 129
ANEXO 5.5 Parámetros y flujo operativo de exploración 131
ANEXO 5.6 Particularidades del explorador de fronteras . . . 131
ANEXO 6 Detalle de validación de odometría 132
ANEXO 6.1 Trayectoria rectilínea de 2 m 133
ANEXO 6.2 Protocolo angular con referencia nominal de 360° 134
ANEXO 7 Resultados complementarios de exploración y mapeo 134
ANEXO 7.1 Exploración autónoma por iteración 135
ANEXO 8 Procedencia y limitaciones de las métricas complementarias 137
ANEXO 8.1 Procedencia de los indicadores de mapeo 137
ANEXO 8.2 Comparación cualitativa de mapas 137
ANEXO 8.3 Mediciones no incluidas en los ensayos 138
ANEXO 8.4 Limitaciones de los contadores de exploración . 138
ANEXO 8.5 Comparación de los mapas con el plano del recinto 139
ANEXO 8.6 Intervalos de confianza de los indicadores 142
ANEXO 9 Convención de rutas y localización del *software* 144
ANEXO 10 Nomenclatura 146

ÍNDICE DE TABLAS

Página

1.1 Comparación general de plataformas y *middleware* para robótica. 7
1.2 Comparación general de alternativas para SLAM 2D. 8

4.1 Parámetros físicos y temporales empleados en la implementación. 36
4.2 *Plugin* de `ros2_control` según entorno de ejecución. 39
4.3 Parámetros principales del controlador diferencial. 43
4.4 Ganancias definitivas del controlador codificadas en el *firmware*. 50
4.5 Configuración de los planificadores global y local utilizados. . . 57

5.1 Métricas diferenciadas de seguimiento y odometría en la trayectoria rectilínea de 2 m. 64
5.2 Desviación angular respecto de la referencia nominal de 360° (5 repeticiones por entorno). 65
5.3 Indicadores de la respuesta al escalón en lazo cerrado, medidos sobre la captura experimental del robot real con referencia de 0.13 m/s. 69
5.4 Indicadores de mapeo en *hardware* real por iteración (resolución 0.05 m/-celda). 71
5.5 Estado de validación por subsistema. 76
A1.1 Costo de los materiales del prototipo. 101
A1.2 Comparación de costo con plataformas comerciales. 102
A1.3 Parámetros generales de impresión 3D. 103
A2.1 Resumen de elementos cinemáticos del modelo. 105
A2.2 Criterios mínimos de aceptación del gemelo digital. 105
A3.1 Paquetes principales del *workspace*. 106
A3.2 Equipamiento de la estación de trabajo y versiones del entorno de ejecución. 107
A3.3 Nodos iniciados por el lanzamiento en *hardware* real. 108
A3.4 Asignación resumida de pines entre Arduino, L298N y *encoders*. 108
A3.5 Correspondencia entre parámetros del cuerpo e identificadores del código. 111
A4.1 Muestras del número de *ticks* por revolución. 112
A4.2 Comandos del protocolo serial implementado en el *firmware*. 115

A4.3 Configuración de la sintonización por PSO (qué se optimiza y con qué hiperparámetros). 121
A4.4 Salida registrada de una ejecución de `SintonizacionAutomatica.py`: modelo FOPDT identificado y ganancias sugeridas por PSO. 122
A4.5 Estabilidad y robustez de los modelos lineales del lazo de velocidad con las ganancias definitivas en los escenarios de retardo de 0 y 0.1 s. 125
A5.1 Parámetros principales de Nav2 y DWB. 127
A5.2 Acciones de recuperación configuradas en Nav2. 129
A5.3 Parámetros principales de `slam_toolbox` cargados en *hardware* real. 130
A5.4 Parámetros principales del explorador de fronteras. 131
A6.1 Desviación de la odometría simulada respecto de la referencia nominal de 2 m (Gazebo Classic). La pose verdadera del simulador no queda registrada. 133
A6.2 Error de odometría en trayectoria rectilínea de 2 m en *hardware* real. El error corresponde a la discrepancia entre la odometría de *encoders* y la distancia física medida. 133
A6.3 Desviación del giro acumulado de `/odom` respecto de la referencia nominal de 360° en simulación. 134
A6.4 Desviación angular física respecto de la referencia nominal de 360° usada por el *script* en *hardware* real. Las desviaciones son bidireccionales (sobre- y subgiros). 134
A7.1 Resultado de las metas de navegación de la exploración, reconstruidas por identificador único a partir del flujo de estado de la acción `NavigateToPose` registrado en cada sesión. Las cinco iteraciones terminan al no quedar fronteras. 135
A8.1 Comparación cualitativa entre mapas de simulación y *hardware* real. 137
A8.2 Distancias geométricas en ambos sentidos y percentil 95, transcritos del informe de comparación existente. 141
A8.3 Distancias del plano del recinto y las mismas distancias medidas en los mapas de las cinco iteraciones válidas. 142
A8.4 Intervalos de confianza al 95 % de los indicadores obtenidos sobre *hardware* real. 143
A9.1 Localización y función de los archivos citados en el documento. 144

ÍNDICE DE ILUSTRACIONES

Página

2.1 Diagrama de bloques simplificado de la solución implementada. 14

3.1 Patrón de comunicación publicación–suscripción: los nodos publicadores envían mensajes a un tópico y los suscriptores los reciben, de forma anónima y desacoplada. 20
3.2 Canales en cuadratura de un *encoder* incremental y decodificación $\times 4$. 22
3.3 Lazo de control de velocidad de una rueda con controlador PID. 26
3.4 Grafo de poses: nodos, restricciones relativas y cierre de lazo. 30

4.1 Vista lateral del prototipo construido. 35
4.2 Modelo del robot (gemelo digital) cargado en Gazebo Classic. 37
4.3 Barrido del LIDAR 2D simulado en el mundo de prueba con obstáculos. 39
4.4 Vista en planta del prototipo construido. 41
4.5 Esquema de conexiones eléctricas y de datos del robot diferencial. 42
4.6 Interacción entre los componentes de la navegación autónoma. 56
4.7 Evolución del mapa de costos global durante una ejecución de navegación y exploración autónomas. 58

5.1 Entorno utilizado para las pruebas experimentales en *hardware* real. 62
5.2 Resultados por repetición de las pruebas rectilíneas y angulares en simulación y *hardware* real. 66
5.3 Respuesta en lazo cerrado del robot real con las ganancias sugeridas por PSO, antes del refinamiento experimental (referencia de 0.13 m/s). 67
5.4 Respuesta experimental en lazo cerrado del robot real con las ganancias definitivas y referencia de 0.13 m/s. 68
5.5 Ejecución de navegación autónoma y exploración por fronteras sobre *hardware* real: avance desde un mapa parcial hasta una representación de mayor cobertura, con la ruta planificada por Nav2 (trazo verde). 75

A2.1 Dimensiones del modelo del robot (gemelo digital): vistas lateral, frontal y en planta. 104
A4.1 Identificación de los modelos FOPDT de ambas ruedas en lazo abierto y comparación simulada de las respuestas en lazo cerrado obtenidas con las ganancias finales asociadas al procedimiento PSO y con la sintonización Lambda. 114
A7.1 Secuencia de una ejecución representativa de exploración por fronteras y navegación autónoma sobre *hardware* real (vista de RViz y cámara cenital del recinto). 136
A8.1 Plano del recinto de ensayo, con las distancias entre muros medidas con huincha de medir. 140
A8.2 Mapas de las cinco iteraciones válidas alineados con el plano del recinto. 141

ÍNDICE DE ALGORITMOS

Página

3.1 Construcción del modelo URDF/`xacro` del robot. 19
4.1 Ciclo ejecutado por la interfaz `DiffDriveArduino`. 44
4.2 Rutinas de reinicio y actualización del controlador discreto ejecutadas por el *firmware*. 50
4.3 Sintonización híbrida del controlador mediante identificación FOPDT y PSO. 52
4.4 Integración operativa de `explore_lite` con Nav2. 60

RESUMEN

Este trabajo presenta el diseño e implementación de un prototipo de robot móvil diferencial de bajo costo para mapeo 2D, navegación autónoma y exploración de interiores mediante ROS 2 Foxy. El sistema integra una Raspberry Pi 4, un Arduino Nano, *encoders* de cuadratura, un sensor RPLidar A1 y un gemelo digital desarrollado en Gazebo Classic. El control de velocidad de cada rueda es un controlador PI incremental, forma del PID heredada del *firmware* original, con ganancias iniciales obtenidas por optimización por enjambre de partículas (PSO) y refinadas en el robot. Su estabilidad se verifica con un modelo FOPDT de cada rueda. El mapeo se implementa con SLAM Toolbox, la navegación con Nav2 y la exploración mediante una estrategia basada en fronteras.

La odometría presenta un error medio de 3.63 % en trayectorias rectas sobre *hardware* real. El control de velocidad, evaluado a 0.13 m/s, alcanza un error estacionario cercano a cero y no muestra sobreimpulso apreciable dentro de la resolución de medida. La exploración autónoma produce mapas válidos en cinco de las seis ejecuciones experimentales, con un índice medio de cobertura libre de 62.2 % y un área libre mapeada media de 3.50 m^2. Las cinco ejecuciones válidas terminan automáticamente cuando el explorador ya no encuentra fronteras, sin intervención manual. De las 60 metas de navegación generadas durante ellas, 54 quedan abortadas, una se completa y cinco no alcanzan a registrar su estado final porque la exploración se detiene mientras están en curso. El análisis del registro asocia 53 de esos abortos al reemplazo de la meta en curso, que el explorador renueva cada 3.3 s y que Nav2 informa como abortada. Frente a un plano medido del recinto, los mapas tienen una distancia media de 3.4 cm. Los resultados verifican funcionalmente el mapeo y la exploración autónoma con un número reducido de repeticiones. La tasa de éxito de la navegación hacia metas fijas no se evalúa y queda como trabajo futuro.

Palabras clave: robot móvil diferencial, SLAM, ROS 2, Nav2, navegación autónoma, exploración de fronteras, PSO, gemelo digital.

ABSTRACT

This thesis presents the design and implementation of a low-cost differential mobile robot prototype for 2D mapping, autonomous navigation, and indoor exploration using ROS 2 Foxy. The system integrates a Raspberry Pi 4, an Arduino Nano, quadrature encoders, an RPLidar A1 sensor, and a digital twin developed in Gazebo Classic. Each wheel's speed is regulated by an incremental PI controller, the form of PID inherited from the original firmware, with initial gains obtained through Particle Swarm Optimization (PSO) and refined on the robot. Its stability is verified on an FOPDT model of each wheel. Mapping is implemented with SLAM Toolbox, navigation with Nav2, and exploration through a frontier-based strategy.

Odometry shows a mean error of 3.63 % in straight-line trajectories on real hardware. Speed control, evaluated at 0.13 m/s, reaches a near-zero steady-state error and shows no appreciable overshoot within the measurement resolution. Autonomous exploration produces valid maps in five of the six experimental runs, with a mean free-space coverage index of 62.2 % and a mean mapped free area of 3.50 m^2. The five valid runs end automatically when the explorer finds no remaining frontiers, without manual intervention. Of the 60 navigation goals generated during these runs, 54 are aborted, one is completed, and five have no recorded final state because exploration stops while they are in progress. Log analysis temporally associates 53 of these aborts with the replacement of the active goal by a new one, since the explorer renews its target every 3.3 s and Nav2 reports the replaced goal as aborted. Compared with a measured floor plan of the test area, the maps show a mean distance of 3.4 cm. The results functionally verify mapping and autonomous exploration with a small number of repetitions. The success rate of navigation toward fixed goals is not evaluated and is left as future work.

Keywords: differential mobile robot, SLAM, ROS 2, Nav2, autonomous navigation, frontier exploration, PSO, digital twin.

INTRODUCCIÓN

La robótica móvil autónoma permite desarrollar sistemas capaces de desplazarse en entornos desconocidos, construir representaciones del espacio y ejecutar trayectorias sin intervención manual permanente. En este contexto, el uso de ROS 2, sensores LIDAR 2D, computadores embebidos como la Raspberry Pi 4 y microcontroladores de bajo costo facilita la implementación de prototipos académicos orientados a navegación, mapeo y exploración de interiores.

Planteamiento del problema

El desarrollo de un robot móvil autónomo para interiores requiere resolver de forma integrada la percepción del entorno, la estimación de la posición, el control del movimiento y la planificación de trayectorias. En una plataforma de bajo costo, estos procesos deben distribuirse adecuadamente entre el *hardware* embebido, encargado del control en tiempo real y la adquisición sensorial, y un computador de mayor capacidad que ejecuta el mapeo y la planificación, manteniendo la coherencia entre simulación y robot real. Por ello, el problema central de este trabajo consiste en determinar si es posible construir un prototipo de robot diferencial que, usando componentes accesibles y herramientas de código abierto, sea capaz de mapear, navegar y explorar entornos interiores de manera autónoma.

Justificación

Este proyecto se justifica por su aporte académico y tecnológico al integrar conocimientos de electrónica, control, robótica móvil, sistemas embebidos y *software* robótico. A diferencia de plataformas comerciales ya integradas, el prototipo

desarrollado permite documentar el proceso completo de construcción, programación, control y validación de un robot diferencial propio, incluyendo el *firmware* de bajo nivel, la interfaz con `ros2_control`, la configuración de SLAM, la navegación autónoma y la exploración de fronteras. Cabe precisar que el *firmware* y la interfaz de *hardware* no se desarrollan desde cero, sino que se adaptan a partir de proyectos de código abierto (Home Brew Robotics Club, s.f.; Newans, 2022a, 2023), cuyo origen y rol se detallan en el ANEXO 3.1. La contribución original reside en su adaptación e integración, la parametrización geométrica del modelo, la identificación FOPDT, la sintonización del controlador PI incremental con refinamiento experimental sobre *hardware* real, con PSO como herramienta de apoyo para obtener las ganancias iniciales, y el análisis de estabilidad de la ley de control ejecutada.

Objetivo general

Diseñar e implementar un prototipo de robot móvil de tracción diferencial de bajo costo, basado en ROS 2, capaz de construir un mapa 2D de su entorno mediante SLAM y navegar de forma autónoma hacia destinos por explorar, generados automáticamente por una estrategia de exploración de fronteras.

Objetivos específicos

OE1. Diseñar y construir un prototipo de robot móvil diferencial, validado previamente mediante un gemelo digital en el simulador Gazebo Classic.

OE2. Implementar el sistema de control de bajo nivel, incluyendo modelo cinemático diferencial, odometría por *encoders* y controlador PID en lazo cerrado sobre las velocidades de las ruedas.

OE3. Implementar la localización, el mapeo 2D y la navegación autónoma del robot mediante SLAM Toolbox y Nav2.

OE4. Integrar una estrategia de exploración autónoma basada en fronteras para generar metas hacia zonas no mapeadas sin intervención manual continua.

Alcances y limitaciones

El alcance del trabajo comprende el diseño, construcción, programación y validación de un prototipo funcional de robot móvil diferencial para interiores. El sistema opera sobre superficies planas, utiliza un LIDAR 2D RPLidar A1 como sensor principal y emplea odometría por *encoders* para estimar el desplazamiento. La validación considera pruebas en simulación y en *hardware* real, abordando odometría, control, mapeo, navegación y exploración. Los ensayos se repiten un número reducido de veces, por lo que sus resultados se informan con intervalos de confianza y no se generalizan a otros entornos. La exactitud de los mapas se evalúa respecto de un plano del recinto con distancias medidas manualmente.

La evaluación de la navegación se realiza sobre los destinos que genera la estrategia de exploración, que son los comprometidos en el objetivo general. No forma parte del alcance un protocolo de navegación dirigida con metas fijas definidas por el operador, con tasa de éxito y error de posición final medido respecto de marcas físicas del recinto. Para esa evaluación se requiere definir las metas, medir las referencias físicas y registrar de forma independiente la pose final. Ese conjunto de mediciones no se obtiene durante los ensayos. En consecuencia, este trabajo no informa tasa de éxito ni error de posición final para metas fijas, y ese ensayo se plantea como trabajo futuro.

La localización comprometida es la que resuelve SLAM Toolbox durante el mapeo en línea, es decir, la estimación de la transformación entre el marco del mapa y el marco odométrico mientras el mapa se construye. La localización sobre un mapa previamente almacenado mediante AMCL no forma parte del alcance. La

pila de navegación utilizada incorpora ese nodo con sus parámetros por defecto, pero no se ajusta para este robot ni se ejecuta en ninguno de los ensayos.

Entre sus limitaciones se considera que el prototipo no incorpora IMU, cámara ni fusión sensorial, por lo que la odometría puede degradarse en trayectorias largas o giros acumulados. Además, el sistema no está orientado a exteriores, superficies irregulares ni obstáculos dinámicos rápidos. La implementación se desarrolla sobre ROS 2 Foxy y Ubuntu 20.04 LTS, con simulación en Gazebo Classic 11; esta combinación se congela al inicio del proyecto para garantizar la compatibilidad de la cadena completa de paquetes sobre el *hardware* disponible. La migración a una distribución con soporte vigente se plantea como trabajo futuro.

Metodología

La metodología es experimental e iterativa, organizada en seis etapas: Diseño del gemelo digital y simulación, Construcción del *hardware*, Control de bajo nivel, SLAM y navegación autónoma, Exploración y validación final, y Documentación.

Organización del documento

El documento avanza desde las alternativas técnicas y las decisiones adoptadas (Estado del Arte y Solución Implementada) hacia los fundamentos de cada elemento (Marco Teórico), la construcción del prototipo y la integración de SLAM, navegación y exploración (Implementación), la validación experimental (Resultados) y el balance final (Conclusiones). La bibliografía, el glosario y los anexos complementan la información técnica del proyecto.

CAPÍTULO 1

ESTADO DEL ARTE

Este capítulo presenta una revisión general de las principales tecnologías utilizadas en robots móviles autónomos para interiores. El análisis se centra en las alternativas disponibles para plataformas de *software*, percepción, mapeo, navegación, exploración y simulación, considerando su capacidad de integración entre sí, sus requerimientos computacionales y su nivel de madurez.

1.1. Panorama general de la robótica móvil

El desarrollo actual de robótica móvil abarca aplicaciones en dominios tan diversos como la logística, la agricultura y las flotas multirrobot, entre otros (Macenski et al., 2023). La diversidad de aplicaciones ha dado lugar a robots con distintas arquitecturas de locomoción. Las plataformas diferenciales son habituales en robots móviles pequeños, por su simplicidad mecánica y su capacidad de girar sobre sí mismas (Corke, 2023, Sec. 4.1; Ferreira et al., 2023). Los vehículos con dirección tipo Ackermann se emplean cuando se requiere un movimiento semejante al de un automóvil. Las bases omnidireccionales permiten desplazamientos laterales en espacios reducidos. Cada configuración ofrece distinta maniobrabilidad y complejidad mecánica. Por ejemplo, un vehículo tipo automóvil no puede girar sobre sí mismo (Corke, 2023, Sec. 4.1).

También existen diferencias en el nivel de autonomía. Algunos sistemas siguen rutas previamente definidas, mientras que otros estiman su posición, construyen mapas, planifican trayectorias y reaccionan ante cambios del entorno. Por ejemplo, un robot logístico puede desplazarse entre estaciones conocidas; un robot de inspección puede recorrer una instalación y registrar mediciones; y una

plataforma de investigación puede explorar un recinto desconocido mientras genera su mapa. Desde el punto de vista computacional, estas funciones pueden concentrarse en un solo computador o distribuirse entre varios dispositivos. El control de los actuadores suele requerir una ejecución rápida y periódica, mientras que la percepción, el mapeo y la planificación demandan mayor capacidad de procesamiento. Por ello, los sistemas robóticos suelen descomponerse en componentes independientes que pueden ejecutarse en un mismo proceso, en varios procesos o en varios computadores (Macenski et al., 2022).

1.2. Plataformas de *software* y *middleware* robótico

La integración de sensores, actuadores y algoritmos puede realizarse mediante una plataforma robótica de propósito general, un *middleware* de comunicación o una arquitectura desarrollada específicamente para el sistema. Entre las alternativas más conocidas se encuentran ROS 1, ROS 2, YARP, Orocos RTT y LCM, además de soluciones propietarias utilizadas en plataformas comerciales. Estas alternativas no ofrecen exactamente el mismo nivel de funcionalidad. Algunas proporcionan bibliotecas, herramientas de visualización, gestión de paquetes y controladores de *hardware*, mientras que otras se concentran en la comunicación entre procesos o en la ejecución de componentes de tiempo real. La Tabla 1.1 presenta una comparación general.

En consecuencia, la selección de una plataforma de *software* depende de la disponibilidad de controladores, los requerimientos temporales, la capacidad computacional, la facilidad de integración, la comunidad disponible y el grado de desarrollo propio que puede asumir el proyecto. La alternativa adoptada para el prototipo y su relación con los restantes componentes se presentan en el Capítulo 2.

Tabla 1.1: Comparación general de plataformas y *middleware* para robótica.

Alternativa	Características generales	Alcance declarado por el proyecto
ROS 1	Amplio conjunto histórico de paquetes, controladores y herramientas para desarrollo robótico.	Cuenta con una comunidad y base de *software* extensa, pero su última distribución oficial se encuentra fuera de soporte desde 2025. (Open Robotics, 2025b)
ROS 2	Plataforma modular para sistemas distribuidos, con diferentes mecanismos de comunicación, calidad de servicio y herramientas para integrar *hardware* y algoritmos.	Ofrece un ecosistema amplio, aunque su capa de comunicaciones puede requerir ajustes, por ejemplo para transmitir mensajes de gran tamaño. (Macenski et al., 2022)
YARP	*Middleware* orientado a la comunicación entre módulos y a la abstracción de dispositivos; se utiliza en plataformas que requieren conectar componentes distribuidos.	El proyecto se define en torno a la comunicación entre módulos y la abstracción de dispositivos, sin declarar una pila propia de navegación móvil (YARP Developers, s.f.).
Orocos RTT	Entorno basado en componentes para control robótico y aplicaciones con requerimientos de tiempo real.	El proyecto se orienta al control determinista por componentes y no distribuye módulos de mapeo ni de planificación de trayectorias (The Orocos Project, s.f.).
LCM	Sistema ligero de publicación y suscripción, orientado al intercambio de mensajes con baja latencia.	El proyecto se define como una biblioteca de mensajería de baja latencia y no incluye componentes de percepción, mapeo ni navegación (LCM Project, s.f.).

1.3. Percepción y mapeo del entorno

En robots de interiores, las principales alternativas de percepción incluyen sensores LIDAR 2D, cámaras RGB-D, sistemas de visión estéreo y sensores LIDAR 3D. Las cámaras entregan mayor información visual y permiten incorporar reconocimiento de objetos, pero aumentan la carga de procesamiento y pueden verse afectadas por las condiciones de iluminación (Tourani et al., 2022). Los sensores LIDAR 3D ofrecen una representación más completa del entorno, porque

no se limitan a un único plano de barrido, a cambio de un mayor volumen de datos que procesar. En robots de interiores, en cambio, el SLAM basado en LIDAR 2D resulta más económico que su equivalente en 3D y constituye la alternativa predominante (Ran et al., 2025).

En relación con el mapeo, existen diferentes soluciones de SLAM 2D, entre ellas GMapping, Hector SLAM, Cartographer y `slam_toolbox`. Estas herramientas presentan diferencias en consumo computacional, dependencia de la odometría, capacidad de cierre de lazo e integración con ROS 2. La Tabla 1.2 resume la comparación general de estas alternativas.

Tabla 1.2: Comparación general de alternativas para SLAM 2D.

Alternativa	**Fortaleza principal**	**Limitación principal**
GMapping	Técnica de mapeo por rejilla con filtro de partículas (Grisetti et al., 2007), con buen desempeño en tiempo real y consumo de memoria acotado en escenas pequeñas (Ran et al., 2025).	Depende de una odometría consistente y carece de detección de cierre de lazo, y su portabilidad a ROS 2 es menor que la de las alternativas más recientes (Macenski & Jambrecic, 2021; Ngoc et al., 2023; Ran et al., 2025).
Hector SLAM	Puede operar con baja dependencia de la odometría de ruedas (Kohlbrecher et al., 2011).	Requiere un LIDAR de alta tasa de barrido para contener la deriva, y presenta deriva en el cierre de lazo en escenas extensas (Ran et al., 2025).
Cartographer	Buena consistencia global y capacidad de cierre de lazo (Hess et al., 2016).	Mayor complejidad de configuración y ajuste (Macenski et al., 2023; Ngoc et al., 2023). En una comparación independiente de cinco algoritmos SLAM 2D, Cartographer presenta el mayor uso medio de CPU (Trejos et al., 2022).
`slam_toolbox`	Integración con ROS 2, grafos de poses y almacenamiento de mapas.	En modo asíncrono puede omitir mediciones si el procesamiento de la anterior se retrasa (Macenski & Jambrecic, 2021).

En consecuencia, la elección del sensor de percepción y de la herramienta

de SLAM depende del costo, la carga computacional, la dependencia de la odometría y el grado de integración con ROS 2.

1.4. Navegación y exploración autónoma

La navegación autónoma combina dos decisiones de diseño: qué planificador global calcula la ruta y qué controlador local la sigue evitando obstáculos. Las alternativas disponibles para cada una difieren en la representación del entorno que requieren, en su costo computacional y en su respuesta ante obstáculos dinámicos.

En la planificación global, los métodos de búsqueda sobre rejilla, como Dijkstra y A* (LaValle, 2006, Sec. 2.2.2), siguen siendo los más utilizados en mapas de ocupación 2D, porque operan directamente sobre la rejilla y permiten replanificar de forma periódica con bajo costo computacional (Macenski et al., 2023). Los métodos basados en muestreo, como RRT y RRT*, se emplean sobre todo en espacios continuos de mayor dimensión o en sistemas con restricciones de movimiento (Karaman & Frazzoli, 2011; LaValle, 2006). Los métodos de optimización de trayectorias mejoran la suavidad y la factibilidad de la ruta, a cambio de un mayor costo de cómputo (Macenski et al., 2023).

En el control local coexisten tres familias. Los métodos reactivos, como el enfoque de ventana dinámica, evalúan un conjunto reducido de velocidades alcanzables y son habituales en robots diferenciales por su bajo costo y su buena respuesta ante obstáculos cercanos. Los métodos geométricos, como Pure Pursuit, siguen la ruta con menor esfuerzo de cálculo, pero responden peor ante obstáculos dinámicos. Los métodos predictivos optimizan una secuencia futura de comandos considerando la dinámica del robot, a costa de una mayor carga computacional (Macenski et al., 2023).

1.4.1. Plataformas para navegación autónoma

Los métodos de planificación y control descritos no se ejecutan de forma aislada, sino a través de una plataforma que los integra y coordina. Para esto existen distintas alternativas, que van desde *frameworks* de navegación ya consolidados hasta arquitecturas desarrolladas de forma específica. Por su parte, Nav2 es el *framework* de navegación de referencia en ROS 2. Su estructura modular permite combinar planificadores como NavFn y Smac con controladores como DWB, Regulated Pure Pursuit y MPPI, y orquestar la navegación mediante árboles de comportamiento (Nav2 Maintainers, 2025). Su predecesor en ROS 1, `move_base`, ha cumplido un rol equivalente durante años, pero está ligado a una distribución que ya no recibe soporte (Open Robotics, 2025b). En un ámbito distinto, Autoware se origina como una pila de navegación orientada a vehículos autónomos (Kato et al., 2018). La documentación oficial actual lo describe como una pila integral de conducción autónoma que cubre desde el sensado hasta el control (Autoware Foundation, 2026). Por ello, su ámbito de aplicación y sus requerimientos de percepción y cómputo siguen siendo distintos a los de una plataforma de interiores como la desarrollada en este trabajo. También existen soluciones propietarias integradas en robots comerciales, cuyo funcionamiento interno no siempre es accesible.

Como alternativa a estos *frameworks*, es posible desarrollar una arquitectura propia, apoyándose en bibliotecas de planificación de movimiento como OMPL (Open Motion Planning Library Project, 2026; Şucan et al., 2012) o integrando los componentes mediante otros *middleware* robóticos. Los algoritmos de planificación y control pueden ser equivalentes, pero cambian las interfaces, las herramientas de supervisión y los mecanismos de comunicación. En términos generales, un *framework* consolidado reduce el esfuerzo de integración y aporta componentes ya probados, mientras que una arquitectura propia ofrece mayor control a cambio de un mayor esfuerzo de desarrollo.

1.4.2. Exploración autónoma

Las estrategias de exploración pueden agruparse en enfoques basados en fronteras, métodos guiados por ganancia de información y propuestas de *active SLAM*. La exploración basada en fronteras, propuesta originalmente por Yamauchi (1997), utiliza como candidatos las regiones que separan el espacio conocido de las zonas todavía no observadas. Herramientas como `explore_lite` representan ejemplos de este enfoque, incluido su *port* a ROS 2 (Alvarez, 2021; Hörner, 2017). Su principal ventaja es la facilidad de integración con mapas de ocupación y sistemas de navegación ya existentes.

Los métodos basados en ganancia de información seleccionan metas estimando cuánto conocimiento adicional podría obtenerse desde cada posición. Estos métodos pueden considerar visibilidad, superficie esperada o reducción de incertidumbre, pero requieren modelos y cálculos adicionales.

Las estrategias de *active SLAM* incorporan simultáneamente la cobertura del entorno, la incertidumbre de localización y la consistencia del mapa. En desarrollos recientes también se investigan métodos basados en muestreo, aprendizaje y coordinación de múltiples robots (Placed et al., 2023). Los enfoques basados en fronteras continúan siendo utilizados por su menor complejidad y facilidad de integración. En cambio, los métodos de ganancia de información y *active SLAM* pueden generar decisiones más informadas, aunque demandan una representación más completa del estado del sistema y una mayor capacidad de procesamiento.

1.5. Simulación de robots móviles

Entre las alternativas disponibles se encuentran simuladores de propósito general, como Gazebo Classic, Webots y CoppeliaSim, además de plataformas orientadas a una mayor fidelidad visual y física, como Isaac Sim. Estas herramien-

tas difieren en la precisión de sus modelos físicos, la representación de sensores, la integración con plataformas robóticas, los recursos computacionales requeridos y la facilidad para construir escenarios de prueba. Gazebo Classic permite simular robots y entornos tridimensionales mediante motores de física, sensores y extensiones basadas en *plugins* (Open Source Robotics Foundation, 2025). El Gazebo actual corresponde a la línea sucesora mantenida por el proyecto y se distribuye separadamente de Gazebo Classic (Open Robotics, 2024). Webots y CoppeliaSim, presentados originalmente en Michel (2004) y Rohmer et al. (2013), continúan disponibles como simuladores robóticos con modelado de robots y entornos tridimensionales, de acuerdo con su documentación oficial reciente (Coppelia Robotics AG, 2025; Cyberbotics Ltd., 2025). Isaac Sim incorpora capacidades avanzadas de representación gráfica, generación de datos sintéticos y simulación de sensores, aunque requiere una mayor capacidad de procesamiento (NVIDIA Corporation, 2025).

Conviene precisar la nomenclatura, porque el nombre Gazebo designa actualmente a dos programas distintos. La familia de versiones numeradas, cuya última entrega es la 11, se denomina ahora Gazebo Classic y se encuentra fuera de soporte desde enero de 2025 (Open Source Robotics Foundation, 2025). Su sucesor, desarrollado inicialmente bajo el nombre Ignition y con versiones identificadas por nombre en lugar de número, es el que hoy se distribuye simplemente como Gazebo (Open Robotics, 2025a).

La selección de un simulador depende del propósito de la prueba. Para validar la cinemática, los marcos de referencia y la comunicación entre componentes, puede ser suficiente un entorno de complejidad moderada. En cambio, las aplicaciones que requieren percepción visual, interacción física detallada o entrenamiento de sistemas de aprendizaje pueden necesitar simuladores con mayor fidelidad.

Conclusión del capítulo

La revisión muestra que las soluciones actuales para robótica móvil se orientan hacia arquitecturas modulares basadas en ROS 2, percepción y mapeo del entorno, planificación y exploración autónomas, y simuladores de mayor fidelidad. Estas alternativas amplían las capacidades del sistema, pero suelen aumentar el costo, el consumo de recursos y la complejidad de implementación. La selección concreta de tecnologías para el prototipo, que busca un equilibrio entre estas capacidades y las restricciones de una plataforma de bajo costo, se presenta en el Capítulo 2.

CAPÍTULO 2

SOLUCIÓN IMPLEMENTADA

Este capítulo describe, de forma general, la solución diseñada antes de entrar al detalle teórico y de implementación. A partir de las alternativas revisadas en el Capítulo 1, se propone un robot móvil diferencial de bajo costo que integra percepción láser 2D, odometría por *encoders* y control PI incremental embebido, una forma clásica del controlador PID, cuyas ganancias se ajustan en el robot a partir de un punto de partida obtenido con optimización por enjambre de partículas (PSO), sobre una arquitectura distribuida en dos entornos: robot y estación de trabajo, comunicados mediante ROS 2. El objetivo es que el sistema sea capaz de mapear, navegar y explorar entornos interiores de forma autónoma. La Fig. 2.1 presenta el diagrama de bloques simplificado del sistema, organizado en cuatro dominios funcionales identificados por color.

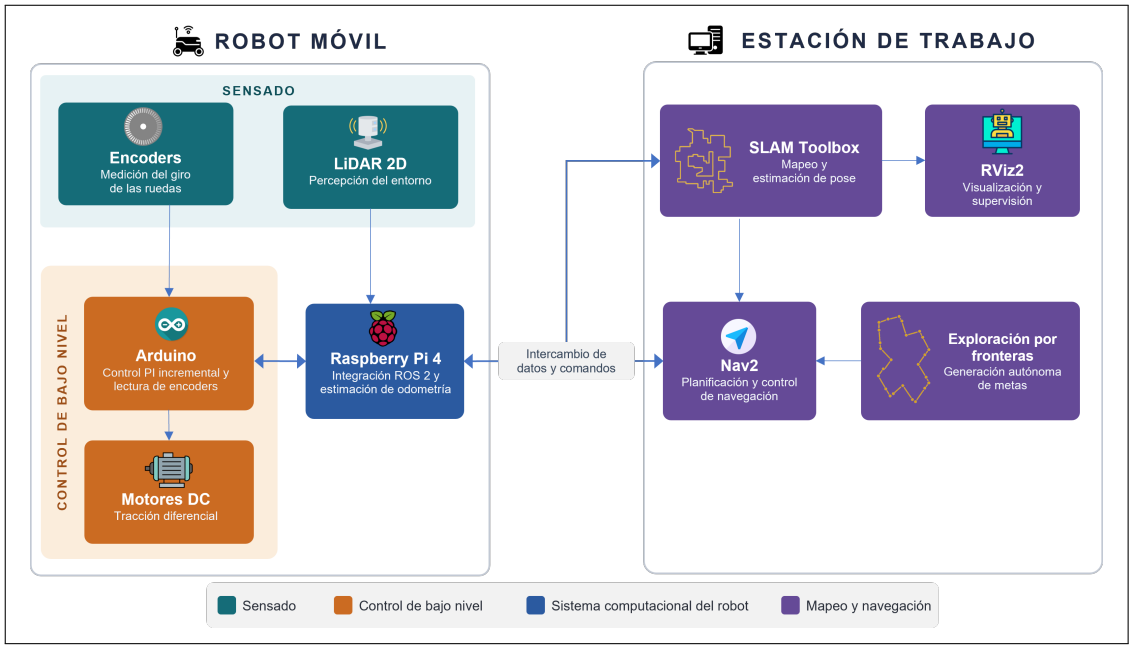


Figura 2.1: Diagrama de bloques simplificado de la solución implementada.

Fuente: Elaboración propia.

Las flechas del diagrama representan el flujo lógico de mediciones, estados, metas y comandos, y no todas corresponden a conexiones físicas directas, ya que parte del intercambio se realiza mediante tópicos, acciones y transformaciones de ROS 2 a través de la red local. La función general de cada dominio es la siguiente:

Sensado. Los *encoders* de cuadratura miden el giro de cada rueda y el LIDAR 2D percibe el entorno; son las dos fuentes de información del sistema.

Control de bajo nivel. El Arduino lee los *encoders* y ejecuta el control de velocidad de cada rueda, un PI incremental que acciona los motores DC de tracción diferencial. Las ganancias de ese controlador se determinan en dos etapas: PSO entrega un primer conjunto a partir de un modelo identificado de cada rueda, y ese conjunto se refina luego sobre el robot real hasta llegar a los valores definitivos.

Sistema computacional del robot. La Raspberry Pi 4 integra ROS 2, estima la odometría a partir de las velocidades de rueda y actúa de puente entre el *hardware* embebido y la estación de trabajo.

Mapeo, navegación y exploración. En la estación de trabajo, `slam_toolbox` construye el mapa y estima la pose, Nav2 planifica y controla la navegación, el explorador de fronteras genera metas autónomas y RViz2 permite la supervisión.

2.1. Selección de tecnologías

Cada dominio del sistema se resuelve eligiendo, entre las alternativas comparadas en el Capítulo 1, la más adecuada para una plataforma diferencial de bajo costo orientada a interiores. Se adopta ROS 2 como *middleware* por su arquitectura distribuida, sin un nodo maestro central, y por su ecosistema de paquetes de mapeo, navegación y exploración (Sec. 1.2) (Macenski et al., 2022). El fin de soporte de ROS 1 (Open Robotics, 2025b) no se usa como criterio de selección,

porque la distribución adoptada, ROS 2 Foxy, también se encuentra fuera de soporte. Esa distribución se mantiene por la compatibilidad de la cadena de paquetes con el *hardware* disponible, como se declara en Alcances y limitaciones. La percepción se resuelve con un LIDAR 2D, suficiente para interiores y de menor costo y carga de procesamiento que las cámaras RGB-D o el LIDAR 3D, y el mapeo con `slam_toolbox`, por su integración nativa, su enfoque basado en grafos de poses y su capacidad de guardar y cargar mapas (Sec. 1.3) (Macenski & Jambrecic, 2021). La navegación se apoya en Nav2, con un planificador global de búsqueda sobre rejilla y un controlador local reactivo, adecuados a un mapa de ocupación 2D y de bajo costo computacional (Sec. 1.4) (Macenski et al., 2023).

La validación previa del diseño se apoya en un gemelo digital construido en Gazebo Classic. Se opta por esa versión y no por el Gazebo actual porque es la que emplean los complementos de simulación y de `ros2_control` del paquete base adaptado, disponibles para ROS 2 Foxy sobre Ubuntu 20.04 (Sec. 1.5).

La sintonización del control de bajo nivel también se resuelve como una decisión de diseño. En lugar de buscar las tres ganancias del controlador por prueba y error desde cero, se identifica un modelo de primer orden con retardo para cada rueda y sobre él se aplica PSO, un método de optimización que no requiere derivadas ni un modelo exacto de la planta y que permite buscar las tres ganancias de forma simultánea. El resultado de esa búsqueda se adopta como punto de partida y no como valor final: las ganancias definitivas se alcanzan ajustando ese punto de partida sobre el robot real, donde intervienen efectos que el modelo no recoge, como la zona muerta del *driver*, la cuantización de los *encoders* y la asimetría entre ambas ruedas.

Para la exploración autónoma se elige el enfoque basado en fronteras (Sec. 1.4), por su menor complejidad y su fácil integración con el mapa de ocupación y con Nav2, frente a los métodos de ganancia de información y de *active SLAM*, que exigen modelos y cómputo adicionales (Alvarez, 2021; Placed et al., 2023). El fundamento teórico de cada uno de estos componentes se desarrolla en el Capítulo 3, y su implementación concreta en el Capítulo 4.

CAPÍTULO 3

MARCO TEÓRICO

Este capítulo presenta los fundamentos generales empleados en el diseño del robot móvil diferencial. La exposición avanza desde el robot como sistema físico (estructura, marcos de referencia y descripción digital) hacia la plataforma de *software* que lo integra, los principios de sensado y actuación, el modelo cinemático y el control de bajo nivel, y finalmente las capas de mapeo, localización, navegación y exploración autónoma.

3.1. El robot móvil: estructura, marcos de referencia y rotaciones

Un robot móvil puede describirse como un conjunto de cuerpos rígidos (eslabones) unidos mediante articulaciones, organizados como un árbol cinemático cuya raíz es la base del robot (Corke, 2023). En una plataforma diferencial, el árbol se compone del chasis, las dos ruedas motrices (articulaciones de revolución accionadas) y los elementos pasivos de apoyo, además de los sensores montados sobre el chasis. Para describir la posición y orientación del robot se asocia un marco de referencia a cada cuerpo de interés. La pose de un robot que se desplaza en el plano queda definida por el vector $q = (x, y, \theta)$, donde x e y son las coordenadas de la base respecto de un marco fijo (m) y θ la orientación (rad). La relación entre un punto expresado en el marco del robot y el mismo punto en el marco fijo se obtiene mediante la matriz de rotación en el plano de la Ec. (3.1):

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \tag{3.1}$$

que, combinada con la traslación (x, y), forma una transformación homogénea. Encadenando transformaciones se relaciona cualquier par de marcos del robot; por ejemplo, la pose del sensor láser respecto del marco global se obtiene componiendo la transformación del mapa a la base del robot con la transformación fija de la base al sensor (Corke, 2023). En robótica móvil es habitual organizar estos marcos en una jerarquía que separa el marco global del mapa, el marco odométrico (continuo pero con deriva) y el marco de la base del robot; las convenciones de orientación de ejes (eje x hacia adelante, y hacia la izquierda, z hacia arriba) y de organización de marcos siguen las recomendaciones REP-103 y REP-105 (Foote & Purvis, 2010; Meeussen, 2010).

3.1.1. Descripción del modelo mediante URDF

El formato URDF (*Unified Robot Description Format*) describe al robot como un árbol de eslabones y articulaciones. Por cada eslabón se declaran su geometría visual, su geometría de colisión y sus propiedades inerciales, y por cada articulación su tipo, sus límites y los marcos que conecta, además de los sensores y las extensiones de simulación. La misma descripción alimenta la visualización, la publicación de transformaciones y el simulador, de modo que el robot real y su modelo digital comparten una única definición geométrica, parametrizada de forma que dimensiones como el radio de rueda se definan una sola vez.

Esa parametrización se consigue con `xacro`, un lenguaje de macros para XML que permite definir constantes, reutilizar bloques y dividir la descripción en módulos.

El Alg. 3.1 resume la organización modular de la descripción URDF/`xacro` desarrollada, en la que un archivo raíz compone el modelo a partir de módulos

reutilizables que se seleccionan según el entorno de ejecución (*hardware* real o simulación).

Algoritmo 3.1 Construcción del modelo URDF/`xacro` del robot.

Require: `sim_mode`, `use_ros2_control`
procedure DefinirRobot ▷ `robot.urdf.xacro`
  incluir `robot_core.xacro` y `inertial_macros.xacro`
  $\mathcal{L} \leftarrow$ {`base_link`, `chassis`, `left_wheel`, `right_wheel`, `caster_wheel`}
  for cada eslabón $\ell \in \mathcal{L}$ **do**
    definir geometría visual, colisión e inercia de ℓ
  end for
  for cada articulación a (fija o continua) entre eslabones de $\mathcal{L}$ **do**
    conectar eslabón padre e hijo de a y fijar su marco
  end for
  definir `laser_frame` y su articulación fija al chasis
  if `sim_mode` **then**
    incluir `lidar_sim.xacro`, `gazebo_control.xacro`
  else
    incluir `lidar.xacro`, `ros2_control.xacro`
  end if
  if `use_ros2_control` **then**
    declarar interfaces de las ruedas: comando (velocidad) y estado (velocidad y posición)
  end if
  return modelo del robot
end procedure

3.1.2. Simulación robótica y Gazebo Classic

Un simulador robótico integra un motor de física, la simulación de sensores y un entorno tridimensional configurable, lo que permite verificar la geometría, la cinemática y la lógica de control antes de operar el *hardware*. Gazebo Classic, el simulador empleado, se integra de forma nativa con ROS 2 mediante los paquetes de puente del ecosistema: el modelo URDF se carga en el mundo simulado y los sensores publican datos con las mismas interfaces que sus contrapartes físicas (Open Source Robotics Foundation, 2025). La brecha simulación–realidad, es decir fricción, zonas muertas, ruido sensorial y latencias, se considera al interpretar

los resultados del Capítulo 5.

3.2. ROS 2: nodos, tópicos y comunicación

ROS 2 es un conjunto de bibliotecas y herramientas de código abierto para construir sistemas robóticos de forma modular y distribuida (Macenski et al., 2022). Su unidad básica es el *nodo*, un proceso que encapsula una función específica, y la comunicación principal ocurre mediante *tópicos*, canales con nombre en los que unos nodos publican mensajes y otros se suscriben para recibirlos. Cada nodo conoce solo el nombre y el tipo del tópico, lo que permite reemplazar un componente sin modificar el resto del sistema. La Fig. 3.1 ilustra este patrón.

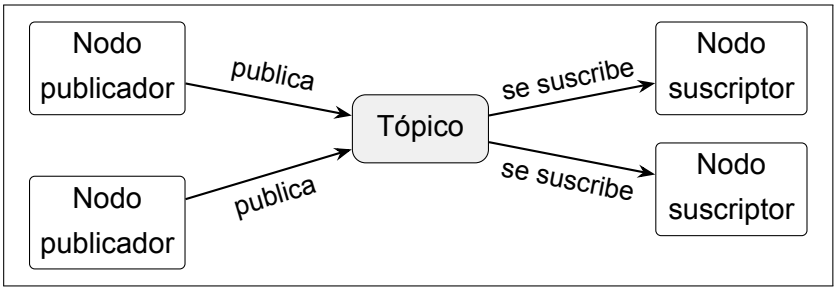


Figura 3.1: Patrón de comunicación publicación–suscripción: los nodos publicadores envían mensajes a un tópico y los suscriptores los reciben, de forma anónima y desacoplada.

Fuente: Elaboración propia.

El transporte se apoya en el estándar DDS (*Data Distribution Service*), que descubre los nodos de la red y negocia políticas de calidad de servicio (Macenski et al., 2022). Gracias a ello los nodos pueden repartirse entre varios computadores sin un maestro central, como ocurre aquí entre la Raspberry Pi 4 y la estación de trabajo. ROS 2 ofrece además servicios y acciones, estas últimas usadas para enviar metas de navegación. El árbol de transformaciones mantiene las relaciones espaciales entre los marcos de la Sec. 3.1.

3.3. Principio de funcionamiento de sensores y actuadores

3.3.1. Sensor láser LIDAR 2D

Un LIDAR mide distancias emitiendo pulsos de luz láser y midiendo el tiempo que tarda en volver la señal reflejada (Corke, 2023, Sec. 6.8). Los LIDAR 2D también pueden medir por triangulación óptica (Ran et al., 2025). En un LIDAR 2D, el emisor y el receptor giran sobre un eje vertical y entregan, en cada revolución, un barrido de pares ángulo y distancia en un plano horizontal, con el que se detectan paredes y obstáculos y se alimentan el SLAM y los mapas de costos (Corke, 2023). Sus limitaciones principales son la incapacidad de detectar objetos fuera del plano de barrido y la degradación ante superficies muy absorbentes o reflectantes. El sensor empleado, RPLidar A1, opera por triangulación y cubre 360° (Shanghai Slamtec Co., Ltd., 2020).

3.3.2. *Encoders* incrementales de cuadratura

Un *encoder* incremental convierte la rotación de un eje en una secuencia de pulsos eléctricos. En los *encoders* magnéticos de efecto Hall, un imán multipolar solidario al eje del motor induce pulsos en dos sensores desfasados, generando dos canales en cuadratura (desfasados 90°). El orden de los flancos entre ambos canales indica el sentido de giro, y contar los flancos de ambos canales cuadruplica la resolución básica del sensor (decodificación $\times 4$). La Fig. 3.2 muestra ambos canales, el desfase entre ellos y los flancos contabilizados.

Cuando el *encoder* está montado en el eje del motor y este acciona la rueda a través de una caja reductora, cada vuelta de la rueda corresponde a N_g vueltas del motor, por lo que la resolución teórica en el eje de salida es la que indica la

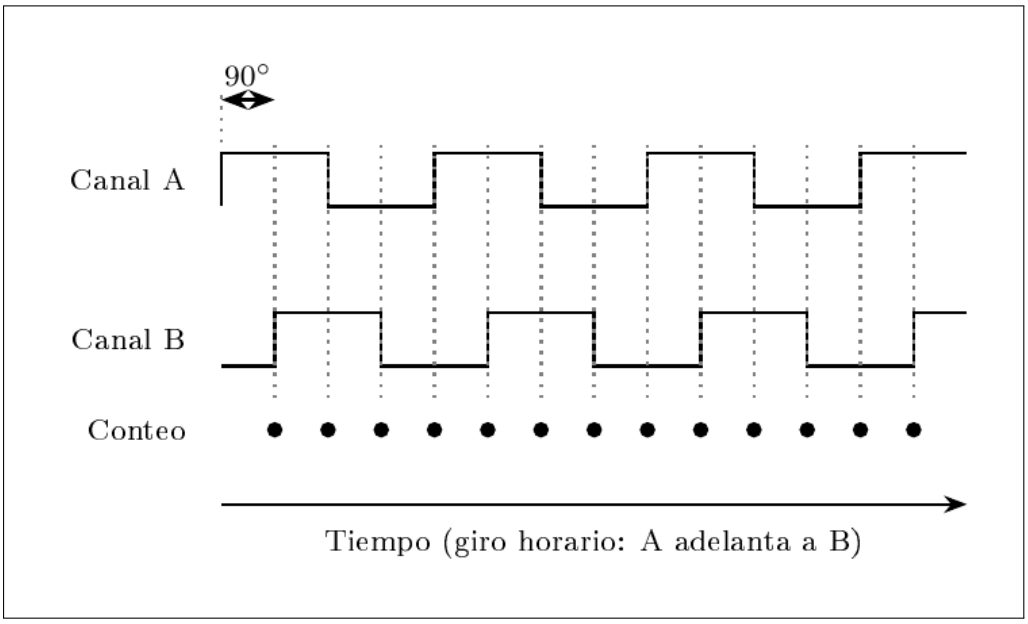


Figura 3.2: Canales en cuadratura de un *encoder* incremental y decodificación $\times 4$.

Fuente: Elaboración propia.

Ec. (3.2):

$$N_e^{teo} = N_{Hall} \cdot c_q \cdot N_g, \tag{3.2}$$

donde N_e^{teo} es la resolución teórica del *encoder* [*ticks*/vuelta del eje de salida], N_{Hall} el número de pulsos por vuelta del eje del motor, $c_q \in \{1, 2, 4\}$ el factor de decodificación en cuadratura y N_g la relación de reducción. En la práctica, tolerancias mecánicas y pérdidas de conteo hacen que la resolución efectiva N_e pueda diferir del valor teórico, por lo que es habitual determinarla experimentalmente. Esta resolución es la que relaciona el conteo acumulado de *ticks* con la posición y la velocidad angular de cada rueda mediante las Ecs. (3.9) y (3.10), base de la odometría desarrollada en la Sec. 3.4; los valores numéricos del prototipo se consolidan en el Capítulo 4.

3.3.3. Motores DC y *driver* de potencia

Un motor DC de imanes permanentes gira con velocidad aproximadamente proporcional a la tensión media aplicada y entrega un torque proporcional a la corriente. Como un microcontrolador no puede suministrar esa potencia, se interpone un *driver* basado en un puente H, que aplica la tensión de la batería al motor en ambas polaridades y regula la magnitud de la velocidad mediante PWM. Los *drivers* reales introducen dos efectos relevantes para el control: una caída de tensión interna y una zona muerta a bajos ciclos de trabajo, en la cual el motor no vence la fricción estática y no gira.

3.4. Cinemática diferencial y odometría

La odometría estima la pose del robot integrando el desplazamiento medido de sus ruedas (Ferreira et al., 2023). Su cálculo es sencillo, lo que la hace adecuada para sistemas embebidos. Sin embargo, acumula deriva por errores sistemáticos, como un radio de rueda mal calibrado, y por errores aleatorios, como el deslizamiento entre la rueda y el suelo (Corke, 2023, Sec. 6.1).

Convención. v y ω sin subíndice denotan la velocidad lineal (m/s) y angular (rad/s) del robot; con subíndice $j \in \{R, L\}$ denotan las magnitudes correspondientes de la rueda derecha o izquierda. La posición angular de rueda se denota φ_j (rad) y el conteo de *encoder* se expresa en *ticks*. El listado completo de símbolos, subíndices y unidades empleados en el documento se reúne en el ANEXO 10.

Considérese un robot diferencial con radio de rueda r, separación entre ruedas b y la forma muestreada $q_k = (x_k, y_k, \theta_k)$ de la pose definida en la Sec. 3.1, donde k identifica la muestra discreta. La relación entre las velocidades angulares de rueda y las velocidades del robot es la del modelo cinemático diferencial estándar, expresada en las Ecs. (3.3) y (3.4) (Corke, 2023; Ferreira et al., 2023):

$$v[k] = \frac{r}{2}\left(\omega_R[k] + \omega_L[k]\right), \tag{3.3}$$

$$\omega[k] = \frac{r}{b}\left(\omega_R[k] - \omega_L[k]\right). \tag{3.4}$$

La relación inversa, utilizada para transformar una consigna del robot en consignas de rueda, es la de la Ec. (3.5):

$$\begin{aligned} \omega_R^{cmd}[k] &= \frac{v^{cmd}[k]}{r} + \frac{b}{2r}\,\omega^{cmd}[k], \\ \omega_L^{cmd}[k] &= \frac{v^{cmd}[k]}{r} - \frac{b}{2r}\,\omega^{cmd}[k]. \end{aligned} \tag{3.5}$$

donde v^{cmd} y ω^{cmd} son las consignas lineal (m/s) y angular (rad/s) del robot, mientras que ω_R^{cmd} y ω_L^{cmd} son las consignas angulares de las ruedas (rad/s). Además, una discretización de Euler del modelo cinemático permite propagar la pose entre muestras mediante las Ecs. (3.6), (3.7) y (3.8):

$$x_{k+1} = x_k + T_k\, v[k] \cos\theta_k, \tag{3.6}$$

$$y_{k+1} = y_k + T_k\, v[k] \sin\theta_k, \tag{3.7}$$

$$\theta_{k+1} = \theta_k + T_k\, \omega[k]. \tag{3.8}$$

donde T_k es el intervalo temporal entre las muestras k y $k+1$ (s). Estas ecuaciones expresan la integración odométrica del modelo cinemático diferencial general (Ferreira et al., 2023). El integrador odométrico efectivamente ejecutado pertenece a la capa de control descrita en el Capítulo 4.

3.4.1. Estimación de estados de rueda a partir de *encoders*

A partir del conteo acumulado de *encoder* $c_j[k]$ (*ticks*), la posición angular de cada rueda se obtiene mediante:

$$\varphi_j[k] = \frac{2\pi}{N_e} c_j[k], \qquad j \in \{R, L\}, \tag{3.9}$$

donde N_e es la resolución efectiva del *encoder* (*ticks*/vuelta), definida en la Sec. 3.3.2. La velocidad angular se estima por diferencias finitas entre dos lecturas consecutivas:

$$\widehat{\omega}_j[k] = \frac{\varphi_j[k] - \varphi_j[k-1]}{\Delta t_k}, \qquad j \in \{R, L\}. \tag{3.10}$$

donde $\widehat{\omega}_j[k]$ es la velocidad angular estimada de la rueda j (rad/s) y Δt_k el tiempo real transcurrido entre las lecturas empleadas en segundos. La definición por casos de Δt_k y la secuencia de lectura ejecutada por la interfaz de *hardware* se desarrollan en el Capítulo 4 y en el ANEXO 3.5.

3.5. Identificación FOPDT

Para obtener un modelo destinado a la sintonización del controlador se identifica, para cada rueda, una estructura de primer orden con retardo FOPDT (*First Order Plus Dead Time*) a partir de la respuesta experimental ante un escalón de PWM. La Ec. (3.11) presenta esa estructura (Maxim & De Keyser, 2023):

$$G_j(S) = \frac{K_{m,j}}{1 + \tau_j S} e^{-L_{d,j} S}, \qquad j \in \{R, L\}. \tag{3.11}$$

donde $G_j(S)$ es la función de transferencia identificada para la rueda j, $K_{m,j}$ la ganancia estática [(m/s)/PWM], τ_j la constante de tiempo (s) y $L_{d,j}$ el retardo puro o tiempo muerto (s). La variable compleja de Laplace se denota con mayúscula, S, para distinguirla a simple vista del segundo (s), unidad de tiempo escrita en redonda; los productos $\tau_j S$ y $L_{d,j}\,S$ son adimensionales. Los parámetros (K_m, τ, L_d) se estiman minimizando el error cuadrático entre la respuesta medida y la respuesta del modelo. Como el retardo se estima en múltiplos del período de captura (Maxim & De Keyser, 2023, Sec. 2.4), un valor menor que ese período solo indica que el retardo es menor que él. La justificación del modelo, la discretización, la cuantización del retardo y las aproximaciones numéricas empleadas por los *scripts* se documentan en el ANEXO 4.6.

3.6. Control PID

El control de velocidad de cada rueda se organiza como un lazo cerrado: la velocidad medida por el *encoder* se compara con la referencia, y la diferencia alimenta al controlador, que ajusta la tensión media entregada al motor. La Fig. 3.3 resume esta estructura.

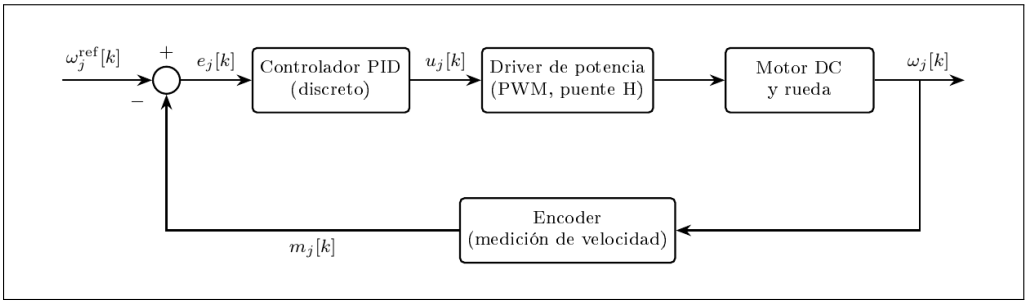


Figura 3.3: Lazo de control de velocidad de una rueda con controlador PID.
Fuente: Elaboración propia.

El controlador de velocidad se ejecuta de forma independiente para cada rueda en el Arduino Nano. La forma continua del PID de la Ec. (3.12), tomada de la formulación clásica de control realimentado (Åström & Hägglund, 2006/2009, Sec. 3.2), solo se utiliza como referencia conceptual:

$$u_j(t) = K_{p,j}^{cont} e_j(t) + K_{i,j}^{cont} \int_0^t e_j(\sigma)\, d\sigma + K_{d,j}^{cont} \frac{de_j(t)}{dt}. \quad (3.12)$$

donde t es el tiempo continuo, $u_j(t)$ la acción de control, $e_j(t)$ el error de velocidad, $K_{p,j}^{cont}$, $K_{i,j}^{cont}$ y $K_{d,j}^{cont}$ las ganancias proporcional, integral y derivativa de la forma continua, respectivamente, y σ la variable de integración.

El controlador no se evalúa de forma continua, sino a intervalos regulares en el microcontrolador, y puede organizarse en forma incremental, en que cada muestra calcula el cambio de la señal de control y lo suma a la salida anterior (Åström & Hägglund, 2006/2009, Sec. 13.4). El controlador que se ejecuta en este trabajo tiene esa forma y no usa acción derivativa, por lo que corresponde a un PI incremental. Su formulación discreta específica, la realización en el *firmware* y las ganancias definitivas se desarrollan en la Sec. 4.4.3 y en el ANEXO 4.4, y la forma incremental del PID y los criterios de estabilidad del lazo muestreado, en el ANEXO 4.12.

3.7. Sintonización del controlador PID mediante PSO

Las ganancias del controlador se obtienen mediante un procedimiento híbrido: identificación FOPDT por rueda (Sec. 3.5), búsqueda de ganancias iniciales mediante optimización por enjambre de partículas y refinamiento experimental posterior sobre el *hardware*. Como referencia analítica complementaria se emplea además la sintonía Lambda, cuyo desarrollo y condiciones de comparación se documentan en el ANEXO 4.2.

3.7.1. Optimización por enjambre de partículas

PSO (*Particle Swarm Optimization*) es una metaheurística poblacional inspirada en el comportamiento colectivo de enjambres. En este trabajo, Gökçe et al. (2021) se utiliza como antecedente de la aplicación de PSO a la sintonización de controladores PID en robótica móvil, pero no como fuente de la ecuación específica implementada. El *script* empleado combina el factor de constricción analizado por Clerc y Kennedy (2002) con un peso de inercia, mecanismo introducido en PSO por Shi y Eberhart (1998). La combinación concreta utilizada aquí, incluido el factor global de constricción y la ley lineal del peso de inercia, corresponde a la implementación ejecutada en este trabajo.

Cada partícula i representa un candidato de solución, en este caso, un conjunto de ganancias del controlador, mediante una posición $\xi_{i,d}$ y una velocidad $V_{i,d}$, ambas adimensionales, con $d \in \{1, 2, 3\}$ asociado a las tres ganancias. En cada generación g, la velocidad de cada partícula se actualiza combinando la velocidad previa, la atracción hacia su mejor posición personal y la atracción hacia la mejor posición global del enjambre:

$$V_{i,d}^{g+1} = \zeta \left[w_{in}^{g} V_{i,d}^{g} + c_1 \rho_{1,i,d}^{g} \left(P_{i,d} - \xi_{i,d}^{g} \right) + c_2 \rho_{2,i,d}^{g} \left(G_{d}^{g} - \xi_{i,d}^{g} \right) \right], \tag{3.13}$$

donde $P_{i,d}$ es la mejor posición personal almacenada, G_d^g la componente del mejor global, w_{in}^g el peso de inercia, c_1 y c_2 los coeficientes cognitivo y social, $\rho_{1,i,d}^g$ y $\rho_{2,i,d}^g$ números aleatorios uniformes en $[0, 1]$ y ζ el factor de constricción. Los valores utilizados para c_1, c_2 y ζ se consolidan en la Tabla A4.3. El empleo conjunto de constricción y peso de inercia en esta ecuación se documenta como una variante implementada en el proyecto y no se atribuye literalmente a una sola de las fuentes citadas.

El peso de inercia decrece linealmente con las generaciones:

$$w_{in}^{g} = \frac{g_{máx} - g}{g_{máx}}, \qquad g = 1, \ldots, g_{máx}, \tag{3.14}$$

donde $g_{máx}$ es el número máximo de generaciones. La disminución lineal del peso de inercia la proponen Shi y Eberhart (1998), que lo reducen desde 1.4 hasta 0. El *script* usa esa misma forma, pero partiendo de 1. Su efecto es reducir progresivamente el peso de la velocidad previa a medida que avanza la búsqueda. La función de costo empleada para la sintonización combina el error cuadrático medio de la velocidad respecto de la referencia con una penalización por sobreimpulso, favoreciendo respuestas rápidas pero amortiguadas. Como toda metaheurística, PSO no garantiza encontrar el óptimo global, y además su costo se evalúa sobre un modelo simplificado de cada rueda. Por eso, las ganancias que entrega se usan como punto de partida y se ajustan después sobre el robot real. Las escalas de búsqueda, las cotas, la configuración del enjambre y las particularidades numéricas de la ejecución se presentan en el Capítulo 4 y en el ANEXO 4.8.

3.8. SLAM basado en grafos

La odometría pura acumula deriva, por lo que se requiere SLAM para estimar simultáneamente la trayectoria del robot y el mapa del entorno (Ngoc et al., 2023; Thrun et al., 2005). En el enfoque basado en grafos, cada nodo del grafo representa una pose del robot y cada arista una restricción relativa entre dos poses, obtenida de la odometría o del ajuste de barridos láser (*scan matching*), que alinea geométricamente dos barridos consecutivos para estimar el desplazamiento entre ellos. Cuando el robot vuelve a observar una zona ya visitada se agrega una restricción de *cierre de lazo*, y la optimización del grafo, un problema de mínimos cuadrados no lineales, redistribuye el error acumulado a lo largo de toda la trayectoria, corrigiendo el mapa y la transformación entre el marco del mapa y el marco odométrico. La Fig. 3.4 representa esta estructura.

El procesamiento asíncrono de los barridos y la detección de cierre de lazo

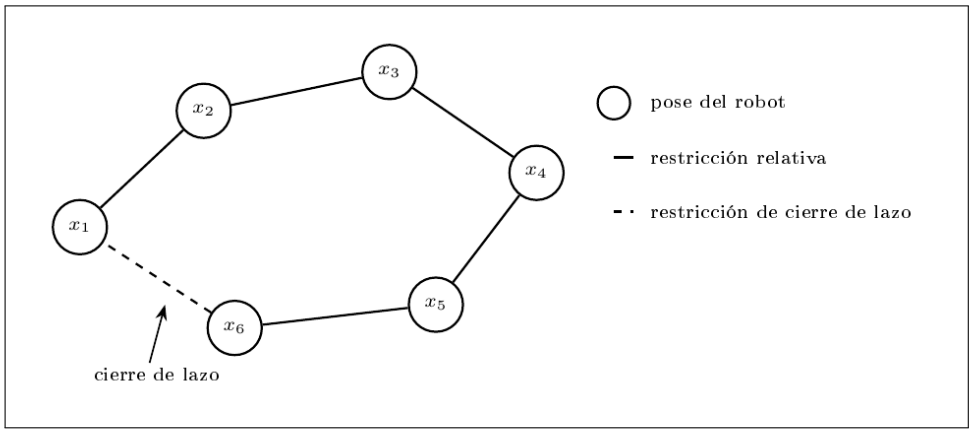


Figura 3.4: Grafo de poses: nodos, restricciones relativas y cierre de lazo.
Fuente: Elaboración propia.

son características que hacen viable esta técnica en plataformas de bajo costo. La técnica utilizada en este trabajo es SLAM Toolbox, una realización del SLAM basado en grafos de poses que construye el mapa de ocupación de forma incremental durante la operación, mantiene y optimiza el grafo mientras el robot se desplaza, y permite almacenar el mapa resultante para reutilizarlo después en la localización (Macenski & Jambrecic, 2021). La comparación con las demás alternativas de SLAM 2D se presenta en el Capítulo 1, y la configuración concreta empleada en la Sec. 4.5.1.

3.9. Localización en mapas conocidos

Una vez construido y almacenado el mapa del entorno, la estimación de la pose puede realizarse sin volver a ejecutar el proceso de mapeo. Para este propósito se emplea la localización de Monte Carlo adaptativa, un método de localización probabilística basado en un filtro de partículas (Thrun et al., 2005). Cada partícula representa una hipótesis de pose. Las hipótesis se propagan con la odometría y se ponderan según la concordancia entre los barridos del LIDAR y el mapa de ocupación almacenado, y luego el remuestreo concentra las partículas en las regiones de mayor probabilidad. Este método estima la transformación

entre el marco del mapa y el odométrico, mientras que la odometría mantiene la transformación entre este último y la base del robot. A diferencia del SLAM de la Sec. 3.8, la localización no construye ni modifica el mapa, sino que sitúa al robot dentro de uno ya conocido. El uso de este método en la pila de navegación empleada y su relación con el modo de localización utilizado en este trabajo se precisan en la Sec. 4.5.2.

3.10. Planificación de rutas y control local

La navegación autónoma opera sobre una representación del entorno en forma de rejilla de ocupación, donde cada celda clasifica el espacio como libre, ocupado o desconocido (Macenski et al., 2023, Sec. V). Sobre esa rejilla se construyen mapas de costo que agregan a los obstáculos una zona de inflación que penaliza transitar cerca de ellos (Nav2 Maintainers, 2025). Sobre esta representación, el planificador global calcula una ruta desde la posición actual hasta la meta. En este trabajo la búsqueda global se resuelve mediante el algoritmo A* sobre la rejilla derivada del mapa de costos. El planificador empleado y la opción de configuración que habilita esta búsqueda se presentan en el Capítulo 4. A* ordena la expansión de los nodos mediante la función de evaluación de la Ec. (3.15) (LaValle, 2006, Sec. 2.2.2):

$$f(n) = g(n) + h(n), \tag{3.15}$$

donde n es el nodo evaluado, g(n) representa el costo acumulado desde el origen hasta dicho nodo y h(n) una estimación del costo restante hasta la meta. La implementación de Nav2 incorpora además los costos de su propia representación, cuyo escalamiento interno no se reconstruye en este trabajo. Por eso no se emplean la admisibilidad de la heurística ni la optimalidad de la ruta como criterio de validación, sino la capacidad de generar una ruta global que el controlador local pueda seguir.

El control local convierte la ruta global en comandos de velocidad ejecutables. El enfoque de ventana dinámica considera únicamente las velocidades (v, ω) alcanzables desde el estado actual dado el límite de aceleración del robot, simula hacia adelante las trayectorias asociadas a cada candidato y las evalúa mediante una suma ponderada de criterios (Macenski et al., 2023, Sec. III-A):

$$J_{DWB}(v, \omega) = \sum_{c \in \mathcal{C}} S_c \, C_c(v, \omega), \tag{3.16}$$

donde $J_{DWB}(v, \omega)$ es el puntaje total de una trayectoria candidata con velocidades lineal v y angular ω; $\mathcal{C}$ es el conjunto de criterios de evaluación, tales como proximidad a obstáculos, alineación con la ruta y avance hacia la meta, $C_c(v, \omega)$ el costo asignado por el criterio c y S_c su peso. Se selecciona el candidato de menor puntaje, lo que permite seguir la ruta global mientras se reacciona ante obstáculos cercanos. La navegación de alto nivel se orquesta mediante un árbol de comportamiento: una estructura modular que encadena la planificación y el seguimiento, y que ante fallos activa acciones de recuperación de complejidad creciente sin modificar la lógica del resto del sistema. Los componentes concretos, sus criterios configurados y las acciones de recuperación se presentan en el Capítulo 4.

3.11. Exploración autónoma basada en fronteras

La exploración autónoma permite que el robot seleccione sus propias metas mientras construye el mapa. En el enfoque basado en fronteras, introducido por Yamauchi (1997) y todavía vigente en formulaciones recientes (Liu et al., 2025), una *celda de frontera* es una celda desconocida adyacente a espacio libre conocido; las celdas de frontera contiguas se agrupan en fronteras, y cada frontera se caracteriza por un punto representativo, su tamaño y su distancia al robot. Las fronteras se ordenan mediante un puntaje que combina la distancia, privilegiando metas cercanas, y el tamaño, privilegiando zonas que aportan más información,

y la mejor frontera se envía como meta al sistema de navegación. Cuando una meta no muestra progreso dentro de un tiempo límite, se incorpora a una lista negra para evitar reintentos improductivos, y el proceso se repite hasta que no quedan fronteras alcanzables, momento en que el mapa se considera completo. Los parámetros y particularidades de la implementación utilizada se presentan en el Capítulo 4 y en el ANEXO 5.6.

Conclusión del capítulo

El capítulo establece, en un nivel general, los fundamentos de cada elemento del sistema enumerados en su introducción, desde la descripción física del robot hasta la exploración por fronteras. Sobre estos fundamentos, el Capítulo 4 y los anexos desarrollan los algoritmos, parámetros y configuraciones con que cada elemento se materializa en el prototipo.

CAPÍTULO 4

IMPLEMENTACIÓN

Este capítulo describe la materialización de la solución implementada y la integración de sus principales componentes. Se presentan las decisiones de diseño, los elementos físicos y de *software* utilizados, y la configuración necesaria para la operación del prototipo tanto en simulación como en *hardware* real. Los detalles complementarios de implementación, configuración y verificación se incluyen en los anexos correspondientes.

4.1. Componentes y parámetros del prototipo

El prototipo corresponde a una plataforma diferencial de bajo costo para interiores: controla el desplazamiento mediante la diferencia de velocidad entre las ruedas motrices izquierda y derecha. El costo de los materiales montados en el robot es de 444 643 CLP y el del proyecto completo, de 582 447 CLP, sin incluir la estación de trabajo necesaria para el mapeo, la navegación y la exploración (Tabla A1.1). Ese costo es menor que el de la plataforma comercial comparable más cercana (Tabla A1.2).

4.1.1. Configuración general del prototipo

La locomoción se implementa mediante dos motorreductores JGA25-370 de 12 V, equipados con *encoders* incrementales de cuadratura. El Arduino Nano realiza la lectura de estos sensores y ejecuta el control de velocidad PI incremental, mientras que el *driver* L298N aplica las señales PWM a los motores. La

Raspberry Pi 4 ejecuta la interfaz de *hardware*, adquiere los barridos del RPLidar A1 y se comunica con la estación de trabajo, donde se ejecutan las funciones de mapeo, navegación y exploración. La arquitectura de alimentación y la distribución eléctrica se detallan en la Sec. 4.3. El chasis y las cubiertas se fabrican mediante impresión 3D FDM en PLA+, tomando como referencia una plataforma diferencial de código abierto (marcuslanzoni, 2020). En la Fig. 4.1 se muestra una vista lateral del prototipo construido, donde se distinguen el chasis impreso y el LIDAR.

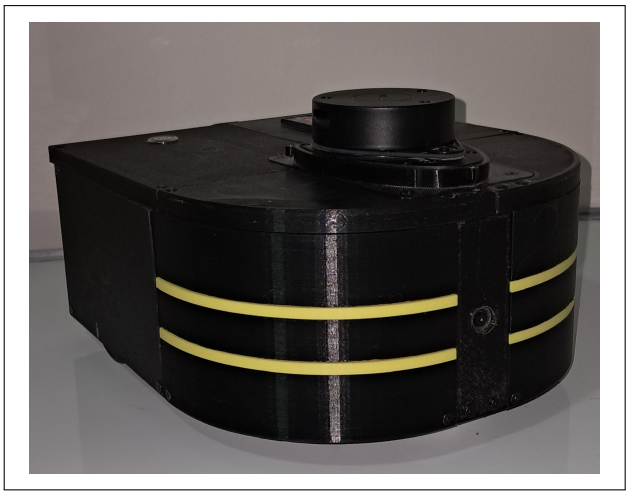


Figura 4.1: Vista lateral del prototipo construido.
Fuente: Elaboración propia.

4.1.2. Parámetros físicos y temporales de implementación

La Tabla 4.1 reúne los parámetros físicos y temporales utilizados por el modelo y la interfaz de *hardware*. Las frecuencias f_{ctrl} y f_{loop} coinciden numéricamente, pero corresponden a mecanismos independientes; la temporalidad interna del *firmware* y la correspondencia entre cada parámetro y su identificador en el código se consolidan en el ANEXO 3.5 y en el ANEXO 3.6.

El valor unificado N_e = 1980 *ticks*/vuelta corresponde al valor efectivamente utilizado por el *firmware* y la interfaz de *hardware* durante los ensayos. La calibra-

Tabla 4.1: Parámetros físicos y temporales empleados en la implementación.

Símbolo	Parámetro	Valor	Uso principal
r	Radio de rueda	0.0335 m	Modelo cinemático, odometría y conversión de velocidad.
b	Separación entre ruedas	0.188 m	Modelo cinemático diferencial.
N_e	Resolución unificada utilizada	1980 *ticks*/vuelta	*Firmware*, interfaz de *hardware* y odometría.
f_{ctrl}	Frecuencia del ciclo de `ros2_control`	30 Hz	Ciclo de lectura, actualización y escritura de la interfaz de *hardware*.
f_{loop}	Frecuencia usada en la conversión de consignas	30 Hz	Conversión de consignas de rueda a *ticks/frame*.

ción experimental por rueda produce promedios ligeramente distintos; las muestras, los promedios y el error sistemático asociado al uso de este valor unificado se conservan en el ANEXO 4.1.

4.2. Gemelo digital del robot

Sobre la base del propósito de validación expuesto en la Sec. 1.5, el gemelo digital se implementa en ROS 2 Foxy mediante URDF/`xacro` y Gazebo Classic 11.11.0. El radio de rueda y la separación entre ruedas de la Tabla 4.1 se emplean como referencia geométrica común en el robot físico, el modelo digital y `diff_drive_controller`. El *backend* `GazeboSystem` entrega directamente los estados de las articulaciones, por lo que no utiliza la conversión por *ticks* ni el planificador del controlador de velocidad del *firmware*. La Fig. 4.2 muestra el modelo cargado en Gazebo Classic.

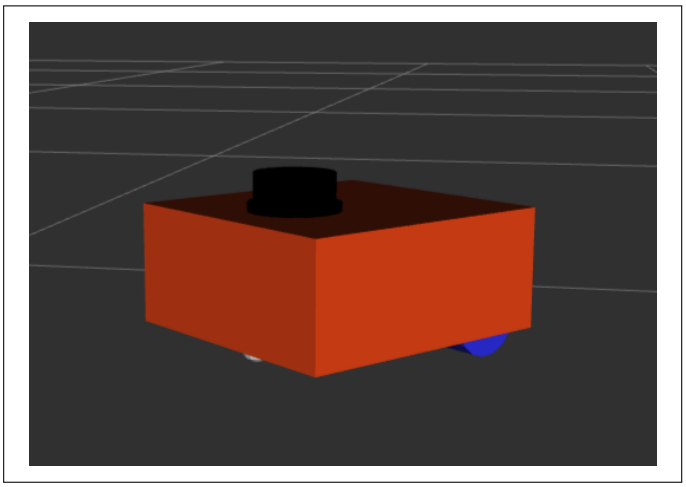


Figura 4.2: Modelo del robot (gemelo digital) cargado en Gazebo Classic.
Fuente: Elaboración propia.

4.2.1. Arquitectura del modelo digital

El modelo digital se organiza mediante una descripción `xacro` modular: la estructura mecánica se define una sola vez y el sensor láser se selecciona según el entorno de ejecución, de modo que *hardware* real y simulación comparten la misma geometría. El inventario de archivos, su función y su localización se consolidan en el ANEXO 9. La geometría del modelo se presenta acotada en la Fig. A2.1 del ANEXO 2.1. La contribución principal en esta etapa corresponde al diseño del modelo URDF/`xacro`, la parametrización geométrica del robot, la definición de marcos de referencia y la integración con los sistemas de control, SLAM y navegación. La atribución completa de paquetes propios, adaptados y externos se documenta en el ANEXO 3.1.

4.2.2. Estructura URDF/xacro y marcos de referencia

El modelo se organiza como un árbol de eslabones y articulaciones cuyo marco principal es `base_link`, solidario al cuerpo del robot. De él se deriva `base_footprint`, el marco de contacto del robot con el suelo, que en este modelo

se declara como un eslabón hijo fijo y coincidente con `base_link`. La organización de estos marcos sigue las convenciones REP-103 y REP-105 descritas en la Sec. 3.1 (Foote & Purvis, 2010; Meeussen, 2010), que fijan la orientación de los ejes y la jerarquía entre el marco del mapa, el odométrico y el de la base. La cadena de transformaciones principal es:

$$\texttt{map} \rightarrow \texttt{odom} \rightarrow \texttt{base_link} \rightarrow \texttt{laser_frame},$$

donde `map` es el marco global del mapa, `odom` el marco odométrico (continuo, con deriva) y `laser_frame` el marco del sensor láser; de esta cadena cuelga, además, la rama fija `base_link`→`base_footprint`. Sobre esta estructura, el marco que `slam_toolbox` y AMCL declaran como base para localizar al robot (`base_link` o `base_footprint`) no es el mismo en simulación que en *hardware* real. Esa asignación por entorno, la correspondencia entre componentes, marcos y tópicos, y la verificación estructural del árbol se documentan en el ANEXO 2.

Dado que `base_footprint` está unido a `base_link` mediante una transformación fija identidad, con traslación y rotación nulas, esta diferencia de identificador no introduce un desfase geométrico entre ambos marcos. Se conserva, no obstante, la asignación específica de cada entorno porque corresponde a la configuración efectivamente utilizada.

4.2.3. Selección del *backend* de control

El modelo utiliza en ambos entornos `ros2_control`, el *framework* de ROS 2 que estandariza el acceso al *hardware* mediante interfaces de comando y de estado, pero carga un *plugin* de *hardware* distinto en cada caso, como resume la Tabla 4.2. La interfaz empleada en el robot real se describe en la Sec. 4.3.3; la trazabilidad completa de la selección se conserva en el ANEXO 2.

Tabla 4.2: *Plugin* de `ros2_control` según entorno de ejecución.

Entorno	*Plugin* de *hardware*	Fuente de los estados
Hardware real	`DiffDriveArduino`	Conteos de *encoder* recibidos desde el Arduino mediante el enlace serial.
Simulación	`GazeboSystem`	Estados de las articulaciones entregados directamente por Gazebo Classic.

4.2.4. Sensor láser simulado y criterios de aceptación

En simulación, el sensor se modela mediante el complemento de simulación correspondiente, publicando datos en `/scan` con el marco `laser_frame`. El modelo reproduce el comportamiento general del RPLidar A1 físico, aunque con diferencias propias de la simulación; estas diferencias se consideran parte de la brecha simulación–realidad y se detallan en el ANEXO 2. La Fig. 4.3 muestra el barrido del LIDAR simulado dentro del mundo de prueba con obstáculos.

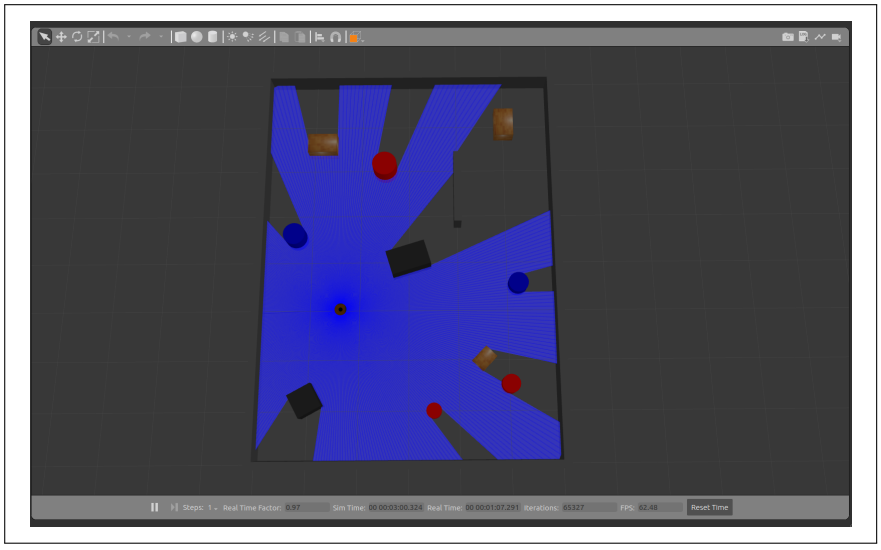


Figura 4.3: Barrido del LIDAR 2D simulado en el mundo de prueba con obstáculos.

Fuente: Elaboración propia.

El barrido cubre el plano horizontal del escenario y se interrumpe en las superficies visibles de los obstáculos, generando zonas de oclusión detrás de ellos.

Esta respuesta permite verificar la coherencia entre la posición del sensor, su marco de referencia y la geometría del mundo simulado antes de utilizar los datos en los procesos de SLAM y navegación. Antes de usar el gemelo digital para pruebas de SLAM y navegación, se aplican los criterios de aceptación resumidos en la Tabla A2.2 del ANEXO 2, donde también se conserva la evidencia estructural.

4.3. Construcción de *hardware* y comunicación con ROS 2

Esta sección describe la arquitectura físico-electrónica del robot y su integración con ROS 2 Foxy sobre la Raspberry Pi 4. Los detalles operativos y de configuración extendida se trasladan al ANEXO 3.

4.3.1. Arquitectura física y de datos

La Fig. 4.4 presenta la vista en planta del prototipo, donde se aprecian el montaje del RPLidar A1 y el panel de control con el interruptor de encendido y el visualizador de tensión y corriente.

La plataforma se organiza en dos dominios principales. El dominio de potencia utiliza un bus de 12 V proveniente de la batería para alimentar los motores DC mediante el *driver* L298N. A partir del mismo bus, un módulo reductor *DC-DC Power Module* de 25 W genera un riel regulado de 5 V para alimentar la Raspberry Pi, el Arduino, el LIDAR y los periféricos. Esta separación reduce la interferencia eléctrica producida por los motores sobre la electrónica de control. El dominio de comunicaciones conecta la Raspberry Pi con el Arduino mediante un enlace serial USB para intercambiar referencias de velocidad y datos de *encoders*. El RPLidar A1 se conecta a un puerto serie UART de la Raspberry Pi, y un nodo de adquisición desarrollado para este trabajo convierte su trama en los barridos utilizados



Figura 4.4: Vista en planta del prototipo construido.
Fuente: Elaboración propia.

por el sistema de percepción (ANEXO 3.4). La asignación de dispositivos y su identificación persistente se documentan en el ANEXO 3. La Fig. 4.5 resume las conexiones eléctricas y de datos del prototipo.

4.3.2. Integración con ROS 2

En *hardware* real, un único archivo de lanzamiento inicia los nodos necesarios para publicar la descripción del robot, activar `ros2_control` y cargar el controlador diferencial junto con el difusor de estados de articulación. El inventario de nodos y la responsabilidad de cada uno se documentan en el ANEXO 3.2. La cadena de transformaciones mantiene la misma convención usada en simulación (Sec. 4.2.2): `slam_toolbox` publica `map`→`odom`, mientras que `diff_drive_controller` publica `odom`→`base_link` a partir de la odometría de *encoders*.

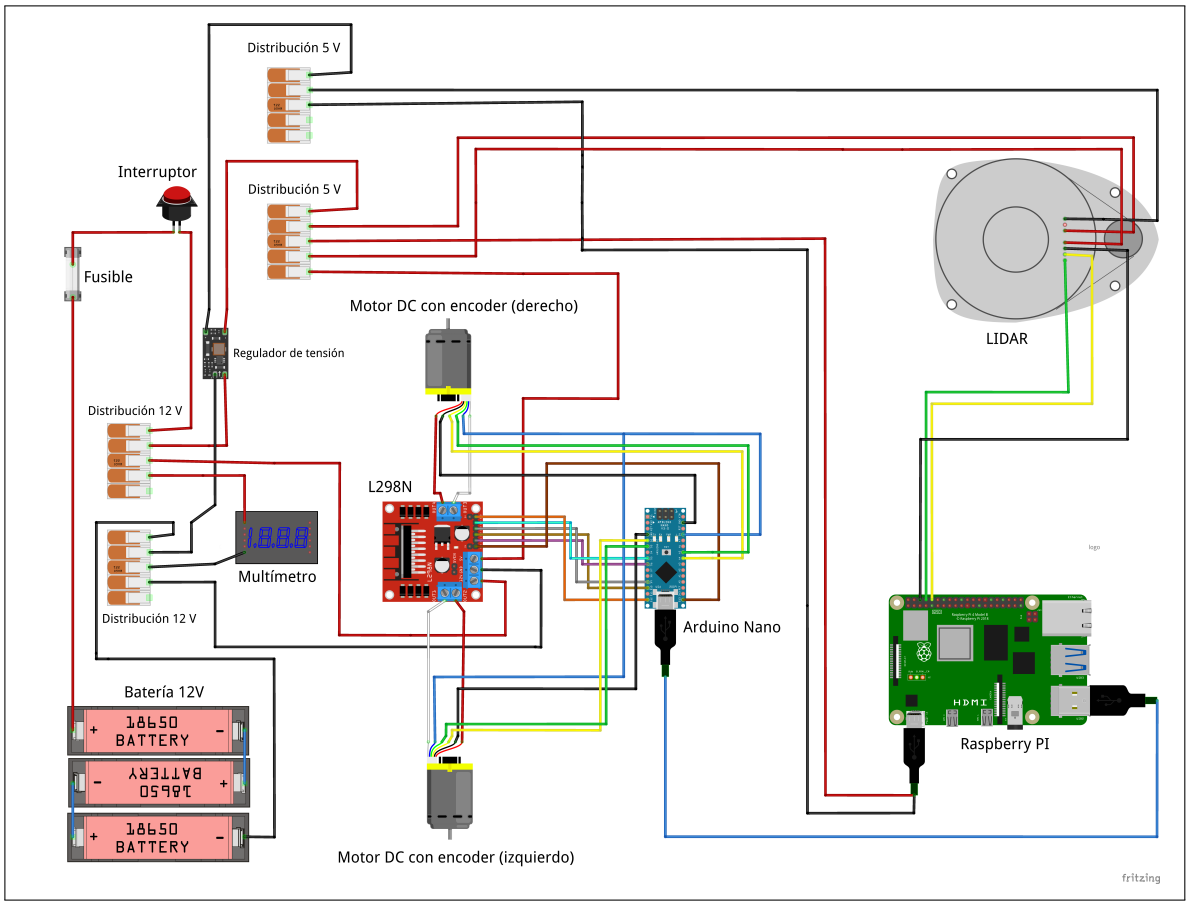


Figura 4.5: Esquema de conexiones eléctricas y de datos del robot diferencial.
Fuente: Elaboración propia.

4.3.3. Control diferencial e interfaz de *hardware*

El controlador diferencial recibe comandos de velocidad lineal y angular, y los convierte en velocidades de rueda mediante el modelo cinemático descrito en el Capítulo 3. Su geometría corresponde al radio de rueda r y a la separación entre ruedas b (Tabla 4.1). Los parámetros principales configurados en el controlador diferencial se resumen en la Tabla 4.3.

La comunicación entre `diff_drive_controller` y el *firmware* se realiza mediante `DiffDriveArduino`, una interfaz de *hardware* adaptada del proyecto `diffdrive_arduino` (Newans, 2022a) e integrada al *framework*

Tabla 4.3: Parámetros principales del controlador diferencial.

Parámetro	Valor	Descripción
Velocidad lineal	–0.13/0.13 m/s	Límites aplicados a la consigna lineal.
Velocidad angular	–0.35/0.35 rad/s	Límites aplicados a la consigna angular.
Publicación de odometría	Habilitada	Publica `odom`→`base_link`.

`ros2_control` (ros-controls, s.f.). La interfaz intercambia con el Arduino, mediante el enlace serial, las referencias de velocidad de rueda y las lecturas de *encoder*. Su ciclo de lectura–actualización–escritura se ejecuta con la frecuencia f_{ctrl} indicada en la Tabla 4.1. En la fase de escritura, las consignas de rueda se convierten a *ticks/frame*; en la fase de lectura, los conteos acumulados $c_j[k]$ se convierten en posición y velocidad angular mediante las Ecs. (3.9) y (3.10), para $j \in \{R, L\}$, con el intervalo Δt_k. Las interfaces expuestas y el desarrollo de ambas conversiones se conservan en el ANEXO 3.5. Las ganancias del controlador no se transmiten en tiempo de ejecución: residen en el *firmware* con los valores obtenidos mediante el procedimiento descrito en la Sec. 4.4.5.

4.3.4. Flujo de datos del sistema

El Alg. 4.1 formaliza el flujo completo entre ROS 2, la interfaz de *hardware* y el *firmware*. Corresponde al ciclo de lectura–actualización–escritura que `ros2_control` repite con la frecuencia f_{ctrl} (Sec. 4.3.3): en cada iteración, la interfaz lee los *encoders* desde el Arduino, reconstruye el estado de cada rueda, actualiza el controlador diferencial y devuelve al *firmware* las nuevas referencias de velocidad. Las conversiones y el intervalo Δt_k empleados se describen en la Sec. 4.3.3 y en el ANEXO 3.5.

El Alg. 4.1 distingue una inicialización que se ejecuta una sola vez, donde se abre el puerto serial y se calcula el paso angular por *tick* $\delta\varphi = 2\pi/N_e$, de un ciclo permanente. Las dos comprobaciones del puerto señalan un error y terminan el ciclo ante pérdida del enlace, y se complementan con la autodetención de seguridad de la Sec. 4.4.4. El intervalo Δt_k y las conversiones se detallan en el

Algoritmo 4.1 Ciclo ejecutado por la interfaz `DiffDriveArduino`.

Require: Comandos $\omega_L^{cmd}, \omega_R^{cmd}$; enlace serial; resolución N_e; frecuencia f_{loop}.
Ensure: Estados de rueda exportados y referencias enviadas al *firmware*.

$t_{prev} \leftarrow$ TiempoActual
Leer parámetros de ruedas y comunicación.
$\delta\varphi \leftarrow 2\pi/N_e$
Abrir el puerto serial.
Esperar 3000 ms y vaciar el búfer de entrada.
Enviar un retorno de carro vacío.
while la interfaz esté iniciada **do**
    if el puerto serial está cerrado **then**
        return `ERROR`
    end if

    $t_k \leftarrow$ TiempoActual
    $\Delta t_k \leftarrow t_k - t_{prev}$
    $t_{prev} \leftarrow t_k$
    `resp` $\leftarrow$ SolicitarSerial(`e`)
    $(c_L[k], c_R[k]) \leftarrow$ ConvertirEnteros(`resp`)
    for $j \in \{L, R\}$ **do**
        $(\varphi_j[k], \widehat{\omega}_j[k]) \leftarrow$ ActualizarEstadoRueda($c_j[k], \Delta t_k$)
    end for
    Exportar $(\widehat{\omega}_L, \varphi_L, \widehat{\omega}_R, \varphi_R)$.
    ActualizarControladorDiferencial.

    if el puerto serial está cerrado **then**
        return `ERROR`
    end if
    for $j \in \{L, R\}$ **do**
        $m_j^{cmd} \leftarrow$ ConvertirATicksFrame($\omega_j^{cmd}, N_e, f_{loop}$)
    end for
    EnviarComando(`m`,m_L^{cmd}, m_R^{cmd})
end while

ANEXO 3.5.

4.3.5. Teleoperación y multiplexación de comandos

La teleoperación permite conducir el robot de forma manual, tanto para pruebas de desplazamiento como para posicionarlo antes de iniciar cada ensayo. Se realiza con un mando inalámbrico cuyas palancas se convierten en comandos de velocidad lineal y angular. Como mecanismo de seguridad, el mando solo publica comandos mientras se mantiene presionado un botón de habilitación, de modo que al soltarlo el robot se detiene.

Los comandos manuales y los de Nav2 se integran mediante el multiplexor `twist_mux`, que selecciona una única fuente según prioridades y descarta las que dejan de emitir. La teleoperación tiene prioridad, así el operador puede retomar el control en cualquier momento. La salida del multiplexor se remapea a la entrada del controlador diferencial (Sec. 4.3.3) y aparece en la arquitectura de navegación de la Fig. 4.6. Los nodos, las prioridades y el remapeo se documentan en el ANEXO 3.

4.4. *Firmware* de bajo nivel en Arduino Nano

El *firmware* `ROSArduinoBridge` parte de un proyecto de código abierto homónimo (Newans, 2022b) y se adapta para este trabajo, incorporando ganancias independientes por rueda, una banda muerta sobre el error y la resolución efectiva de *encoders* del prototipo, sin modificar la estructura de su ley de control. Cumple cuatro funciones principales: comunicación serial con ROS 2 mediante `ros2_control`, lectura de *encoders*, control de velocidad PI incremental por rueda y autodetención de seguridad ante pérdida de comandos.

4.4.1. Arquitectura modular y constantes físicas

El *firmware* se organiza en módulos independientes, cuya descomposición, junto con el inventario de archivos y rutas, se conserva en el ANEXO 4. Utiliza la geometría y la resolución efectiva de *encoders* de la Tabla 4.1 para expresar las referencias y mediciones en *ticks/frame*. La temporalidad interna del controlador es independiente del ciclo de `ros2_control`, y su programación se documenta en el ANEXO 3.5.

4.4.2. Comunicación serial y lectura de *encoders*

La comunicación entre la Raspberry Pi y el Arduino se realiza mediante el protocolo serial textual del enlace descrito en la Sec. 4.3. El conjunto de comandos implementado incluye lectura de *encoders*, referencias de velocidad, carga de ganancias y accionamiento directo en lazo abierto, y se presenta en la Tabla A4.2 del ANEXO 4.3.

La lectura de los *encoders* no ocurre dentro del bucle principal. Cada cambio de nivel en los canales de un *encoder* dispara una interrupción de cambio de pin, cuya rutina actualiza el contador de esa rueda. De este modo el conteo avanza mediante interrupciones, en paralelo a la ejecución del programa principal y sin depender directamente de su tiempo de ciclo. Este mecanismo permite registrar transiciones mientras el bucle atiende el protocolo serial o el control de velocidad, sin asumir que todas las transiciones puedan registrarse bajo cualquier condición temporal. El programa principal solo lee el valor acumulado cuando lo necesita: en cada ciclo del controlador para calcular la velocidad de cada rueda, y al responder el comando de lectura de *encoders*, a partir del cual la Raspberry Pi estima la odometría. Los conteos recibidos por el enlace serial se interpretan mediante la resolución efectiva N_e (Tabla 4.1). El tratamiento de la lectura no atómica de los contadores se conserva en el ANEXO 4.1.

4.4.3. Control de velocidad en el *firmware*

El controlador de velocidad se ejecuta de forma independiente para cada rueda en el Arduino Nano, con la referencia y la medición expresadas en *ticks/frame* y un período de ejecución $T_{PID,sched}$ = 33 ms. La ley se implementa en tiempo discreto con una forma incremental acumulativa, en la que el término asociado a $K_{d,j}$ se calcula sobre la diferencia de la medición. Las Ecs. (4.1), (4.2) y (4.3) resumen el incremento de control, la actualización del integrador y la salida saturada:

$$\Delta u_j^{long}[k] = \mathrm{trunc}_0\left(\frac{K_{p,j}e_j^{db}[k] - K_{d,j}\left(m_j^{int}[k] - m_j^{int}[k-1]\right) + I_j^{int}[k]}{K_o}\right), \tag{4.1}$$

$$I_j^{int}[k+1] = \begin{cases} \mathrm{trunc}_0\left(I_j^{int}[k] + K_{i,j}e_j^{db}[k]\right), & |u_j^{raw}[k]| < 255, \\ I_j^{int}[k], & \text{en otro caso,} \end{cases} \tag{4.2}$$

$$u_j[k] = \mathrm{sat}_{[-255,255]}\left(u_j[k-1] + \Delta u_j^{long}[k]\right). \tag{4.3}$$

donde $K_{p,j}$, $K_{i,j}$ y $K_{d,j}$ son las ganancias de la rueda j que el *firmware* original denomina proporcional, integral y derivativa, cuya acción efectiva se aclara más adelante; $e_j^{db}[k]$ es el error discreto de velocidad tras la banda muerta; $m_j^{int}[k]$ la medición de velocidad en la escala interna del *firmware*; $I_j^{int}[k]$ el estado integral, actualizado según la condición de saturación; $\Delta u_j^{long}[k]$ el incremento de control; $u_j[k]$ la salida acumulada saturada; K_o el divisor de escalamiento de la salida, común a ambas ruedas; y $\mathrm{sat}_{[-255,255]}(\cdot)$ la saturación al intervalo PWM permitido. Se define $u_j^{raw}[k] = u_j[k-1] + \Delta u_j^{long}[k]$, y trunc_0 representa la conversión entera con truncamiento hacia cero, dentro del rango representable del tipo de destino.

La estructura anterior resume el controlador ejecutado en el *firmware* y no coincide exactamente con la forma incremental estándar de un PID. En particular, el término $K_{p,j}e_j^{db}[k]$ forma parte del incremento que posteriormente se suma a la salida anterior $u_j[k-1]$, por lo que su efecto también se acumula entre ciclos. Además, el estado $I_j^{int}[k]$ acumula separadamente el término $K_{i,j}e_j^{db}[k]$ cuando su

actualización está habilitada. En consecuencia, las ganancias reportadas en este trabajo caracterizan esta realización discreta específica y no deben interpretarse según el nombre que reciben en el *firmware*. Esta estructura es la misma del *firmware* original (Home Brew Robotics Club, s.f.; Newans, 2022b) y no se modifica en este trabajo ni durante los ensayos, por lo que los resultados experimentales del Capítulo 5 corresponden al controlador efectivamente implementado. Su origen y los cambios introducidos respecto del original se detallan en el ANEXO 4.5.

Si se desprecian la saturación y los truncamientos enteros, y se parte de un reinicio con la salida acumulada, la medición previa y el estado integral en cero, la salida acumulada puede escribirse sin incrementos, como indica la Ec. (4.4):

$$u_j[k] = \frac{K_{p,j}\, S_j^e[k] - K_{d,j}\, m_j^{int}[k] + S_j^I[k]}{K_o}, \tag{4.4}$$

donde $S_j^e[k] = S_j^e[k-1] + e_j^{db}[k]$ es el error acumulado desde el reinicio y $S_j^I[k] = S_j^I[k-1] + I_j^{int}[k]$ la acumulación del estado integral, ambos con valor inicial cero. Esta forma muestra el efecto real de cada ganancia. $K_{p,j}$ multiplica el error acumulado, por lo que actúa como ganancia integral. $K_{d,j}$ multiplica la medición actual, por lo que actúa como ganancia proporcional sobre la medición y no como acción derivativa. $K_{i,j}$ actúa sobre una acumulación del estado integral, es decir, como una segunda integración. La ley ejecutada no contiene una acción derivativa efectiva.

Al omitir el estado adicional y prescindir de la banda muerta, la saturación y el truncado, la ley idealizada coincide con la forma incremental del PID descrita por Åström y Hägglund (2006/2009, Sec. 13.4) y presentada en la Ec. (A4.1) del ANEXO 4.12, con la integral aproximada por diferencia hacia atrás, sin acción derivativa y con ponderación nula del punto de consigna en la acción proporcional. En esa correspondencia, $K_{d,j}/K_o$ cumple el papel de la ganancia proporcional y $K_{p,j}/K_o$ el del coeficiente integral por muestra, de modo que la ley idealizada es un controlador PI incremental, es decir, un PID sin acción derivativa, y así se denomina en este trabajo. El nombre PID se conserva para la familia de controladores, para el OE2 y para los nombres del *firmware* y de sus *scripts*, como K_p, K_i, K_d o `doPID()`. Con ponderación nula, la referencia solo entra en el término integral, lo que evita

saltos de la señal de control y reduce la sobreelongación ante cambios del punto de consigna (Åström & Hägglund, 2006/2009, Sec. 3.4). La rueda derecha opera en esa condición, porque su estado integral permanece en cero, y en la izquierda ese estado solo crece con errores grandes (Sec. 5.2).

Esta ley se conserva en lugar de reemplazarla por otra formulación del PID o por una ley escrita desde cero, por tres razones. La primera es que corresponde a un controlador conocido y con uso documentado en robótica móvil, el PI incremental. El PI es el controlador más común (Åström & Hägglund, 2006/2009, Sec. 3.6) y el PID, el algoritmo de control más usado en sistemas robóticos (Raj & Kos, 2024), también en robots móviles (Bitriá & Palacín, 2022). Su forma incremental, de la que este PI es el caso sin acción derivativa, se usa en el lazo de velocidad de robots diferenciales de laboratorio (Casini et al., 2010), de vehículos autoequilibrados de dos ruedas (Ma et al., 2024), de andadores robóticos controlados con Arduino (Mori et al., 2024) y de plataformas con ROS 2 que, como la de este trabajo, envían las consignas de velocidad a la placa de control por puerto serie (Wu et al., 2026). También regula el deslizamiento de las ruedas motrices de vehículos agrícolas (Sun et al., 2022) y se usó comúnmente en los primeros controladores de motores (Åström & Hägglund, 2006/2009, Sec. 13.4). La segunda es que conservarla permite reutilizar una implementación ya integrada con la interfaz `diffdrive_arduino` (Newans, 2022a), que usa el protocolo serial de este *firmware*, y concentrar el desarrollo en la navegación y la exploración. Mantener un algoritmo por compatibilidad con el equipo existente es, además, una de las razones por las que los fabricantes conservan la forma incremental (Åström & Hägglund, 2006/2009, Sec. 13.4). La tercera es que la forma incremental es útil aun cuando la integración se realiza dentro del controlador, porque simplifica la protección contra el *windup* del integrador y permite realizar la mayoría de los cálculos solo con incrementos (Åström & Hägglund, 2006/2009, Secs. 13.1 y 13.4), propiedad por la que también se adopta en implementaciones de referencia del PID (Sundström et al., 2024). La elección no responde a una comparación de desempeño con otras leyes, que no forma parte de los objetivos del trabajo y queda como trabajo futuro.

La ejecución concreta incorpora variables enteras, truncamiento hacia ce-

ro, bandas muertas distintas por rueda y actualización condicional del integrador. Las ecuaciones exactas y sus tipos de datos se conservan en el ANEXO 4.4. Las ganancias definitivas se presentan en la Tabla 4.4; el orden de ejecución, los reinicios y la saturación se detallan en el Alg. 4.2.

Tabla 4.4: Ganancias definitivas del controlador codificadas en el *firmware*.

Parámetro	Rueda izquierda	Rueda derecha
K_p	16.00	16.42
K_d	20.30	20.05
K_i	0.075	0.001
K_o	50	50

Las ganancias se aplican en la escala interna del *firmware*. Sin el término asociado a K_i, equivalen a un PI incremental con ganancia proporcional K_d/K_o de 0.41 y 0.40 PWM por *tick/frame* y tiempo integral de 42 y 40 ms (ANEXO 4.12).

Algoritmo 4.2 Rutinas de reinicio y actualización del controlador discreto ejecutadas por el *firmware*.

Require: Conteos c_L, c_R; referencias `TargetTicksPerFrame`; ganancias y límites de saturación definidos previamente.
Ensure: Salidas PWM y estados internos actualizados.

procedure resetPID
    Cargar las ganancias predeterminadas de ambas ruedas.
    Leer los conteos actuales de los *encoders*.
    Inicializar `output`, `PrevInput` e `ITerm` en cero.
end procedure

procedure updatePID
    Leer $c_L[k]$ y luego $c_R[k]$.
    if `moving`= 0 **then**
        if algún `PrevInput` es distinto de cero **then**
            resetPID
        end if
        return
    end if
    doPID(controlador derecho,1)
    doPID(controlador izquierdo,2)
    setMotorSpeeds(`leftPID.output`,`rightPID.output`)
end procedure

El Alg. 4.2 presenta la secuencia de llamadas de la rutina de actualización. Las operaciones internas del cálculo por rueda, es decir, la banda muerta asi-

métrica, la saturación de la salida acumulada y la actualización condicional del integrador, se detallan en el ANEXO 4.4.

La forma incremental del PI es una realización clásica, que entrega el cambio de la señal de control en lugar de su valor (Åström & Hägglund, 2006/2009, Sec. 13.4). Sin saturación, equivale a la forma absoluta del mismo PI (Wittenmark et al., 2002, Sec. 8), por lo que su estabilidad es la de un PI clásico con iguales ganancias. El PI es adecuado para procesos de dinámica esencialmente de primer orden (Åström & Hägglund, 2006/2009, Sec. 3.6), como el modelo identificado (Sec. 4.4.5). El producto del período de 33 ms por la frecuencia de cruce, cercano a 0.07, queda dentro del intervalo recomendado de 0.05 a 0.14 para la aproximación al diseño continuo (Wittenmark et al., 2002, Secs. 6 y 8). Aun así, la verificación se realiza sobre el lazo discreto, con las ganancias y el modelo de cada rueda.

Se analiza la idealización lineal de las Ecs. (4.1) a (4.3) con el modelo FOPDT de cada rueda, sin saturación, banda muerta ni truncamientos enteros (ANEXO 4.12). Se consideran dos escenarios de retardo, $L_{d,j} = 0$ y $L_{d,j} = 0.1$ s. El segundo es un escenario de sensibilidad, porque el período de captura no establece una cota física del retardo. Los modelos analizados son estables en ambos escenarios, con márgenes de fase de 53° a 57° y de 41° a 44°, respectivamente, y sensibilidad máxima de 1.35 a 1.85, dentro del intervalo típico de 1.2 a 2.0 (Wittenmark et al., 2002, Sec. 5). El menor margen de retardo del modelo sin retardo es de unos 0.42 s (Tabla A4.5). Estos márgenes caracterizan los modelos lineales y no constituyen una garantía global para el programa no lineal ni una medición del retardo del robot.

4.4.4. Bucle principal y autodetención

El bucle principal atiende el *parser* serial, programa la ejecución del controlador de velocidad y evalúa la autodetención. Si transcurre el intervalo de autodetención sin recibir comandos de movimiento, el *firmware* detiene los motores,

evitando que el robot continúe desplazándose ante pérdida de conexión o falla de algún nodo de navegación. Ambos intervalos programados se documentan en el ANEXO 3.5 y en el ANEXO 4.3.

4.4.5. Sintonización del controlador asistida por PSO

Las ganancias del controlador se determinan mediante el procedimiento híbrido descrito en la Sec. 3.7, que encadena identificación FOPDT, optimización por PSO y refinamiento experimental, formalizado en el Alg. 4.3. La configuración del enjambre, las escalas, las cotas y las funciones de costo se conservan en el ANEXO 4.9.

Algoritmo 4.3 Sintonización híbrida del controlador mediante identificación FOPDT y PSO.

Require: Datos de lazo abierto por rueda; período T_{cap}; configuración del PSO.
Ensure: Ganancias definitivas por rueda.
1: Cargar las señales de entrada, velocidad y tiempo.
2: **for** $j \in \{L, R\}$ **do**
3: $v_j^{filt} \leftarrow \text{FiltrarVelocidad}(v_j^{med})$
4: Definir J_{ID} como el MSE de identificación (ANEXO 4.6).
5: $(K_{m,j}, \tau_j, L_{d,j}) \leftarrow \text{PSO}(J_{ID})$
6: Definir J_{PID} como el costo MSE + sobreimpulso (ANEXO 4.8).
7: $(K_{p,j}, K_{d,j}, K_{i,j}) \leftarrow \text{PSO}(J_{PID})$
8: **end for**
9: Refinar las ganancias sobre el robot real.
10: Registrar las ganancias definitivas en el *firmware*.
11: **return** $(K_{p,j}, K_{d,j}, K_{i,j})$, para $j \in \{L, R\}$.

Captura de respuesta en lazo abierto

La identificación parte de una prueba en lazo abierto. Se aplica un escalón de PWM y se registran los conteos de *encoder* cada $T_{cap} = 0.1$ s durante 10 s, calculando la velocidad de cada rueda a partir del incremento de conteo. El

procedimiento reproducible, el valor del escalón y el tratamiento del período de muestreo se conservan en el ANEXO 4.9.

Identificación FOPDT y optimización por PSO

La identificación FOPDT ajusta los parámetros (K_m, τ, L_d) de la Ec. (3.11) por PSO sobre la respuesta en lazo abierto y utiliza el modelo obtenido para simular el controlador. Sobre ese modelo, una segunda búsqueda por PSO obtiene las ganancias iniciales. El optimizador aplica la formulación de la Sec. 3.7.1, con la actualización de velocidad e inercia de las Ecs. (3.13) y (3.14). Las cotas, el escalado, la transformación de salida y las aproximaciones numéricas de la planta y del controlador se documentan en el ANEXO 4.6 y en el ANEXO 4.8.

Los retardos identificados, $L_{d,L}$ = 0.0001 s y $L_{d,R}$ = 0.0692 s (Tabla A4.4), son menores que el período de captura de 0.1 s y se cuantizan a cero muestras, por lo que el modelo ajustado equivale a uno de primer orden sin retardo (Sec. 3.5). La estructura FOPDT se conserva porque contiene ese caso y permite estudiar escenarios de retardo en el análisis de estabilidad, como se detalla en el ANEXO 4.6.

Refinamiento experimental

Las ganancias sugeridas por PSO se usan como punto de partida y luego se ajustan sobre el robot real (Etapa 3 del Alg. 4.3), hasta obtener los valores definitivos de la Tabla 4.4. La ejecución de partida sugiere un K_i casi nulo y valores de K_p y K_d cercanos a 11 y 35 en ambas ruedas. Al cargar esas ganancias y aplicar un escalón de 0.13 m/s aparecen dos comportamientos que el modelo FOPDT no anticipa (Fig. 5.3).

El primero es un error estacionario persistente. Con K_i casi nulo, la rueda izquierda se estabiliza un escalón de cuantización por debajo de la referencia, por

lo que su K_i se aumenta a 0.075, mientras que la derecha converge por sí sola y mantiene un valor mínimo de 0.001. El segundo es un arranque lento asociado a la zona muerta del *driver*, descrita en la Sec. 3.3.3, que el modelo de primer orden no representa. Para atravesar ese tramo se aumenta K_p a 16.00 y 16.42, lo que en la ley ejecutada acelera el crecimiento de la salida mientras persiste el error. Ese aumento, sobre una velocidad cuantizada por los *encoders*, vuelve excesiva la acción asociada a K_d, que en esta ley actúa en proporción a la medición, por lo que K_d se reduce desde valores cercanos a 35 hasta 20.30 y 20.05. La salida completa de la ejecución de partida se conserva en la Tabla A4.4 del ANEXO 4.10, y el aporte efectivo del PSO a las ganancias finales se analiza en la Sec. 5.2.

4.5. Arquitectura de control y navegación: SLAM, Nav2 y exploración

Las capas de *hardware*, *firmware*, gemelo digital y control de bajo nivel se desarrollan en las secciones anteriores; esta sección enfatiza la orquestación de los subsistemas mediante `ros2_control`, `slam_toolbox`, Nav2 y `explore_lite`. La correspondencia entre archivos de configuración, entorno y función se reúne en el ANEXO 9, y los parámetros completos se trasladan al ANEXO 5.

4.5.1. Mapeo con `slam_toolbox`

La técnica de SLAM por grafos de poses descrita en la Sec. 3.8 se materializa en este trabajo mediante SLAM Toolbox (Macenski & Jambrecic, 2021), un sistema de SLAM 2D por grafos de poses diseñado para ROS 2. Procesa los barridos de forma asíncrona, lo que evita bloquear el sistema cuando la carga computacional crece, e incorpora detección de cierre de lazo. El robot construye el mapa del entorno mediante `slam_toolbox`, en su modo asíncrono en línea. En la arquitectura distribuida ejecutada, este nodo no se ejecuta en la Raspberry Pi

4 sino en la estación de trabajo, que tiene un procesador de clase portátil (Tabla A3.2). La Raspberry Pi 4 publica `/scan`, `/odom` y las transformaciones asociadas al robot. El uso de CPU del nodo no se mide (ANEXO 8). Las configuraciones de *hardware* y simulación comparten la estructura del algoritmo, pero emplean parámetros específicos para cada entorno; los valores cargados se consolidan en la Tabla A5.3.

4.5.2. Localización sobre un mapa conocido con AMCL

La localización de Monte Carlo adaptativa presentada en la Sec. 3.9 se implementa mediante AMCL (*Adaptive Monte Carlo Localization*), el nodo que Nav2 incorpora para este propósito. AMCL estima la transformación entre los marcos `map` y `odom`, mientras que la odometría mantiene la transformación entre `odom` y el marco base del robot. En este trabajo la localización se resuelve mediante `slam_toolbox` durante el mapeo en línea, que estima la transformación `map`→`odom` a medida que construye el mapa. Ese es el modo empleado en la totalidad de los ensayos descritos en el Capítulo 5. La localización sobre un mapa previamente almacenado mediante AMCL corresponde a un modo distinto, que la pila de navegación ofrece con sus parámetros por defecto y que queda fuera del alcance declarado en la sección de alcances y limitaciones. Ese nodo no se ajusta para este robot ni se ejecuta durante los ensayos.

4.5.3. Navegación autónoma con Nav2

Nav2 recibe una meta de navegación, calcula una ruta global y genera comandos de velocidad para ejecutarla evitando obstáculos. La Fig. 4.6 sintetiza esta interacción y la sitúa dentro del flujo completo del sistema: la percepción del RPLidar A1 alimenta a `slam_toolbox` y a los mapas de costos; `bt_navigator` coordina la planificación global y el control local; y los comandos resultantes se arbitran en `twist_mux` antes de llegar al control diferencial de bajo nivel descrito

en la Sec. 4.3.3.

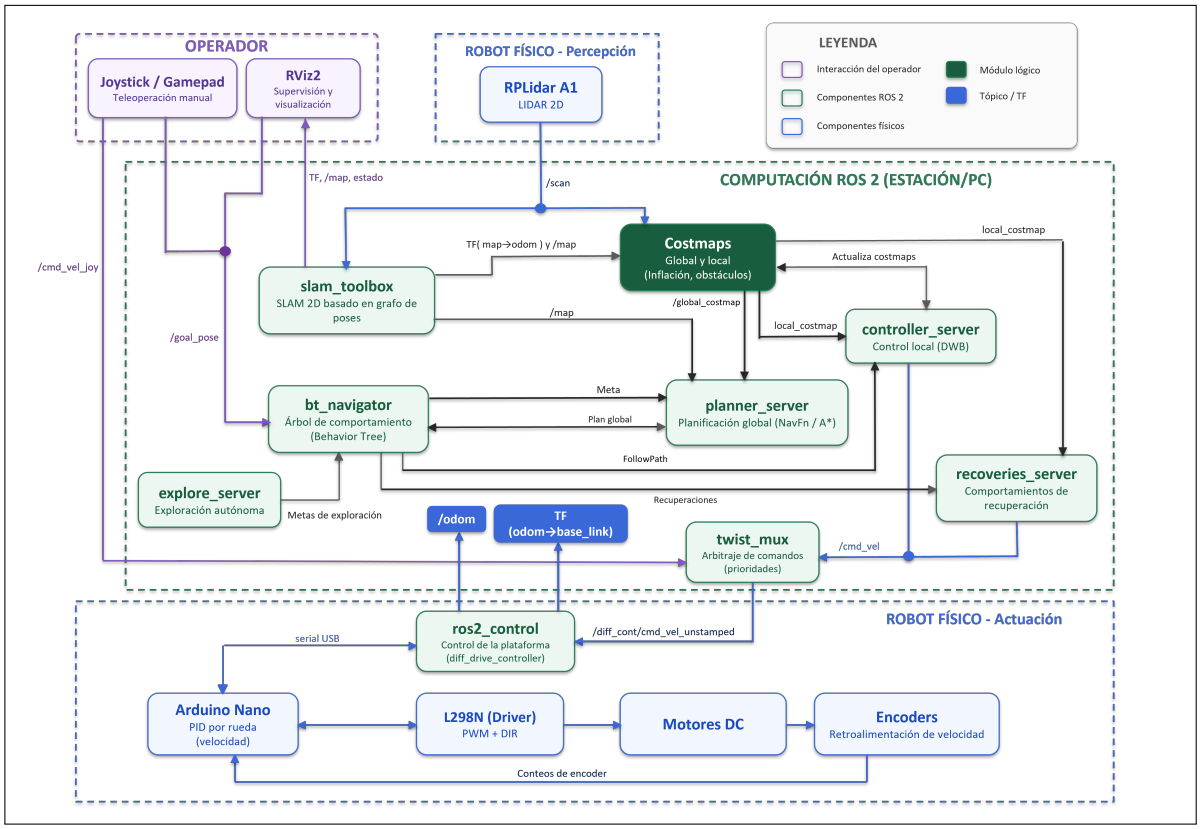


Figura 4.6: Interacción entre los componentes de la navegación autónoma.
Fuente: Elaboración propia.

La realimentación de los *encoders* cierra el control de velocidad de las ruedas y actualiza la odometría utilizada por los módulos superiores. El diagrama organiza los nodos según su rol dentro de la arquitectura ROS 2 y no según el equipo que los ejecuta. En la configuración distribuida empleada, `slam_toolbox`, Nav2 y `explore_lite` se ejecutan en la estación de trabajo, mientras que la interfaz `ros2_control` y la publicación de `/scan`, `/odom` y las transformaciones asociadas residen en la Raspberry Pi 4 (Sec. 4.3).

La configuración de los planificadores global y local se resume en la Tabla 4.5. El controlador local evalúa las trayectorias candidatas mediante la Ec. (3.16).

Tabla 4.5: Configuración de los planificadores global y local utilizados.

Componente	*Plugin*	Configuración principal
Planificador global	NavFn	A* habilitado, espacio desconocido permitido y tolerancia de 0.5 m.
Controlador local	DWB	Críticos de obstáculos, oscilación, alineación y distancia respecto de la ruta y de la meta.

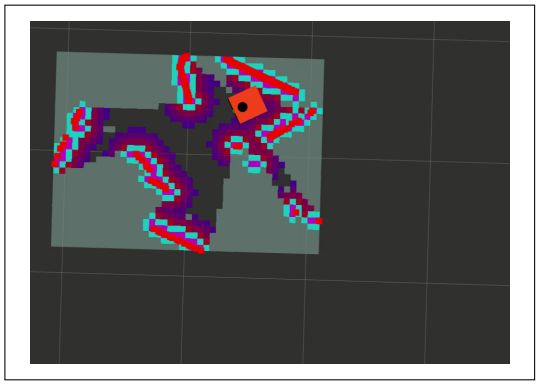
La relación entre los límites del controlador local y del controlador diferencial, junto con los identificadores completos de ambos *plugins*, se documenta en el ANEXO 5.

4.5.4. Mapas de costos

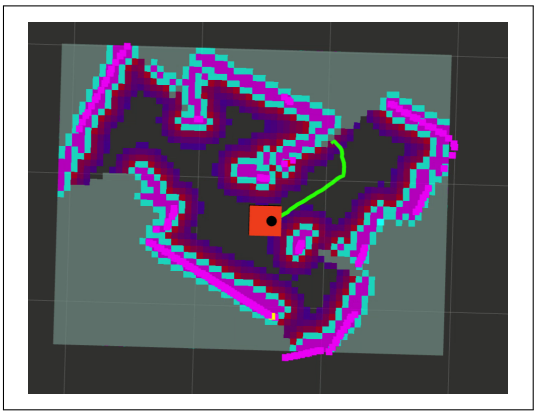
Nav2 utiliza dos mapas de costos. El `global_costmap` entrega la representación usada por el planificador global, mientras que el `local_costmap` se actualiza alrededor del robot y permite al controlador DWB reaccionar ante obstáculos cercanos. Ambos mapas incorporan una capa de inflación, necesaria para mantener una distancia de seguridad respecto de los obstáculos. En *hardware* real ambos mapas de costos emplean un radio de inflación de 0.15 m, valor inferior al configurado en simulación; sus implicancias sobre la navegación en un recinto reducido se discuten en la Sec. 5.4. La geometría completa de los *costmaps* se presenta en el ANEXO 5. La Fig. 4.7 ilustra el uso del mapa de costos global durante una ejecución de navegación y exploración autónomas, desde la detección de fronteras hasta la cobertura alcanzada del recinto.

4.5.5. Árbol de comportamiento y recuperación

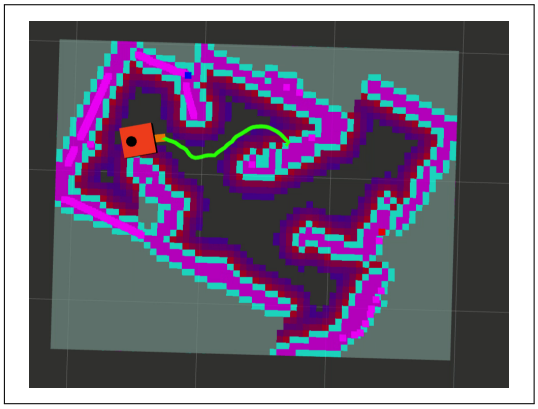
La navegación de alto nivel se organiza mediante el árbol de comportamiento descrito en la Sec. 3.10. El árbol ejecutado replanifica la ruta de forma periódica y aplica recuperaciones en dos niveles: ante fallos de planificación o seguimiento



(a) Fronteras detectadas en la fase inicial.



(b) Ruta planificada por Nav2 hacia una meta.



(c) Cobertura avanzada del recinto.

Figura 4.7: Evolución del mapa de costos global durante una ejecución de navegación y exploración autónomas.

Fuente: Elaboración propia.

actúa sobre el *costmap* correspondiente; si falla la navegación completa, ejecuta una secuencia de recuperación configurada, detallada en la Tabla A5.2.

4.5.6. Exploración autónoma de fronteras

Para mapear entornos desconocidos sin intervención manual continua, se integra el módulo de exploración por fronteras seleccionado en la Sec. 2.1 (`explore_lite`), que identifica, sobre el *costmap* global, las fronteras entre espa-

cio libre conocido y zonas no observadas, y envía metas de navegación a Nav2. De esta forma, se cierra el ciclo percepción–mapeo–navegación–exploración sobre la misma arquitectura. Los parámetros completos se conservan en la Tabla A5.4 del ANEXO 5, y el flujo de detección, selección, envío de metas y gestión de la lista negra se resume en el Alg. 4.4.

Dentro del Alg. 4.4, el puntaje de cada frontera combina distancia y tamaño, de modo que se prefieren las cercanas y las que prometen descubrir más entorno. La meta se envía con orientación identidad, porque solo importa la posición de la frontera, y la lista negra B evita reintentar metas improductivas. Como Nav2 informa como abortada también la meta que el explorador reemplaza (Sec. 5.4), esa lista recibe además puntos de metas que no fallan. Si no queda ninguna frontera, el entorno alcanzable se considera mapeado. El cálculo del punto representativo, el escalamiento de la distancia y la programación del temporizador se documentan en el ANEXO 5.6.

4.5.7. Transferencia simulación–*hardware*

La transferencia al *hardware* conserva la arquitectura de navegación descrita en la Sec. 4.5.3, manteniendo la estructura de tópicos y marcos, pero seleccionando archivos de configuración específicos para el entorno real. La correspondencia entre los archivos equivalentes de simulación y *hardware* se detalla en el ANEXO 5. Además, se ajustan límites cinemáticos, frecuencias, tolerancias e inflación de los mapas de costos según las capacidades del prototipo físico.

Algoritmo 4.4 Integración operativa de `explore_lite` con Nav2.

Require: *Costmap* global C; lista negra B; tamaño mínimo de frontera; tiempo máximo sin progreso.
Ensure: Metas `NavigateToPose` seleccionadas desde fronteras alcanzables.

Ejecutar una planificación inicial.
Programar las planificaciones periódicas.
while la exploración esté activa **do**
    $q \leftarrow$ ObtenerPoseRobot
    $\mathcal{F} \leftarrow$ DetectarFronteras(C, q)
    for cada frontera $F \in \mathcal{F}$ **do**
        Calcular p_F^{impl} y U(F) según el ANEXO 5.6.
    end for
    Eliminar las fronteras menores que `min_frontier_size`.
    Ordenar $\mathcal{F}$ por costo ascendente.
    if $\mathcal{F} = \varnothing$ **then**
        Cancelar la meta activa.
        Detener la exploración.
        return
    end if
    $F^* \leftarrow$ primera frontera de $\mathcal{F}$ cuyo punto no pertenezca a B.
    if no existe F^* **then**
        $B \leftarrow \varnothing$
    else
        $p^* \leftarrow p_{F^*}^{impl}$
        if la meta no presenta progreso durante `progress_timeout` **then**
            $B \leftarrow B \cup \{p^*\}$
            Repetir la selección con la lista negra actualizada.
        else if p^* está a 0.01 m o más de la meta anterior **then**
            Enviar p^* a `NavigateToPose` sin cancelar la meta en curso.
        end if
    end if
    Esperar el siguiente período de $1/f_{exp}$.
end while
Al recibir el resultado de una meta enviada al punto p**:**
if `estado` = `SUCCEEDED` **then**
    Ejecutar una nueva planificación de inmediato.
else if `estado` = `ABORTED` **then**
    $B \leftarrow B \cup \{p\}$
else if `estado` = `CANCELED` **then**
    Conservar B sin cambios.
end if

Conclusión del capítulo

La implementación integra componentes accesibles: plataforma diferencial, *encoders*, Arduino Nano, Raspberry Pi 4 y RPLidar A1, con una arquitectura de *software* basada en ROS 2. El gemelo digital, construido sobre los mismos parámetros geométricos del prototipo, permite validar de forma previa la cinemática, los marcos de referencia y la configuración de control, facilitando la transferencia hacia el *hardware* real. La comunicación mediante `ros2_control`, `diff_drive_controller` y el *plugin* `DiffDriveArduino` cierra el ciclo entre comandos de navegación, control PI incremental de ruedas (con ganancias iniciales obtenidas por PSO y refinadas en el robot, y con estabilidad verificada sobre el modelo FOPDT identificado) y publicación de odometría, con autodetención de seguridad ante pérdida de comandos. Sobre esta base, la orquestación de `slam_toolbox`, Nav2 y `explore_lite` articula percepción, mapeo, navegación y exploración autónoma sobre una misma estructura de tópicos y marcos, tanto en simulación como en *hardware* real. Su desempeño se evalúa en el Capítulo 5.

CAPÍTULO 5

RESULTADOS EXPERIMENTALES

Este capítulo presenta los resultados obtenidos durante la validación del prototipo de robot móvil diferencial. La odometría diferencial y la sintonización del controlador de velocidad se evalúan en simulación y en *hardware* real; el mapeo con `slam_toolbox`, la navegación autónoma y la exploración de fronteras se evalúan sobre el *hardware* real, en el entorno físico disponible. Los resultados complementarios de exploración y mapeo se presentan en el ANEXO 7. El equipamiento de la estación de trabajo y las versiones de sistema operativo, *middleware* y paquetes con que se ejecutan todos los ensayos se consolidan en el ANEXO 3.1. El entorno físico en que se realizan las pruebas sobre *hardware* real se muestra en la Fig. 5.1.

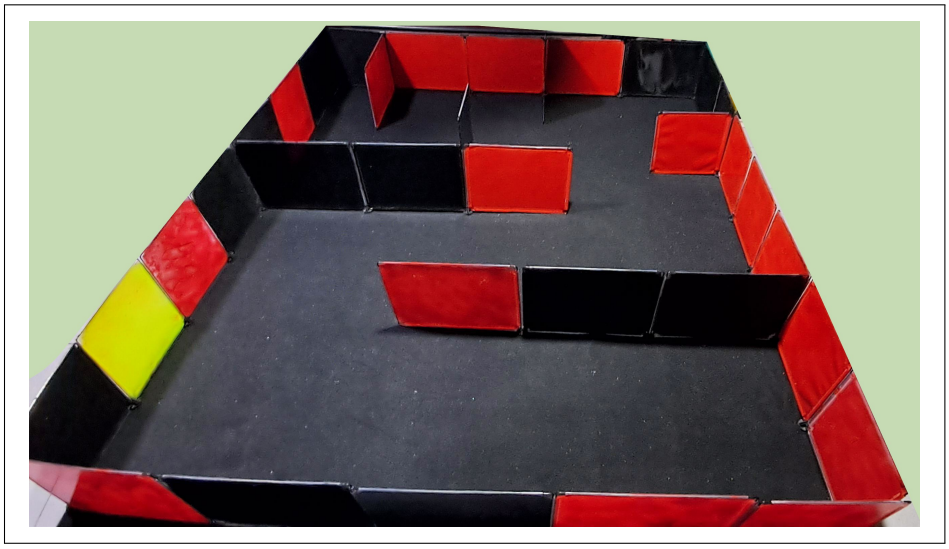


Figura 5.1: Entorno utilizado para las pruebas experimentales en *hardware* real.
Fuente: Elaboración propia.

5.1. Evaluación de odometría y control diferencial

La evaluación rectilínea utiliza una referencia nominal de 2 m. El *script* publica una velocidad lineal de 0.13 m/s durante $2/0.13 \approx 15.4$ s y luego detiene el robot, por lo que la referencia no es una consigna de posición sino el producto de la velocidad comandada por la duración del comando. En *hardware* real, la distancia física se mide con cinta métrica y el error odométrico corresponde a la discrepancia entre la odometría de *encoders* y esa medición. En simulación, la consulta de la pose verdadera a Gazebo Classic no entrega datos durante los ensayos y el *script* registra la propia odometría como distancia de respaldo. Por ello, en simulación no se dispone de una verdad de terreno independiente.

Las pruebas angulares se analizan respecto de una referencia nominal de 360° definida por el protocolo, que no corresponde a una consigna efectiva de vuelta completa; por ello, sus resultados expresan desviación respecto de ese valor nominal y no un error odométrico angular aislado. El desarrollo del protocolo y el límite que origina esta distinción se conservan en el ANEXO 6. Para distinguir ambos fenómenos se utilizan las métricas de las Ecs. (5.1) y (5.2):

$$E_{seg} = \frac{\left|d_{real} - d_{ref}\right|}{d_{ref}} 100\,\%, \tag{5.1}$$

$$E_{odom} = \frac{\left|d_{odom} - d_{real}\right|}{d_{real}} 100\,\%. \tag{5.2}$$

donde d_{ref} es la distancia comandada (m), d_{real} la distancia física medida o verdad de terreno (m), d_{odom} la distancia calculada mediante odometría (m), E_{seg} el error de seguimiento (%) y E_{odom} el error odométrico (%). La desviación angular normalizada se expresa respecto de esa referencia nominal fija mediante la Ec. (5.3):

$$D_{\theta,360} = \frac{\left|\Delta\theta_{obs} - \Delta\theta_{360}\right|}{\left|\Delta\theta_{360}\right|}\,100\,\%, \qquad \Delta\theta_{360} = 360^{\circ}. \tag{5.3}$$

donde $D_{\theta,360}$ es la desviación angular normalizada [%], $\Delta\theta_{360}$ la referencia nominal fija y $\Delta\theta_{obs}$ el giro observado. La Tabla 5.1 consolida las métricas diferenciadas obtenidas en la trayectoria rectilínea de 2 m.

Tabla 5.1: Métricas diferenciadas de seguimiento y odometría en la trayectoria rectilínea de 2 m.

Métrica	**Simulación**	***Hardware* real**
Error medio de seguimiento E_{seg}	3.39 %*	7.18 %
Desviación estándar de E_{seg} (n = 5)	2.04 %*	0.43 %
Error medio odométrico E_{odom}	NA	3.63 %
Desviación estándar de E_{odom} (n = 5)	NA	0.44 %

* En simulación se calcula con la distancia de odometría, porque la pose verdadera de Gazebo Classic no queda registrada.

En *hardware* real, el error medio de seguimiento de 7.18 % corresponde a una distancia física media de aproximadamente 1.856 m frente a los 2 m definidos por el protocolo. Este error debe distinguirse del error odométrico de 3.63 %: el primero caracteriza cuánto se aparta el desplazamiento físico de la referencia comandada, mientras que el segundo compara la estimación de odometría con la distancia realmente recorrida.

Los registros permiten separar el 7.18 % en dos partes, aunque no aislar sus causas físicas. La odometría media de las cinco repeticiones es 1.924 m, es decir, los *encoders* registran un avance 3.8 % menor que los 2 m nominales. Esta parte ya está presente en la medición de los *encoders* y es compatible con el tiempo de subida de las ruedas (Tabla 5.3), porque la referencia se define por tiempo y no por posición. La simulación, que no reproduce la zona muerta del *driver*, presenta una desviación del mismo orden (3.39 %). La parte restante, 0.067 m o 3.4 % de la referencia, corresponde a la sobreestimación de la distancia por la odometría, que expresada respecto de la distancia real es el error odométrico de 3.63 %. Su baja dispersión (0.44 %) indica un sesgo sistemático, compatible con un error de escala en el radio efectivo de rueda o con deslizamiento persistente. Estos

efectos no se separan mediante ensayos independientes. El valor N_e = 1980 no explica el sesgo, porque su diferencia con los promedios medidos haría que la odometría subestimara la distancia entre 0.2 % y 0.6 %.

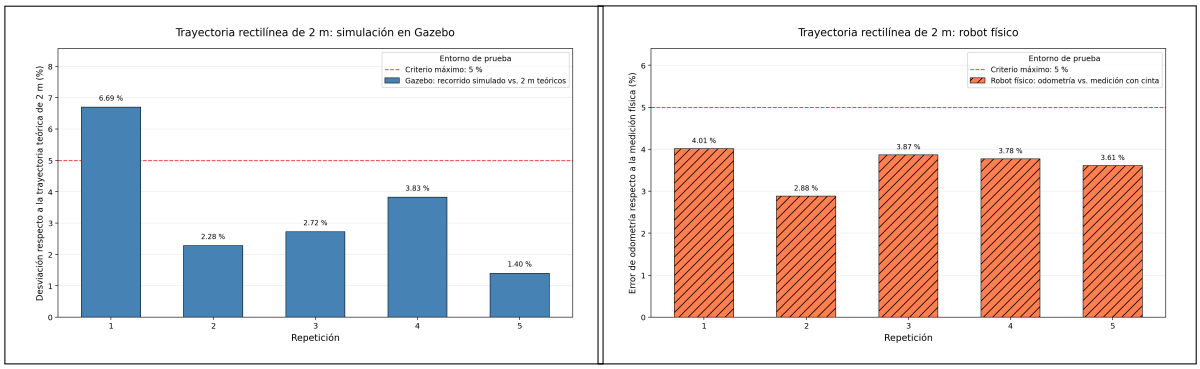
Los resultados correspondientes al protocolo angular se resumen en la Tabla 5.2; su comparación entre simulación y *hardware* se desarrolla en la Sec. 5.5.

Tabla 5.2: Desviación angular respecto de la referencia nominal de 360° (5 repeticiones por entorno).

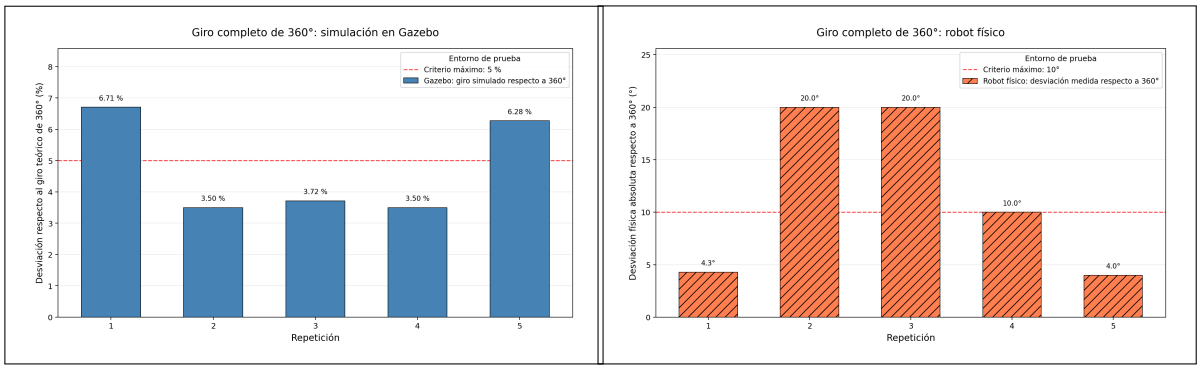
Métrica	**Simulación**	***Hardware* real**
Desviación angular media	17.1°	11.7°
Desviación angular normalizada $D_{\theta,360}$	4.74 %	3.24 %
Desviación estándar	5.8° (1.61 %)	8.0° (2.22 %)

Esta métrica no es un error odométrico angular aislado. En simulación, el giro observado es el giro acumulado de `/odom` al final de un ensayo de duración fija. En *hardware* real, es el giro físico medido al final del ensayo. En ambos casos se compara con la referencia nominal de 360 grados y no con el giro efectivamente comandado. Los giros observados, entre 335.9° y 347.4° en simulación y entre 340° y 380° en *hardware* real, superan los 315° que se obtendrían si el límite de 0.35 rad/s actuara de forma ideal y constante durante todo el ensayo (ANEXO 6). Por tanto, los registros no son coherentes con esa hipótesis y la causa de la diferencia no se determina con los datos disponibles. Un giro comandado a un ángulo fijo y controlado, planteado como trabajo futuro, entregaría un error angular directo. El desglose por repetición de ambos protocolos (trayectoria rectilínea y protocolo angular, en simulación y *hardware* real) se presenta en la Fig. 5.2.

La desviación angular observada integra los efectos del límite de velocidad, la duración del ensayo, la dinámica física y el procedimiento de medición. La incorporación de una IMU se plantea como mejora futura.



(a) Trayectoria rectilínea en simulación. **(b)** Trayectoria rectilínea en *hardware* real.



(c) Protocolo angular en simulación. **(d)** Protocolo angular en *hardware* real.

Figura 5.2: Resultados por repetición de las pruebas rectilíneas y angulares en simulación y *hardware* real.

Fuente: Elaboración propia.

5.2. Evaluación del control de velocidad

El seguimiento en lazo cerrado se verifica en dos momentos del procedimiento de sintonización descrito en la Sec. 4.4.5: con las ganancias sugeridas por PSO y con las ganancias definitivas obtenidas tras el refinamiento. Ambas capturas se realizan sobre el robot real, con una referencia de 0.13 m/s y el mismo protocolo, conservado en el ANEXO 4.11. La Fig. 5.3 presenta la respuesta con las ganancias sugeridas por PSO.

Con las ganancias del PSO, ninguna de las dos ruedas sostiene la refe-

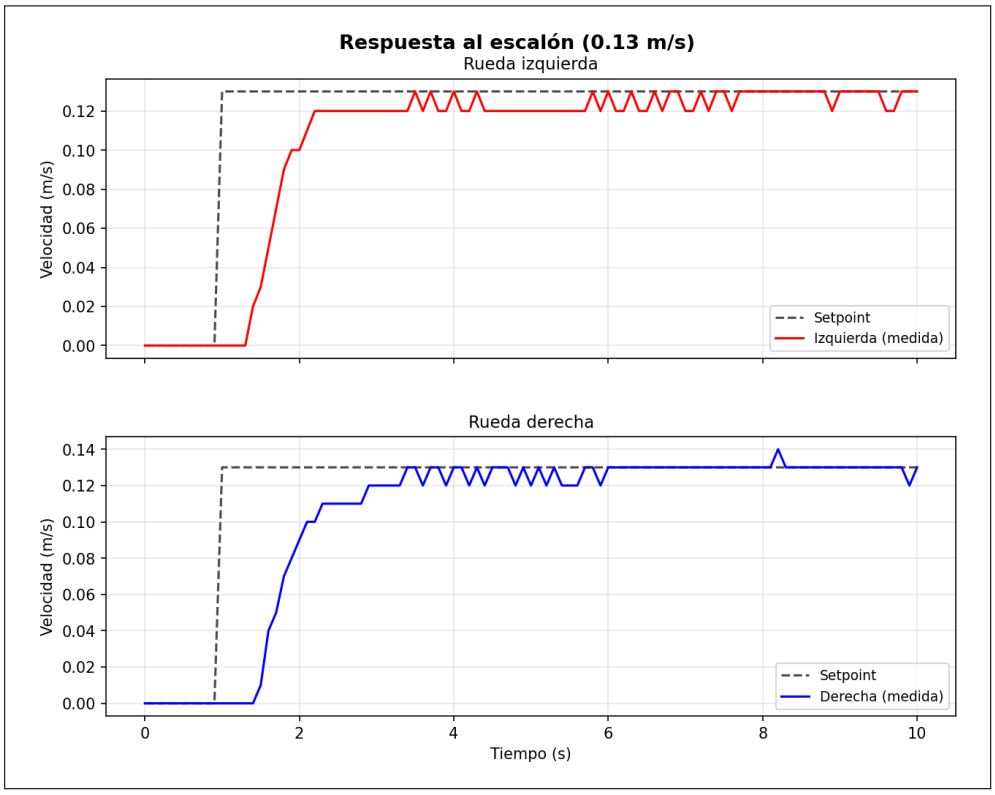


Figura 5.3: Respuesta en lazo cerrado del robot real con las ganancias sugeridas por PSO, antes del refinamiento experimental (referencia de 0.13 m/s).

Fuente: Elaboración propia.

rencia durante todo el ensayo. La rueda izquierda se estabiliza en 0.12 m/s y solo alcanza los 0.13 m/s de forma intermitente, manteniendo un error estacionario cercano a 0.01 m/s. La rueda derecha sube por tramos, con una detención en torno a 0.11 m/s antes de continuar, y recién se mantiene en la referencia después de los 6 s. En ambos casos la subida es más lenta que la obtenida con las ganancias definitivas. La Fig. 5.4 presenta la respuesta obtenida con las ganancias definitivas, bajo el mismo protocolo y la misma referencia.

Con las ganancias definitivas desaparece el error estacionario de la rueda izquierda y la detención intermedia de la derecha, y la referencia se alcanza cerca de un segundo antes. En ambas ruedas, la velocidad medida converge hacia la referencia de 0.13 m/s. La rueda derecha presenta una respuesta más uniforme, mientras que la izquierda muestra pequeñas variaciones discretas alrededor del

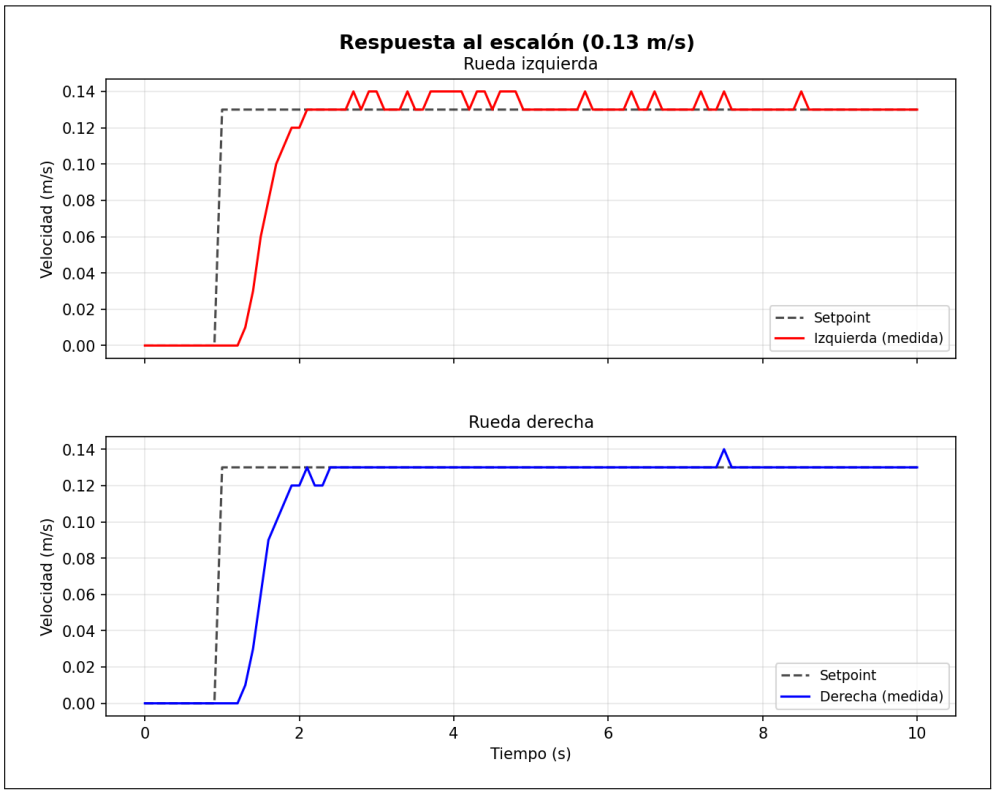


Figura 5.4: Respuesta experimental en lazo cerrado del robot real con las ganancias definitivas y referencia de 0.13 m/s.

Fuente: Elaboración propia.

valor objetivo. La figura constituye una verificación cualitativa del seguimiento sobre el robot real. Los indicadores de la Tabla 5.3 se calculan sobre la captura de la Fig. 5.4, con la referencia de 0.13 m/s. La comparación ilustrativa con el modelo FOPDT y la sintonía Lambda se conserva en el ANEXO 4.2. En esa simulación, las ganancias finales producen sobreimpulsos de 10.7 % a 15.6 %. Esos valores provienen de un modelo que no reproduce la zona muerta del *driver*, las bandas muertas asimétricas ni los tipos enteros del *firmware*, por lo que no se comparan directamente con la Tabla 5.3.

Ambas ruedas alcanzan la referencia sin sobreimpulso apreciable. La velocidad informada por el *firmware* tiene una resolución de 0.01 m/s, cercana a un 8 por ciento de la referencia. Por eso la banda del 2 por ciento queda por debajo de esa resolución y no se reporta un tiempo de establecimiento al 2 por ciento. La

Tabla 5.3: Indicadores de la respuesta al escalón en lazo cerrado, medidos sobre la captura experimental del robot real con referencia de 0.13 m/s.

Indicador	Rueda izquierda	Rueda derecha
Velocidad estacionaria medida ($t \geq 7$ s)	0.128 m/s	0.130 m/s
Error estacionario	0.002 m/s	0.000 m/s
Tiempo de subida 10 %–90 %	0.8 s	1.4 s
Sobreimpulso máximo	Sin sobreimpulso significativo	Sin sobreimpulso significativo

respuesta se estabiliza en torno a la referencia dentro de los primeros segundos, dentro de la resolución de medida. La comparación completa entre las ganancias del *firmware* y la referencia analítica Lambda se conserva en la Fig. A4.1, con carácter ilustrativo.

La ganancia integral difiere entre ruedas, con $K_{i,L} = 0.075$ y $K_{i,R} = 0.001$. El valor izquierdo se fija durante el refinamiento experimental, porque con las ganancias sugeridas por PSO esa rueda mantiene un error estacionario bajo la referencia (Fig. 5.3). El efecto de estas ganancias está condicionado por el almacenamiento entero del estado integral (ANEXO 4.4). En la rueda derecha, el producto $K_{i,R}e_R^{db}[k]$ solo alcanzaría una unidad con errores de 1000 *ticks/frame*, muy superiores a los del ensayo, por lo que su estado integral permanece en cero y $K_{i,R}$ no tiene efecto en la ley ejecutada. En la rueda izquierda, el estado integral solo puede crecer en magnitud en ciclos con $|e_L^{db}[k]| \geq 14$ *ticks/frame*, cerca de 0.045 m/s. En el escalón de 0.13 m/s, de unos 41 *ticks/frame*, esto ocurre principalmente durante el arranque, y el valor acumulado queda sumado al numerador del incremento de control. Por ello, la mejora observada con las ganancias definitivas se informa como un resultado empírico del conjunto de ganancias y no como el efecto de una acción integral convencional.

El OE2 compromete un controlador PID en lazo cerrado sobre las velocidades de las ruedas, y no la optimización de sus ganancias, por lo que el PSO se emplea solo como herramienta de apoyo para obtener un punto de partida heurístico para las tres ganancias. El refinamiento experimental ajusta ese punto de partida, con aumentos de K_p de 28 % y 63 %, reducciones de K_d de 41 % y 45 %

respecto de los valores informados y un aumento de varios órdenes de magnitud en K_i, hasta lograr el seguimiento de la referencia sobre el robot real. El ajuste se explica porque el modelo FOPDT no representa la zona muerta del *driver* ni la aritmética entera del *firmware*, porque el costo evalúa un único escalón sin penalizar el esfuerzo de control y porque los K_d informados superan la cota de 25 que evalúa el costo (ANEXO 4.10). Aun así, las ganancias del PSO dan un lazo estable y más conservador, y quedan en el mismo orden de magnitud que las definitivas. Por tanto, el aporte del PSO es el de una semilla de búsqueda y no el de un ajuste final validado por sí solo. El retardo identificado resulta menor que el período de captura, por lo que no introduce un desplazamiento temporal efectivo en el modelo discreto. En la práctica el modelo se comporta como uno de primer orden. El detalle se conserva en el ANEXO 4.6. En cambio, el análisis de estabilidad considera los escenarios de retardo de 0 y 0.1 s, sin interpretar el segundo como una cota física medida.

5.3. Resultados de mapeo con `slam_toolbox`

El mapeo se evalúa sobre el *hardware* real a lo largo de varias sesiones prácticas dentro del entorno experimental, de las cuales cinco iteraciones producen un mapa válido. La inspección visual de los mapas obtenidos muestra paredes definidas y consistencia global. Esta valoración es cualitativa. El análisis *post hoc* no incorpora una detección automática de cierre de lazo, por lo que ese indicador queda registrado como no determinado en las cinco iteraciones válidas. La métrica utilizada corresponde al índice de cobertura libre de la Ec. (5.4), definido como la proporción de celdas observadas como libres respecto del total de celdas susceptibles de ser exploradas:

$$C_{libre} = 100 \frac{N_{libre}}{N_{libre} + N_{desconocido}}, \tag{5.4}$$

donde N_{libre} es el número de celdas clasificadas como libres, $N_{desconocido}$ el número

de celdas desconocidas y C_{libre} el índice de cobertura libre (%). Las celdas ocupadas no se incluyen en el denominador. La clasificación se realiza sobre la imagen de ocupación generada por `slam_toolbox`, según los umbrales de intensidad indicados en el ANEXO 8. El área libre mapeada se calcula a partir del número de celdas libres mediante la Ec. (5.5):

$$A_{libre} = N_{libre}\, \rho_{map}^2, \tag{5.5}$$

donde A_{libre} es el área libre mapeada [m^2] y ρ_{map} la resolución del mapa [m/celda]. La Tabla 5.4 consolida los indicadores de mapeo obtenidos en las cinco iteraciones válidas sobre *hardware* real, recuperados *post hoc* del *rosbag* y del mapa PGM de cada sesión; su procedencia y alcance se documentan en el ANEXO 8.

Tabla 5.4: Indicadores de mapeo en *hardware* real por iteración (resolución 0.05 m/celda).

Iter.	Dur. (min)	Dist. odom. (m)	Cobertura (%)	Área libre (m^2)
1	3.45	6.305	82.9	3.510
2		Incompleta (sin mapa)		
3	2.16	3.536	53.5	3.540
4	2.27	4.144	54.2	3.478
5	2.50	3.850	67.1	3.483
6	2.30	4.066	53.4	3.478
μ	2.54	4.38	62.22	3.50
σ	0.53	1.10	12.94	0.03

μ: media aritmética. σ: desviación estándar (estimador muestral, n – 1). Ambas sobre las cinco iteraciones válidas.

Estos indicadores caracterizan el mapeo obtenido en las cinco iteraciones válidas de exploración y no constituyen una medida del error absoluto del mapa. El contraste cualitativo de los mapas obtenidos se documenta en la Tabla A8.1 del ANEXO 8. El índice de cobertura libre se calcula sobre la imagen de ocupación completa, cuyos límites no están anclados al recinto físico. Por eso es un indicador interno relativo y no una fracción absoluta del área del recinto. Esto explica

que el área libre mapeada se mantenga casi constante entre iteraciones mientras el índice de cobertura varía de forma amplia, ya que este último depende de cuántas celdas desconocidas quedan dentro de la imagen. Por su mayor estabilidad, el área libre mapeada y el término de cada ejecución por ausencia de fronteras (Sec. 5.4) se toman como evidencia principal de la exploración, y el índice de cobertura como indicador complementario.

La comparación de los indicadores anteriores entre iteraciones informa sobre la repetibilidad del mapeo, con un coeficiente de variación del área libre de 0.8 %, pero no sobre su exactitud, porque una distorsión que se repita en todas las iteraciones no alteraría esa repetibilidad. Por eso, cada mapa se compara con un plano del recinto medido manualmente, mediante la distancia de cada punto de los muros del plano a la celda ocupada más cercana del mapa (Trejos et al., 2022). La distancia media es de 3.4 cm, del orden de la resolución de 0.05 m/-celda. Las mayores diferencias, de 6 a 9 cm, corresponden a los tabiques cortos, y los tres pasillos resultan en total 12 cm más anchos en los mapas que en el plano. El plano, el procedimiento y los resultados se presentan en el ANEXO 8.5. El informe existente también contiene la comparación mapa→plano y percentiles: las medias inversas varían entre 4.09 y 4.94 cm, y el percentil 95 conjunto plano→mapa es de 8.4 cm (Tabla A8.2). Estas medidas complementan la media de 3.4 cm sin convertirla en una medida de exactitud global.

5.4. Navegación autónoma y exploración

Los resultados de navegación y exploración se analizan en dos niveles complementarios. El primero corresponde al resultado de cada ejecución completa de exploración, considerando si el robot opera de forma autónoma, se desplaza por el entorno, incorpora nuevas observaciones y produce un mapa válido. El segundo corresponde al resultado de las metas individuales generadas durante dicha exploración.

Esta distinción permite evaluar por separado el funcionamiento global de la exploración y el resultado que Nav2 informa para cada meta. Por tanto, el resultado de una meta individual no se interpreta directamente como el resultado de toda la ejecución de exploración.

La exploración autónoma produce un mapa válido en cinco de las seis ejecuciones experimentales. En estas cinco ejecuciones, el índice de cobertura libre varía entre 53.4 % y 82.9 %, con una media de 62.2 %, mientras que el área libre mapeada se mantiene entre 3.478 y 3.540 m^2, con una media de 3.50 m^2.

Las cinco ejecuciones válidas terminan de forma automática, sin intervención del operador. En cada una, el nodo `explore_server` informa que no quedan fronteras en el mapa de costos, detiene la exploración y solicita cancelar la meta en curso. El *script* de ensayo detecta ese aviso, guarda el mapa y cierra el registro. La iteración 2, excluida del análisis, no llega a ese término.

En total se generan 60 metas de navegación durante las cinco ejecuciones válidas. De ellas, 59 alcanzan la etapa de ejecución. Respecto de su estado final, 54 metas quedan abortadas, una se completa y cinco no tienen estado terminal en el último mensaje registrado. Estas cinco corresponden a la última meta de cada ejecución, que el explorador cancela automáticamente al detenerse. El registro se cierra antes de que esa cancelación termine, por lo que la meta figura en ejecución o en cancelación.

En este sistema, el estado abortado no equivale a un fallo de navegación. El explorador vuelve a elegir su destino cada 3.3 s, período asociado a `planner_frequency` (Tabla A5.4), y envía una meta nueva cada vez que el punto de la frontera elegida cambia en 0.01 m o más, sin cancelar antes la meta en curso. En la versión de Nav2 empleada, cuando llega una meta nueva mientras otra está en ejecución, la anterior se termina con estado abortado (Nav2 Maintainers, 2021). Por eso, el mismo estado registra tanto los reemplazos hechos por el explorador como los fallos propios de Nav2.

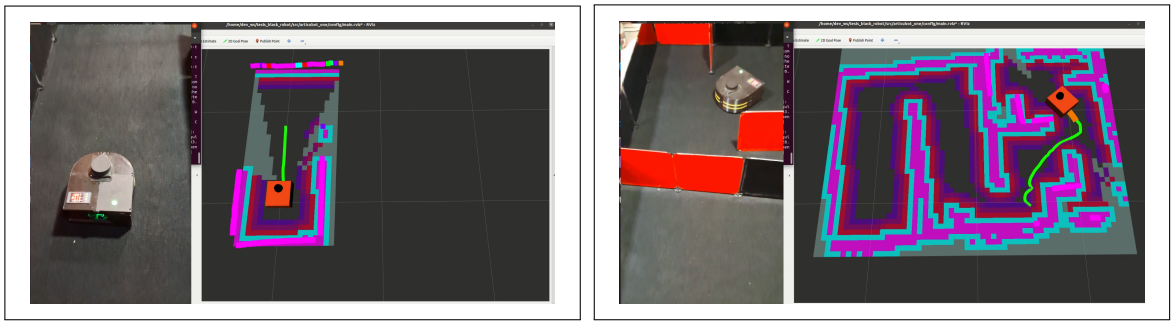
Para estudiar esa asociación se revisa el orden de los mensajes de esta-

do de cada meta, según el procedimiento del ANEXO 8. En 53 de los 54 abortos se observa un mensaje en el que la meta siguiente ya figura mientras la anterior sigue aceptada o en ejecución. Esos abortos se clasifican como asociados temporalmente a reemplazos, de acuerdo con el criterio implementado; no se exige que la nueva meta ya esté en ejecución. El aborto restante corresponde a la primera meta de la iteración 1, que ya figura abortada en el primer mensaje registrado y no puede clasificarse. El tiempo mediano entre la aceptación de la meta anterior y la siguiente es de 3.3 s en las cinco ejecuciones, igual al período de reevaluación del explorador (Tabla A7.1). La coexistencia es compatible con el mecanismo de reemplazo de Nav2, pero no identifica de forma independiente la causa de cada aborto ni permite descartar fallos concurrentes. Tampoco es posible saber si las metas sustituidas habrían llegado a completarse.

En consecuencia, la proporción de metas completadas, 1 de 60 (1.7 %), indica cuántas veces el robot llega a menos de 0.10 m del punto de una frontera antes de que el explorador cambie de destino. No corresponde a una tasa de éxito de la navegación ni de la exploración. La navegación hacia metas fijas, que permitiría medir esa tasa, queda fuera del alcance declarado.

Los diecisiete avisos de lista de fronteras agotada registrados en las cinco iteraciones válidas, once de ellos en la iteración 1, se relacionan con este mismo comportamiento. El explorador agrega a su lista negra el punto de toda meta que termina abortada, también cuando es él quien la reemplaza. Así puede descartar la frontera que sigue y, cuando todas las candidatas quedan en la lista, la vacía para volver a intentarlo. Este efecto de la integración entre `explore_lite` y Nav2 no impide terminar la exploración, pero su efecto sobre la duración del recorrido no se cuantifica. En la Fig. 5.5 se muestran dos instantes representativos de una ejecución de navegación y exploración sobre *hardware* real, desde un mapa parcial hasta la mayor cobertura alcanzada.

Un registro en video (ver en YouTube) de una ejecución de demostración de la navegación y la exploración sobre *hardware* real está disponible en línea (Valderas Neculqueo, 2026). Las capturas de la Fig. 5.5 provienen de ese registro.



(a) Mapa parcial en una etapa temprana de la exploración.

(b) Mayor cobertura con ruta planificada hacia la frontera seleccionada.

Figura 5.5: Ejecución de navegación autónoma y exploración por fronteras sobre *hardware* real: avance desde un mapa parcial hasta una representación de mayor cobertura, con la ruta planificada por Nav2 (trazo verde).

Fuente: Elaboración propia.

La mejor iteración alcanza un índice de cobertura libre de 82.9 %. El radio de inflación de 0.15 m aplicado en el recinto reducido (Sec. 4.5.4) no se evalúa por separado, por lo que su efecto sobre la navegación no se cuantifica. En conjunto, los ensayos verifican la integración funcional, cuantifican la cobertura y documentan el resultado de las metas de exploración según la Tabla A7.1. Las cinco iteraciones finalizan de forma automática al no quedar fronteras, y cada una genera un mapa válido.

En las cinco ejecuciones válidas, Nav2 conduce al robot durante 12.7 min y 21.9 m de recorrido odométrico, sin intervención manual. El registro no permite cuantificar por separado los fallos propios de Nav2. Estos indicadores describen la exploración, pero no miden si el robot llega a un destino dado ni con qué error. La navegación dirigida con metas fijas definidas por el operador, distinta de las metas que genera el explorador, queda fuera del alcance declarado en Alcances y limitaciones y no se ensaya, por lo que este trabajo no informa tasa de éxito ni error de posición final para ese caso. El protocolo que permitiría medirla se describe en el ANEXO 8.3. La Tabla 5.5 sintetiza el estado de validación alcanzado por cada subsistema.

Tabla 5.5: Estado de validación por subsistema.

Subsistema	Estado	Observación
Odometría	Validada cuantitativamente en *hardware* real (recta de 2 m, n = 5)	Error odométrico medio de 3.63 %, con intervalo de confianza al 95 % de 3.08 % a 4.18 %. En simulación no se registra verdad de terreno independiente. El protocolo angular no mide error odométrico angular
Control de velocidad	Evaluado experimentalmente y verificado analíticamente	Indicadores de la Tabla 5.3 calculados sobre la captura real en lazo cerrado a 0.13 m/s. Estabilidad y márgenes verificados con el modelo FOPDT identificado (Tabla A4.5). Un único punto de operación
SLAM	Verificado funcionalmente y caracterizado cuantitativamente	Cobertura y área libre cuantificadas en cinco iteraciones válidas. Posición de los muros comparada con el plano medido del recinto, con una distancia media de 3.4 cm (ANEXO 8.5)
Nav2	Verificada funcionalmente	Ejecuta 59 metas del explorador durante 21.9 m de recorrido. De los 54 abortos, 53 cumplen el criterio temporal de asociación a reemplazos (Tabla A7.1); el resultado de las metas no mide la tasa de éxito de navegación. El protocolo con metas fijas queda fuera del alcance declarado (Alcances y limitaciones)
Exploración autónoma	Verificada funcionalmente en *hardware* real	Cinco de seis ejecuciones terminan automáticamente al no quedar fronteras, sin intervención manual y con un mapa válido. Cobertura libre media de 62.2 % y área libre mapeada media de 3.50 m^2
Localización	Resuelta por `slam_toolbox` durante el mapeo en línea y verificada funcionalmente en las cinco iteraciones válidas	La localización sobre un mapa almacenado con AMCL queda fuera del alcance declarado (Alcances y limitaciones) y no se ensaya

5.5. Coherencia entre simulación y *hardware*

En la trayectoria rectilínea de 2 m, las desviaciones de la Tabla 5.1 no miden la misma magnitud en ambos entornos. La de simulación se calcula con la propia odometría y la de *hardware* real con la distancia física medida, de modo que el error odométrico solo puede separarse del error de seguimiento en *hardware* real. En el protocolo angular, la desviación media respecto de la referencia nominal es mayor en simulación que en *hardware* real, con una dispersión menor (Tabla 5.2).

El mapeo y la exploración se evalúan con indicadores únicamente sobre *hardware* real. En simulación se construyen mapas en sesiones previas a los ensayos en *hardware*, sin calcular indicadores, por lo que el contraste entre ambos entornos es solo cualitativo y se basa en inspección visual (Tabla A8.1). Las limitaciones metodológicas de cada ensayo se detallan en el ANEXO 8. Por su parte, la caracterización cuantitativa del consumo energético y de la carga computacional no forma parte de los ensayos realizados y se plantea como trabajo futuro; su alcance se precisa en el ANEXO 8.

5.6. Alcance estadístico de los ensayos

Los ensayos sobre *hardware* real se repiten cinco veces en odometría, seis veces en exploración, de las cuales cinco son válidas, y una vez por conjunto de ganancias en el control de velocidad. Con el intervalo de confianza al 95 % de la media (Devore et al., 2021, Sec. 8.2), el error odométrico queda entre 3.08 % y 4.18 %, el área libre mapeada entre 3.46 y 3.53 m^2 y la distancia al plano del recinto entre 3.1 y 3.7 cm, mientras que la cobertura libre, de 46.2 % a 78.3 %, y la desviación angular, de 1.8° a 21.6°, quedan poco acotadas. La proporción de exploraciones válidas solo permite afirmar con 95 % de confianza que supera el 35.9 %, según el intervalo de Clopper y Pearson (Vincent et al., 2024). Como contraste, la prueba binomial exacta unilateral de cinco éxitos en seis ejecuciones,

bajo la hipótesis nula de una proporción de éxito de 50 %, entrega p = 0.109375, por lo que no permite rechazar esa hipótesis al nivel de 0.05 (Devore et al., 2021, Cap. 9). Como con seis ejecuciones solo un resultado sin fallas permitiría rechazarla, el número de ensayos se considera bajo. Por eso, y por el alcance de un único recinto, los resultados se presentan como una verificación funcional en el entorno ensayado.

Se considera admisible un mínimo de 6 a 10 repeticiones por ensayo para obtener intervalos con una variabilidad aceptable, con 10 por condición como objetivo de partida y un total sujeto a la dispersión y a la precisión requerida. Ese criterio es coherente con las 6 a 15 repeticiones que la guía Eurachem considera necesarias para estimar una desviación estándar (Cantwell, 2025, Sec. 5.3.2), aunque no se refiere a ensayos robóticos. Alcanzarlo queda como trabajo futuro. El cálculo, los intervalos de todos los indicadores, las repeticiones necesarias para acotarlos mejor y su comparación con la literatura se presentan en el ANEXO 8.6.

Conclusión del capítulo

Los resultados muestran un error odométrico medio de 3.63 % en trayectorias rectas sobre *hardware* real y una desviación angular media de 11.7° respecto de la referencia nominal del protocolo. La respuesta en lazo cerrado de la Fig. 5.4 se mide en *hardware*, y sobre esa misma captura se calculan los indicadores de la Tabla 5.3, que muestran seguimiento de la referencia sin sobreimpulso apreciable. Por su parte, el sistema de mapeo con `slam_toolbox` genera mapas utilizables en *hardware* real. La exploración por fronteras alcanza un índice medio de cobertura libre de 62.2 % (σ = 12.9) en cinco iteraciones, con un área libre mapeada media de 3.50 m^2, y sus mapas quedan a una distancia media de 3.4 cm del plano medido del recinto. La integración funcional y la cobertura quedan verificadas con un número reducido de repeticiones. La navegación hacia metas fijas definidas por el operador queda fuera del alcance declarado.

CONCLUSIONES

El presente trabajo tiene como objetivo diseñar e implementar un prototipo de robot móvil diferencial capaz de generar mapas 2D, ejecutar navegación autónoma y realizar exploración basada en fronteras mediante ROS 2 Foxy. El desarrollo integra diseño mecánico, electrónica de potencia, *firmware* de bajo nivel, gemelo digital en Gazebo Classic, mapeo con `slam_toolbox`, navegación con Nav2 y exploración autónoma.

En relación con la plataforma física, se construye un robot diferencial de bajo costo con la configuración descrita en el Capítulo 4, en la que el Arduino ejecuta la lectura de *encoders* y el control de cada rueda, la Raspberry Pi ejecuta la interfaz de *hardware*, el controlador diferencial y la adquisición del LIDAR, y la estación de trabajo ejecuta SLAM, navegación, exploración y supervisión.

El gemelo digital desarrollado en Gazebo Classic permite verificar la geometría, la cinemática, la cadena de transformaciones y la interfaz de control antes de operar el robot físico. En simulación, la desviación media respecto de la referencia nominal de 2 m es de 3.39 %, calculada con la propia odometría porque la pose verdadera de Gazebo Classic no queda registrada durante los ensayos. Ese valor no separa el error odométrico del error de seguimiento. En *hardware* real, el error odométrico medio es de 3.63 % y el error físico de seguimiento es de 7.18 %. La selección del *backend* y su trazabilidad se documentan en el ANEXO 2.

Respecto del control de bajo nivel, el *firmware* ejecuta de forma independiente para cada rueda la ley de control heredada del *firmware* original, que corresponde a un controlador PI en forma incremental con la acción proporcional sobre la medición, más un término integral adicional, y cuyas ganancias no cumplen la función que indica su nombre. El análisis del lazo discreto con el modelo FOPDT identificado muestra estabilidad de los modelos lineales considerados, con márgenes de fase de 53° a 57° sin retardo y de 41° a 44° en el escenario de retardo de 0.1 s, que no se interpreta como una cota física medida. La Fig. 5.4 corresponde

a una adquisición experimental en lazo cerrado, y los indicadores de la Tabla 5.3 se calculan sobre esa misma respuesta medida. Ambas ruedas alcanzan la referencia sin sobreimpulso apreciable. Los estados internos, la temporalidad y las aproximaciones numéricas se documentan en los anexos de implementación y sintonización.

El mapeo con `slam_toolbox` permite construir mapas 2D en simulación y *hardware* real. Los indicadores de cobertura libre y de área mapeada se cuantifican sobre *hardware* real durante las iteraciones de exploración. Frente a un plano medido del recinto, los mapas tienen una distancia media de 3.4 cm. La navegación con Nav2 queda integrada mediante planificación global, control local, mapas de costo y acciones de recuperación. La exploración por fronteras se ejecuta en cinco iteraciones válidas, que terminan automáticamente al no quedar fronteras y sin intervención manual, con un índice medio de cobertura libre de 62.2 % (σ = 12.9). De las 60 metas generadas, 54 quedan abortadas y una se completa. De esos abortos, 53 cumplen el criterio temporal de asociación a reemplazos del explorador, que reevalúa su destino cada 3.3 s. El mecanismo de reemplazo de Nav2 puede terminar la meta anterior como abortada; el criterio no determina de forma independiente la causa de cada caso. El detalle metodológico se conserva en el ANEXO 8. El cumplimiento de los objetivos específicos se resume de la siguiente forma:

OE 1 – Construcción y gemelo digital: *Cumplido.* Se diseña y construye la plataforma diferencial física y se verifica su gemelo digital con el *backend* de `ros2_control` ejecutado en Gazebo Classic. La verificación es funcional, según los criterios de la Tabla A2.2, y no incluye una comparación cuantitativa entre simulación y *hardware* sobre una misma variable medida, porque la distancia simulada se obtiene de la propia odometría.

OE 2 – Control de bajo nivel: *Cumplido parcialmente.* Se implementan la lectura de *encoders*, la estimación de los estados de rueda y el control de velocidad en el *firmware*; la posición y velocidad de rueda se exportan mediante `DiffDriveArduino`, y la odometría se integra en `diff_drive_controller`. La estabilidad de la ley ejecutada, un PI incremental heredado de

`ROSArduinoBridge`, se verifica sobre su idealización lineal con el modelo FOPDT identificado. Sin embargo, ese análisis no incluye los efectos no lineales del *firmware* y el desempeño se evalúa en un único punto de operación, como se detalla en la Sec. 4.4.3 y en la Sec. 5.2.

OE 3 – Localización, mapeo y navegación: *Cumplido parcialmente.* El mapeo y la localización durante la construcción del mapa se implementan y verifican funcionalmente en *hardware* real, junto con la caracterización cuantitativa de cobertura y área libre mapeada. Nav2 se integra y conduce al robot hacia las metas del explorador en las cinco ejecuciones válidas, con 21.9 m de recorrido autónomo. De los 54 abortos, 53 cumplen el criterio temporal de asociación a reemplazos, sin que ello permita descartar fallos concurrentes. Por tanto, el resultado de las metas no permite medir la tasa de éxito de la navegación. El carácter parcial proviene de que no se obtiene un indicador cuantitativo del desempeño de la navegación.

OE 4 – Exploración autónoma: *Cumplido.* La estrategia de exploración por fronteras se integra y ejecuta sobre el robot real, y genera 60 metas hacia regiones no exploradas sin intervención manual. Cinco de las seis ejecuciones terminan de forma automática por el criterio del propio explorador, al no quedar fronteras en el mapa de costos, y producen mapas válidos con un índice medio de cobertura libre de 62.2 % y un área libre mapeada media de 3.50 m^2. En la ejecución restante el robot no se desplaza y la causa no se determina. La exploración queda verificada funcionalmente y caracterizada, y sus mapas se comparan con un plano medido del recinto. Su principal limitación es que el explorador agrega a la lista negra las metas que él mismo reemplaza, efecto de la integración con Nav2 que no impide terminar la exploración.

Respecto de la pregunta de investigación planteada en la Introducción, los resultados muestran que es posible construir un prototipo diferencial capaz de generar mapas 2D y ejecutar un proceso de exploración autónoma en el entorno ensayado. Cinco de las seis ejecuciones experimentales producen mapas válidos, lo que permite verificar funcionalmente la integración entre exploración, navega-

ción y mapeo. Con seis ejecuciones, esa proporción se considera descriptiva y no se generaliza como tasa de desempeño (Sec. 5.6).

El resultado de las metas individuales no contradice esa conclusión, porque 53 de los 54 abortos se asocian temporalmente a reemplazos del propio explorador. Sí deja pendiente medir cuantitativamente el desempeño de la navegación, lo que requiere un protocolo con metas fijas.

Limitaciones

- **Deriva odométrica:** la odometría basada solo en *encoders* acumula error, especialmente en giros y trayectorias largas.
- **Espacio de pruebas reducido:** el recinto disponible limita la ejecución de recorridos extensos y condiciona la geometría de los mapas de costo.
- **Registro de navegación:** los contadores automáticos de la sesión no identifican metas únicas, por lo que el resultado de las metas de exploración se reconstruye *a posteriori* desde el flujo de estado de la acción. No se ejecuta un protocolo de navegación dirigida con metas fijas, por lo que no se dispone de tasa de éxito ni de error de posición final para ese caso.
- **Exactitud de los mapas:** se evalúa contra un plano medido con huincha. El espesor de los muros explica a lo sumo 4 de los 12 cm en que los pasillos resultan más anchos, y el resto puede deberse a la medición o al mapa.
- **Métricas del control de velocidad:** los indicadores de la Tabla 5.3 provienen de la captura experimental en *hardware* real, mientras que los sobreimpulsos de 10.7 % a 15.6 % provienen de una simulación ilustrativa sobre un modelo FOPDT reidentificado. No deben interpretarse como una única prueba.
- **Modelo y sintonización:** el período de captura de 0.1 s no resuelve el retardo del modelo FOPDT, el único escalón de PWM no representa la zona muerta

del *driver* y la simulación del costo del PSO no replica la aritmética entera del *firmware*.

- **Fidelidad de la simulación:** Gazebo Classic no reproduce completamente fricción, zona muerta del *driver*, ruido sensorial ni latencias reales.
- **Distribución en fin de vida:** ROS 2 Foxy ya no recibe soporte oficial.
- **Validez externa:** los resultados provienen de un único recinto, cuya área libre mapeada media es de 3.50 m^2, y de cinco iteraciones, por lo que no se generalizan a entornos mayores o de geometría distinta. Los intervalos de confianza son amplios en la cobertura libre, la desviación angular y la proporción de exploraciones válidas (Tabla A8.4).
- **Ley de control de bajo nivel:** el *firmware* ejecuta la ley acumulativa heredada de `ROSArduinoBridge`, en la que K_p actúa como ganancia integral, K_d como ganancia proporcional sobre la medición y K_i como segunda integración (Ec. (4.4)). Su estabilidad se verifica con un análisis lineal, y no se compara con un PID incremental estándar ni se caracteriza a velocidades bajas.
- **Particularidades de implementación conservadas:** la lectura no atómica de los *encoders*, la posición global del PSO copiada desde la posición actual y sin semilla fija, y el costo de fronteras no homogéneo en unidades se documentan tal como se ejecutan. Su efecto sobre los resultados no se cuantifica, salvo el de la copia de la posición global, que no altera el resultado (ANEXO 4.8).
- **Integración entre explorador y Nav2:** el explorador envía una meta nueva sin cancelar la anterior, y Nav2 informa la meta reemplazada como abortada. Por eso el resultado de las metas no mide la tasa de éxito de la navegación, y el explorador agrega a su lista negra fronteras que no fallan. El efecto sobre la duración de la exploración no se cuantifica.

Trabajo futuro

1. Ejecutar el protocolo cuantitativo de navegación con metas fijas en un entorno de mayor superficie, registrando cada meta y su resultado de forma unívoca, con la tasa de éxito y el error de posición final.
2. Integrar una IMU mediante el paquete de fusión sensorial `robot_localization` para reducir la deriva angular.
3. Migrar el sistema a una distribución mantenida de ROS 2 y portar en el mismo paso el gemelo digital a una versión vigente de Gazebo, dado que Gazebo Classic ya no recibe soporte.
4. Evaluar la navegación en espacios estrechos con una configuración acorde a la geometría del entorno.
5. Registrar por separado los reemplazos de meta y los fallos de Nav2, cancelando la meta en curso antes de enviar la siguiente o conservando el registro de `bt_navigator`, y evaluar el efecto de `planner_frequency` sobre la duración de la exploración.
6. Extender la evaluación del control de velocidad a otras referencias, en especial a velocidades bajas y cambios de sentido, y compararlo con un PID incremental estándar, incorporando un estado integral en punto flotante y lectura atómica de *encoders*.
7. Extender el sistema a la localización sobre un mapa almacenado mediante AMCL, comparando la pose estimada con posiciones marcadas físicamente en el recinto.
8. Repetir el protocolo angular con un giro controlado a un ángulo fijo y verificar la velocidad angular efectiva registrada en `/odom`.
9. Registrar la pose verdadera de Gazebo Classic en los ensayos simulados para separar el error odométrico del error de seguimiento en simulación.

10. Medir el recinto con un instrumento de mayor precisión y repetir con esas medidas la comparación de los mapas obtenidos en *hardware* real con el plano del recinto.

11. Alcanzar el mínimo admisible de 6 a 10 repeticiones por ensayo, con 10 por condición como objetivo de partida, y ajustar el total según la dispersión y la precisión requerida (ANEXO 8.6).

12. Repetir la identificación FOPDT con un muestreo más rápido y escalones de distinta amplitud, y reformular el PSO con la aritmética entera del *firmware*, salida recortada y semilla fija.

BIBLIOGRAFÍA

1. Alvarez, C. (2021). *m-explore-ros2: ROS 2 port of m-explore (explore_lite)* [Versión 1.0.0; primer *port* funcional para ROS 2 Foxy, publicado el 1 de agosto de 2021]. Consultado el 12 de septiembre de 2026, desde https://github.com/robo-friends/m-explore-ros2

2. Amadi, C. A., Mbanisi, K., & Smit, W. J. (2024, septiembre). *An Introduction to the ros2_control framework using a low cost Differential Drive Robot* [*Preprint* no arbitrado. El robot Lidarbot descrito en el trabajo se distribuye en https://github.com/TheNoobInventor/lidarbot]. https://doi.org/10.13140/RG.2.2.15748.54408

3. Åström, K. J., & Hägglund, T. (2009). *Control PID avanzado* (S. Dormido Bencomo & J. L. Guzmán Sánchez, Trad.). Pearson Educación. (Fecha inicial de publicación 2006)

4. Autoware Foundation. (2026). *Autoware Documentation* [Documentación oficial vigente; la versión estable 1.9.0 fue publicada en 2026]. Consultado el 12 de septiembre de 2026, desde https://docs.autoware.org/main/home/

5. Bitriá, R., & Palacín, J. (2022). Optimal PID Control of a Brushed DC Motor with an Embedded Low-Cost Magnetic Quadrature Encoder for Improved Step Overshoot and Undershoot Responses in a Mobile Robot Application. *Sensors*, *22*(20), 7817. https://doi.org/10.3390/s22207817

6. Cantwell, H. (Ed.). (2025). *Eurachem Guide: The Fitness for Purpose of Analytical Methods. A Laboratory Guide to Method Validation and Related To-*

pics (3.ª ed.). Eurachem. Consultado el 22 de octubre de 2026, desde https://www.eurachem.org/images/stories/Guides/pdf/MV_guide_3rd_ed_V1_EN.pdf

7. Casini, M., Garulli, A., Giannitrapani, A., & Vicino, A. (2010). A Matlab-Based Remote Lab for Multi-Robot Experiments. *IFAC Proceedings Volumes*, *42*(24), 162-167. https://doi.org/10.3182/20091021-3-JP-2009.00031
8. Clerc, M., & Kennedy, J. (2002). The Particle Swarm—Explosion, Stability, and Convergence in a Multidimensional Complex Space. *IEEE Transactions on Evolutionary Computation*, *6*(1), 58-73. https://doi.org/10.1109/4235.985692
9. Coppelia Robotics AG. (2025, 14 de mayo). *CoppeliaSim Version History: Version 4.10.0* [Documentación oficial de la versión 4.10.0]. Consultado el 12 de septiembre de 2026, desde https://manual.coppeliarobotics.com/en/versionInfo.htm
10. Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python* (3.ª ed., Vol. 146). Springer. https://doi.org/10.1007/978-3-031-06469-2
11. Cyberbotics Ltd. (2025). *Webots Reference Manual: Release R2025a* [Documentación oficial de Webots, *release* R2025a]. Consultado el 12 de septiembre de 2026, desde https://cyberbotics.com/doc/reference/index?version=released
12. Devore, J. L., Berk, K. N., & Carlton, M. A. (2021). *Modern Mathematical Statistics with Applications* (3.ª ed.). Springer. https://doi.org/10.1007/978-3-030-55156-8

13. Ferreira, M., Moreira, L., & Lopes, A. (2023). Differential Drive Kinematics and Odometry for a Mobile Robot Using TwinCAT. *Electronic Research Archive*, *31*(4), 1789-1803. https://doi.org/10.3934/era.2023092

14. Foote, T., & Purvis, M. (2010). *REP 103: Standard Units of Measure and Coordinate Conventions* [Norma vigente del ecosistema ROS, sin revisión posterior]. Consultado el 1 de mayo de 2026, desde https://www.ros.org/reps/rep-0103.html

15. Francis, A., Pérez-D'Arpino, C., Li, C., Xia, F., Alahi, A., Alami, R., Bera, A., Biswas, A., Biswas, J., Chandra, R., Chiang, H.-T. L., Everett, M., Ha, S., Hart, J., How, J. P., Karnan, H., Lee, T.-W. E., Manso, L. J., Mirsky, R., … Martín-Martín, R. (2025). Principles and Guidelines for Evaluating Social Robot Navigation Algorithms. *ACM Transactions on Human-Robot Interaction*, *14*(2), 1-65. https://doi.org/10.1145/3700599

16. Gökçe, B., Koca, Y. B., Aslan, Y., & Gökçe, C. O. (2021). Particle Swarm Optimization-Based Optimal PID Control of an Agricultural Mobile Robot. *Comptes rendus de l'Académie bulgare des Sciences*, *74*(4), 568-575. https://doi.org/10.7546/CRABS.2021.04.12

17. Grisetti, G., Stachniss, C., & Burgard, W. (2007). Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters. *IEEE Transactions on Robotics*, *23*(1), 34-46. https://doi.org/10.1109/TRO.2006.889486

18. Hess, W., Kohler, D., Rapp, H., & Andor, D. (2016). Real-Time Loop Closure in 2D LIDAR SLAM. *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 1271-1278. https://doi.org/10.1109/ICRA.2016.7487258

19. Home Brew Robotics Club. (s.f.). *ros_arduino_bridge: ROS + Arduino = Robot* [*Firmware* `ROSArduinoBridge` para *encoders*, motores y PID. Adaptado en este trabajo a partir de la bifurcación mantenida por Josh Newans]. Consultado el 12 de septiembre de 2026, desde https://github.com/hbrobotics/ros_arduino_bridge

20. Hörner, J. (2017). *m-explore: ROS packages for multi-robot exploration (explore_lite)* [Proyecto original para ROS 1; la implementación utilizada en esta tesis corresponde al *port* para ROS 2 citado por separado]. Consultado el 22 de septiembre de 2025, desde https://github.com/hrnr/m-explore

21. Iqbal, K. (2023, 19 de junio). *1.4: An Electro-Mechanical System Model* [Sección del libro abierto *Introduction to Control Systems*, University of Arkansas at Little Rock]. Engineering LibreTexts. Consultado el 18 de octubre de 2026, desde https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Introduction_to_Control_Systems_(Iqbal)/01%3A_Mathematical_Models_of_Physical_Systems/1.04%3A_An_Electro-Mechanical_System_Model

22. Karaman, S., & Frazzoli, E. (2011). Sampling-Based Algorithms for Optimal Motion Planning. *The International Journal of Robotics Research*, *30*(7), 846-894. https://doi.org/10.1177/0278364911406761

23. Kato, S., Tokunaga, S., Maruyama, Y., Maeda, S., Hirabayashi, M., Kitsukawa, Y., Monrroy, A., Ando, T., Fujii, Y., & Azumi, T. (2018). Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems. *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*, 287-296. https://doi.org/10.1109/ICCPS.2018.00035

24. Kohlbrecher, S., von Stryk, O., Meyer, J., & Klingauf, U. (2011). A Flexible and Scalable SLAM System with Full 3D Motion Estimation. *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 155-160. https://doi.org/10.1109/SSRR.2011.6106777

25. LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. https://doi.org/10.1017/CBO9780511546877

26. LCM Project. (s.f.). *LCM Documentation* [Documentación oficial]. Consultado el 17 de julio de 2026, desde https://lcm-proj.github.io/lcm/

27. Liu, C., Zhang, D., Liu, W., Sui, X., Huang, Y., Ma, X., Yang, X., & Wang, X. (2025). Enhancing autonomous exploration for robotics via real time map optimization and improved frontier costs. *Scientific Reports*, *15*, 12261. https://doi.org/10.1038/s41598-025-97231-9

28. Ma, Y., Meng, F., & Xiong, S. (2024). Design and Implementation of a Two-Wheeled Self-Balancing Car Using a Fuzzy Kalman Filter. *Applied Sciences*, *14*(12), 5296. https://doi.org/10.3390/app14125296

29. Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, *7*(66), eabm6074. https://doi.org/10.1126/scirobotics.abm6074

30. Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, *6*(61), 2783. https://doi.org/10.21105/joss.02783

31. Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern & capable mobile ro-

botics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, *168*, 104493. https://doi.org/10.1016/j.robot.2023.104493

32. marcuslanzoni. (2020, junio). *Vacuum Robot* [Modelo tridimensional publicado en Thingiverse]. Consultado el 22 de septiembre de 2025, desde https://www.thingiverse.com/thing:4459513

33. Maxim, A., & De Keyser, R. (2023). Estimation of First Order Plus Dead Time and Second Order Plus Dead Time models from noisy step response data. *Asian Journal of Control*, *25*(3), 1791-1804. https://doi.org/10.1002/asjc.2999

34. Meeussen, W. (2010). *REP 105: Coordinate Frames for Mobile Platforms* [Norma vigente del ecosistema ROS, sin revisión posterior]. Consultado el 1 de mayo de 2026, desde https://www.ros.org/reps/rep-0105.html

35. Michel, O. (2004). Cyberbotics Ltd. Webots: Professional Mobile Robot Simulation. *International Journal of Advanced Robotic Systems*, *1*(1), 39-42. https://doi.org/10.5772/5618

36. Mori, Y., Yokoyama, S., Yamashita, T., Kawamura, H., & Mori, M. (2024). Development of Assistance Level Adjustment Function for Variable Load on a Forearm-Supported Robotic Walker. *Sensors*, *24*(19), 6456. https://doi.org/10.3390/s24196456

37. Nav2 Maintainers. (s.f.). *bringup_launch.py* [Código fuente de Nav2 para ROS 2 Foxy, rama foxy-devel]. Consultado el 22 de octubre de 2026, desde https://github.com/ros-navigation/navigation2/blob/foxy-devel/nav2_bringup/bringup/launch/bringup_launch.py

38. Nav2 Maintainers. (2021). *Código fuente de Nav2 para ROS 2 Foxy, rama foxy-devel*. Consultado el 20 de septiembre de 2026, desde https://github.com/ros-navigation/navigation2/tree/foxy-devel

39. Nav2 Maintainers. (2025). *Navigation2 Stack (Nav2) Documentation*. Consultado el 12 de septiembre de 2026, desde https://docs.nav2.org/

40. Newans, J. (2022a). *`diffdrive_arduino`: A ros2_control hardware interface for differential drive robots* [Interfaz de *hardware* entre `diff_drive_controller` y un Arduino con *firmware* `ros_arduino_bridge`]. Consultado el 12 de septiembre de 2026, desde https://github.com/joshnewans/diffdrive_arduino

41. Newans, J. (2022b). *`ros_arduino_bridge`: ROS + Arduino = Robot* [Bifurcación de `hbrobotics/ros_arduino_bridge`. Base del *firmware* `ROSArduinoBridge` adaptado en este trabajo]. Consultado el 19 de septiembre de 2026, desde https://github.com/joshnewans/ros_arduino_bridge

42. Newans, J. (2023). *`articubot_one`: ROS 2 mobile robot package* [Proyecto *Articulated Robotics*. Base del paquete de descripción, control, SLAM y navegación adaptado en este trabajo]. Consultado el 12 de septiembre de 2026, desde https://github.com/joshnewans/articubot_one

43. Ngoc, H. T., Vinh, N. N., Nguyen, N. T., & Quach, L.-D. (2023). Efficient Evaluation of SLAM Methods and Integration of Human Detection with YOLO Based on Multiple Optimization in ROS2. *International Journal of Advanced Computer Science & Applications*, *14*(11), 300-310. https://doi.org/10.14569/IJACSA.2023.0141130

44. NVIDIA Corporation. (2025). *NVIDIA Isaac Sim*. Consultado el 12 de septiembre de 2026, desde https://developer.nvidia.com/isaac/sim/

45. Open Motion Planning Library Project. (2026). *The Open Motion Planning Library (OMPL)* [Documentación oficial; versión 2.0.2 publicada el 14 de agosto de 2026]. Consultado el 12 de septiembre de 2026, desde https://ompl.kavrakilab.org/

46. Open Robotics. (2024). *Gazebo Simulator* [Sitio del Gazebo moderno, sucesor de Gazebo Classic]. Consultado el 22 de septiembre de 2025, desde https://gazebosim.org/

47. Open Robotics. (2025a). *Gazebo Classic Migration*. Consultado el 12 de septiembre de 2026, desde https://gazebosim.org/docs/latest/gazebo_classic_migration/

48. Open Robotics. (2025b). *ROS 1 Noetic Ninjemys Goes End-of-Life 2025-05-31*. Consultado el 17 de julio de 2026, desde https://www.ros.org/blog/noetic-eol/

49. Open Source Robotics Foundation. (2025). *Gazebo Classic* [Sitio oficial de Gazebo Classic, con el aviso de fin de soporte de enero de 2025]. Consultado el 12 de septiembre de 2026, desde https://classic.gazebosim.org/

50. Placed, J. A., Strader, J., Carrillo, H., Atanasov, N., Indelman, V., Carlone, L., & Castellanos, J. A. (2023). A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers. *IEEE Transactions on Robotics*, *39*(3), 1686-1705. https://doi.org/10.1109/TRO.2023.3248510

51. Raj, R., & Kos, A. (2024). Discussion on Different Controllers Used for the Navigation of Mobile Robot. *International Journal of Electronics and Telecommunications*, *70*(1), 229-239. https://doi.org/10.24425/ijet.2024.149535

52. Ran, Y., Xu, X., Tan, Z., & Luo, M. (2025). A Review of 2D Lidar SLAM Research. *Remote Sensing*, *17*(7), 1214. https://doi.org/10.3390/rs17071214

53. ROBOTIS. (2026a). *TURTLEBOT3 Burger RPi4 4GB [INTL]* [Tienda internacional de ROBOTIS. Precio de catálogo de 681.30 USD, sin envío ni impuestos de destino]. Consultado el 26 de septiembre de 2026, desde https://en.robotis.com/shop_en/item.php?it_id=901-0118-201

54. ROBOTIS. (2026b). *TURTLEBOT3 Waffle Pi RPi4 4GB [INTL]* [Tienda internacional de ROBOTIS. Precio de catálogo de 1 681.40 USD, sin envío ni impuestos de destino]. Consultado el 26 de septiembre de 2026, desde https://en.robotis.com/shop_en/item.php?it_id=901-0119-202

55. Rohmer, E., Singh, S. P. N., & Freese, M. (2013). V-REP: A Versatile and Scalable Robot Simulation Framework [Actualmente distribuido como CoppeliaSim]. *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1321-1326. https://doi.org/10.1109/IROS.2013.6696520

56. ros-controls. (s.f.). *ros2_control Documentation – Foxy* [Documentación oficial de la versión Foxy; esta distribución se encuentra fuera de soporte]. Consultado el 12 de septiembre de 2026, desde https://control.ros.org/foxy/

57. Servicio de Impuestos Internos. (2024). *Dólar observado 2024* [Cotizaciones diarias y promedios mensuales del dólar observado, determinado según el Compendio de Normas de Cambios Internacionales del Banco Central de

Chile]. Consultado el 26 de septiembre de 2026, desde https://www.sii.cl/valores_y_fechas/dolar/dolar2024.htm

58. Shanghai Slamtec Co., Ltd. (2020). *RPLIDAR A1 Low Cost 360 Degree Laser Range Scanner: Introduction and Datasheet* [Hoja de datos rev. 3.0, documento LD108, modelo A1M8]. Consultado el 12 de septiembre de 2026, desde https://wiki.slamtec.com/download/attachments/83066883/LD108_SLAMTEC_rplidar_datasheet_A1M8_v3.0_en.pdf?api=v2&modificationDate=1677786044000&version=1

59. Shi, Y., & Eberhart, R. C. (1998). A Modified Particle Swarm Optimizer. *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*, 69-73. https://doi.org/10.1109/ICEC.1998.699146

60. Şucan, I. A., Moll, M., & Kavraki, L. E. (2012). The Open Motion Planning Library. *IEEE Robotics & Automation Magazine*, *19*(4), 72-82. https://doi.org/10.1109/MRA.2012.2205651

61. Sun, C., Sun, P., Zhou, J., & Mao, J. (2022). Travel Reduction Control of Distributed Drive Electric Agricultural Vehicles Based on Multi-Information Fusion. *Agriculture*, *12*(1), 70. https://doi.org/10.3390/agriculture12010070

62. Sundström, E., Hägglund, T., Bauer, M., Eker, J., & Soltesz, K. (2024). Reference Implementation of the PID Controller. *IFAC-PapersOnLine*, *58*(7), 370-375. https://doi.org/10.1016/j.ifacol.2024.08.090

63. The Orocos Project. (s.f.). *Orocos Project Documentation: Real-Time Toolkit* [Documentación oficial]. Consultado el 17 de julio de 2026, desde https://docs.orocos.org/

64. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.

65. Tourani, A., Bavle, H., Sanchez-Lopez, J. L., & Voos, H. (2022). Visual SLAM: What Are the Current Trends and What to Expect? *Sensors*, *22*(23), 9297. https://doi.org/10.3390/s22239297

66. Trejos, K., Rincón, L., Bolaños, M., Fallas, J., & Marín, L. (2022). 2D SLAM Algorithms Characterization, Calibration, and Comparison Considering Pose Error, Map Accuracy as Well as CPU and Memory Usage. *Sensors*, *22*(18), 6903. https://doi.org/10.3390/s22186903

67. Valderas Neculqueo, S. (2026, 29 de septiembre). *Navegación y exploración autónoma de interiores utilizando un robot móvil de tracción diferencial* [Video] [Publicado en YouTube]. Consultado el 29 de septiembre de 2026, desde https://www.youtube.com/watch?v=iTTC10eGDmc

68. Vincent, J. A., Nishimura, H., Itkina, M., Shah, P., Schwager, M., & Kollar, T. (2024). How Generalizable Is My Behavior Cloning Policy? A Statistical Approach to Trustworthy Performance Evaluation. *IEEE Robotics and Automation Letters*, *9*(10), 8619-8626. https://doi.org/10.1109/LRA.2024.3445635

69. Wittenmark, B., Åström, K. J., & Årzén, K.-E. (2002). *Computer Control: An Overview* [IFAC Professional Brief]. International Federation of Automatic Control. Consultado el 18 de octubre de 2026, desde https://www.ifac-control.org/publications/list-of-professional-briefs/pb_wittenmark_etal_final.pdf/view

70. Wu, M., Wang, H., Wang, W., Miao, S., & Hu, S. (2026). A ROS 2-Based Robotic Platform for Mobile Occupant Sensing and Edge Perception in Buildings. *Buildings*, *16*(17), 3551. https://doi.org/10.3390/buildings16173551

71. Yamauchi, B. (1997). A Frontier-Based Approach for Autonomous Exploration. *Proceedings of the 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 146-151. https://doi.org/10.1109/CIRA.1997.613851

72. YARP Developers. (s.f.). *YARP: Yet Another Robot Platform* [Documentación oficial]. Consultado el 17 de julio de 2026, desde https://www.yarp.it/latest/what_is_yarp.html

GLOSARIO

Arduino Arduino es una plataforma de *hardware* y *software* de código abierto para crear proyectos electrónicos interactivos, pensada para ser fácil de usar y muy extendida en robótica. xi, xvi, xvii, 15, 34, 39, 40, 43, 46, 61, 79, 101, 105, 106, 108, 113

IMU *Inertial Measurement Unit*. Unidad inercial que integra acelerómetros, giroscopios (y a veces magnetómetros) para estimar orientación y movimiento. 4, 84

LIDAR *Light Detection and Ranging*. Sensor que utiliza pulsos de luz láser para medir distancias y generar representaciones 2D/3D del entorno con alta precisión. 1, 3, 7, 8, 15, 16, 21, 30, 40, 79, 130, 138

PWM *Pulse Width Modulation*. Técnica de control que regula la potencia promedio entregada a una carga variando el ciclo útil de un pulso periódico. 23, 25, 34, 116, 118

ROS 2 *Robot Operating System 2* es un conjunto de bibliotecas y herramientas de código abierto para desarrollar *software* de robótica de forma modular y distribuida. xvi, xvii, 1, 4, 8–11, 13–16, 19, 20, 36, 38, 40, 43, 45, 54, 56, 61, 79, 83, 84, 105–107, 109

ANEXOS

Convención de rutas. Las rutas de código son relativas a `src/`; el inventario de archivos, directorios padre y funciones se presenta en el ANEXO 9.

ANEXO 1. Construcción, componentes y montaje

Este anexo reúne información complementaria sobre los componentes, el montaje físico y la reproducción del prototipo.

ANEXO 1.1. Costo de los materiales

La Tabla A1.1 resume los valores pagados por los materiales del proyecto, en pesos chilenos nominales, según los registros de compra entre abril de 2024 y julio de 2026. Los kits de tornillería, conectores y cables se contabilizan por su precio completo, aunque solo se usa una parte de ellos. El total del robot comprende los elementos montados en la plataforma. El grupo de operación y desarrollo reúne elementos que no van montados en el robot, como la red dedicada, el cargador, las baterías de repuesto y los accesorios usados para acceder a la Raspberry Pi durante el desarrollo. Ningún monto incluye la estación de trabajo. La comparación de estos montos con plataformas comerciales se presenta en el ANEXO 1.2.

Tabla A1.1: Costo de los materiales del prototipo.

Elemento	Cantidad	Costo (CLP)
Componentes principales del robot		
Raspberry Pi 4 Modelo B de 8 GB, carcasa de aluminio y microSD de 64 GB	1	91 759
RPLidar A1	1	83 324
Arduino Nano V3 y placa base con bornes de tornillo	1	8 000
Motorreductor JGA25-370 de 12 V con *encoder* y rueda	2	18 038
Driver L298N	1	3 000
Módulo reductor DC-DC de 25 W	1	18 858
Batería de 12 V y 3000 mAh con adaptador de montaje	1	25 658
Rueda loca	1	1 893
Subtotal componentes principales		**250 530**
Energía, montaje y cableado del robot		
Interruptor, portafusible, conectores XT60 y bloque de distribución	Varios	9 547
Filamento PLA+ de 1 kg	1	13 353
Tornillería, separadores y tuercas de inserción térmica	6 kits	127 234
Cables, conectores y terminales	Varios	43 979
Subtotal energía, montaje y cableado		**194 113**
Total del robot		**444 643**
Operación y desarrollo, fuera del robot		
Enrutador de viaje GL.iNet GL-MT300N-V2	1	22 140
Cables de red RJ45 Cat-6	3	8 376
Cargador de baterías de 12 V	1	16 169
Baterías de 12 V de repuesto	2	41 436
Transmisor y receptor HDMI inalámbrico, cables micro HDMI y *hub* USB	Varios	49 683
Subtotal operación y desarrollo		**137 804**
Total del proyecto		**582 447**

ANEXO 1.2. Comparación con plataformas comerciales

La Tabla A1.2 contrasta el costo del prototipo con el de dos plataformas comerciales comparables, orientadas a docencia e investigación con ROS. Los montos en pesos se convierten a dólares estadounidenses con un tipo de cambio referencial de 963 CLP/USD, que corresponde al promedio de los valores mensuales del dólar observado de octubre a diciembre de 2024 (Servicio de Impuestos Internos, 2024). Se elige ese período porque el 68 % del gasto del prototipo co-

rresponde a 2024. Los precios comerciales son precios de catálogo vigentes a la fecha de consulta y no incluyen envío, aranceles ni impuestos de internación, por lo que su costo real en Chile sería mayor.

Tabla A1.2: Comparación de costo con plataformas comerciales.

Plataforma	Costo (USD)	Costo (CLP)	Observación
Este prototipo, robot	462[a]	444 643	Materiales montados en el robot (Tabla A1.1).
Este prototipo, proyecto completo	605[a]	582 447	Agrega red dedicada, cargador, baterías de repuesto y accesorios de desarrollo.
TurtleBot 3 Burger RPi4 4 GB (ROBOTIS, 2026a)	681[b]	656 092	LIDAR 2D de 360°, Raspberry Pi 4, controlador con IMU, actuadores con controlador y *encoder* integrados, batería y cargador.
TurtleBot 3 Waffle Pi RPi4 4 GB (ROBOTIS, 2026b)	1 681[b]	1 619 188	Agrega cámara y una base de mayor tamaño y capacidad de carga.

[a] Convertido desde pesos chilenos al tipo de cambio referencial de 963 CLP/USD.
[b] Precio de catálogo de la tienda internacional de ROBOTIS, consultado en septiembre de 2026, sin envío ni impuestos de destino. Convertido a pesos al mismo tipo de cambio.

El costo del robot es un 32 % menor que el del TurtleBot 3 Burger, la plataforma comercial más cercana, y el del proyecto completo, un 11 % menor. Esa diferencia debe leerse junto con el equipamiento de cada plataforma. El Burger incluye una unidad de medición inercial en su controlador, que el prototipo no posee, y actuadores con controlador y *encoder* integrados, que el prototipo resuelve con componentes separados de menor costo. La comparación indica, por tanto, que el prototipo se ubica por debajo del costo de la plataforma comercial de entrada, y no que ofrezca las mismas prestaciones a menor precio.

ANEXO 1.3. Parámetros de impresión 3D

La Tabla A1.3 reúne los parámetros generales utilizados en la fabricación de las piezas por impresión 3D.

Tabla A1.3: Parámetros generales de impresión 3D.

Parámetro	Valor
Material	PLA+
Altura de capa	0.2 mm
Relleno	20–30 %
Paredes perimetrales	3
Temperatura de boquilla	205–215 °C
Temperatura de cama	55–60 °C
Velocidad de impresión	50 mm/s
Tolerancia de ensamblaje	0.2–0.4 mm

ANEXO 2. Gemelo digital y verificación estructural

La organización de los archivos del modelo se describe en la Sec. 4.2.1. Como trazabilidad adicional, el lanzamiento `articubot_one/launch/launch_sim.launch.py` fija `use_ros2_control=true` y `sim_mode=true`; por ello, instancia `gazebo_ros2_control/GazeboSystem`. El archivo alternativo `gazebo_control.xacro` no corresponde al *backend* seleccionado por ese lanzamiento. Los identificadores completos de *plugin* resumidos en la Tabla 4.2 son `diffdrive_arduino/DiffDriveArduino` en *hardware* real y `gazebo_ros2_control/GazeboSystem` en simulación.

El archivo `robot_core.xacro` define `base_footprint` como un enlace hijo fijo y coincidente con `base_link`, mediante una transformación de traslación y rotación nulas. El marco base declarado en la configuración depende del entorno: en simulación, `slam_toolbox` y AMCL emplean `base_footprint`; en *hardware* real, ambos emplean `base_link`. Los *costmaps* mantienen `base_link` en ambos entornos.

El sensor láser simulado se declara en `lidar_sim.xacro` mediante el complemento de sensor de rango de Gazebo Classic, con 360 muestras por barrido, apertura de ± 3.14 rad, tasa de actualización de 10 Hz y rango de 0.08 m a 12 m. La declaración no incorpora un término de ruido y su rango mínimo es más permisivo que el del sensor físico; ambas diferencias corresponden a la brecha simulación–realidad señalada en la Sec. 4.2.4.

En *hardware* real, el nodo de adquisición del ANEXO 3.4 publica 230 sectores por barrido y descarta las lecturas bajo 0.15 m, de modo que la resolución angular y el rango mínimo de ambos entornos tampoco coinciden.

ANEXO 2.1. Validación estructural del URDF

La auditoría del URDF verifica la jerarquía de eslabones y articulaciones, la ausencia de marcos sueltos y la coherencia de la cadena TF principal (Sec. 4.2.2). La geometría acotada del modelo, con las dimensiones de chasis, ruedas, rueda loca y sensor, se presenta en la Fig. A2.1. Los elementos cinemáticos verificados en esa auditoría se resumen en la Tabla A2.1.

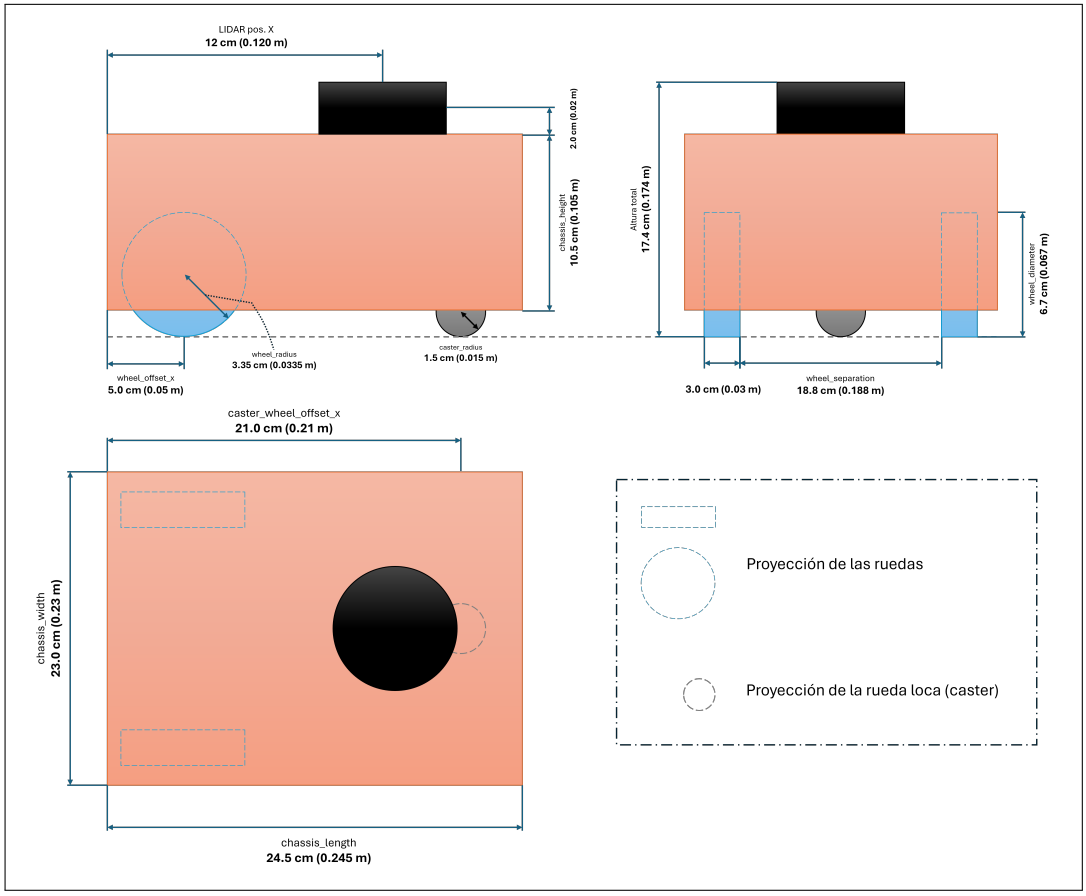


Figura A2.1: Dimensiones del modelo del robot (gemelo digital): vistas lateral, frontal y en planta.

Fuente: Elaboración propia.

Tabla A2.1: Resumen de elementos cinemáticos del modelo.

Elemento	Descripción
Links	`base_link`, `base_footprint`, `chassis`, ruedas y sensor.
Joints	Articulaciones fijas para chasis y sensor; articulaciones continuas para ruedas.
Marcos principales	`odom`, `base_link`, `base_footprint`, `laser_frame`.

La correspondencia entre componentes, marcos y tópicos del modelo digital es la siguiente: la base diferencial publica la transformación `odom`→`base_link` y la odometría en `/odom`; el sensor láser opera en `laser_frame` y publica en `/scan`; y la transformación estática `base_link`→`laser_frame` se publica en `/tf_static`.

ANEXO 2.2. Criterios de aceptación del gemelo digital

Tabla A2.2: Criterios mínimos de aceptación del gemelo digital.

Criterio	Verificación	Resultado esperado
Cadena TF completa	Marcos `odom`, `base_link` y `laser_frame`	Sin marcos sueltos
Publicación de sensores	Tópicos `/scan` y `/odom`	Frecuencias estables
Respuesta cinemática	Avance recto y giro en Gazebo Classic	Movimiento coherente con el modelo diferencial
Paridad geométrica	Radio y separación de ruedas	Igualdad con parámetros del prototipo

Los criterios anteriores se confirman durante la puesta en marcha del modelo digital y complementan la evidencia estructural de la Tabla A2.1.

ANEXO 3. *Hardware*, red e integración con ROS 2

Este anexo documenta información complementaria sobre el entorno de ejecución, la red de comunicación, los paquetes principales y la integración entre Raspberry Pi, Arduino y sensores.

ANEXO 3.1. Entorno de ejecución y *workspace*

La misma distribución de ROS 2 se ejecuta en la estación de trabajo y en la Raspberry Pi 4. La comunicación entre ambas se realiza mediante una red Wi-

Fi dedicada, con IP estáticas. La Tabla A3.1 detalla los paquetes principales del *workspace* y su rol.

Tabla A3.1: Paquetes principales del *workspace*.

Paquete	Rol en el sistema	Origen
`articubot_one`	Descripción URDF/`xacro`, configuración de control, SLAM, navegación y simulación.	Adaptado de (Newans, 2023)
`diffdrive_arduino`	Interfaz `ros2_control` entre ROS 2 y el Arduino.	Adaptado de (Newans, 2022a)
`ROSArduinoBridge`	*Firmware* de bajo nivel para *encoders*, motores y control de velocidad.	Adaptado de (Newans, 2022b), bifurcación de (Home Brew Robotics Club, s.f.)
`explore_lite`	Exploración autónoma basada en fronteras (proyecto `m-explore`).	Adaptado de (Alvarez, 2021), *port* a ROS 2 de (Hörner, 2017)
`pid_config`	*Scripts* de captura, identificación FOPDT y sintonización PSO.	Elaboración propia

Los paquetes `articubot_one`, `diffdrive_arduino`, `ROSArduinoBridge` y `explore_lite` provienen de bases de código abierto de terceros, cuyos repositorios de origen se indican en la última columna de la Tabla A3.1. Todos son modificados y configurados por el autor para esta plataforma. El uso de `ros2_control` sobre robots diferenciales de bajo costo cuenta con antecedentes documentados que orientan esta integración (Amadi et al., 2024). El aporte propio sobre estos paquetes se describe en la Justificación de la Introducción.

La Tabla A3.2 registra el equipamiento de la estación de trabajo y las versiones exactas del entorno con las que se obtienen los resultados del Capítulo 5. El equipamiento embarcado en el robot se describe en la Sec. 4.1.

Tabla A3.2: Equipamiento de la estación de trabajo y versiones del entorno de ejecución.

Elemento	Valor
Estación de trabajo	
Procesador	Intel Core i7-10610U, 4 núcleos, 8 hilos, 4.90 GHz
Memoria RAM	32 GB
Arquitectura	x86-64
Entorno de software	
Sistema operativo	Ubuntu 20.04.6 LTS
Núcleo Linux	5.15.0-139-generic
Distribución ROS 2	Foxy Fitzroy (`ros-foxy-ros-core` 0.9.2)
Middleware de comunicación	`rmw_cyclonedds_cpp`
Dominio ROS 2	`ROS_DOMAIN_ID = 0`
Simulador	Gazebo Classic 11.11.0
`gazebo_ros_pkgs`	3.5.3
`slam_toolbox`	2.4.1
`nav2_bringup`	0.4.7
`ros2_control`	0.11.0

ANEXO 3.2. Nodos del lanzamiento en *hardware* real

El archivo `launch_robot.launch.py` inicia los nodos resumidos en la Tabla A3.3. Los dos controladores no se lanzan como nodos independientes: se cargan mediante *spawners* sobre el `controller_manager`, con los nombres `diff_cont` y `joint_broad` declarados en `my_controllers.yaml`. La descripción del robot se publica desde el lanzamiento anidado `rsp.launch.py`, con `use_ros2_control=true` y `use_sim_time=false`.

Tabla A3.3: Nodos iniciados por el lanzamiento en *hardware* real.

Nodo o controlador	Paquete o tipo	Rol en el sistema
`robot_state_publisher`	`robot_state_publisher`	Publica las transformaciones del robot a partir de la descripción URDF/`xacro`.
`controller_manager`	`controller_manager` (ejecutable `ros2_control_node`)	Gestiona la interfaz de *hardware* y el ciclo de los controladores.
`diff_cont`	`diff_drive_controller/DiffDriveController`	Convierte comandos de velocidad del robot en referencias de rueda y publica la odometría.
`joint_broad`	`joint_state_broadcaster/JointStateBroadcaster`	Publica el estado de las articulaciones.

ANEXO 3.3. Asignación de pines y dispositivos

La Tabla A3.4 resume la asignación de pines entre el Arduino, el *driver* L298N y los *encoders*.

Tabla A3.4: Asignación resumida de pines entre Arduino, L298N y *encoders*.

Función	Pin Arduino	Destino
PWM avance izquierdo	D6	IN3 L298N
PWM retroceso izquierdo	D10	IN4 L298N
PWM avance derecho	D9	IN1 L298N
PWM retroceso derecho	D5	IN2 L298N
Habilitación izquierda	D13	ENB L298N
Habilitación derecha	D12	ENA L298N
Encoder izquierdo A/B	D3/D2	*Encoder* izquierdo
Encoder derecho A/B	A4/A5	*Encoder* derecho

Para evitar cambios de nombres en los puertos serie, se utilizan reglas `udev`. El Arduino se expone como `/dev/motor_controller`; el RPLidar puede exponerse como `/dev/rplidar` o `/dev/rlidar_gpio`, según el modo de conexión.

La teleoperación utiliza los nodos `joy_node` y `teleop_twist_joy`. Sus comandos se integran mediante `twist_mux`; la salida se remapea al tópico `/diff_cont/cmd_vel_unstamped`, entrada del controlador diferencial. El botón de habilitación actúa como *dead-man switch*.

ANEXO 3.4. Nodo de adquisición del RPLidar A1

En *hardware* real, el sensor no se lee con el controlador del fabricante. El lanzamiento `rplidar.launch.py` inicia el nodo `rplidar_serial_publisher.py`, desarrollado para este trabajo dentro del paquete `articubot_one`, que se comunica con el RPLidar A1 a través de `/dev/rlidar_gpio` a 115200 baudios. Al arrancar, el nodo detiene y reinicia el sensor, consulta su información y su estado de salud, y no inicia la publicación si no logra leer ese estado o si el sensor informa un error. Luego activa el modo de barrido estándar. El nodo ofrece además los servicios `~/stop_scan` y `~/start_scan` para detener y reanudar el barrido.

Cada 50 ms el nodo lee el búfer serie, descarta los paquetes que no cumplen los bits de verificación del protocolo y agrupa las mediciones por vuelta. Una vuelta se cierra cuando el ángulo retrocede más de 187.5° respecto de la medición anterior, y se publica solo si reúne al menos 180 mediciones. En caso contrario no se publica, parte de las mediciones siguientes se descarta y el resto se agrega a la misma vuelta.

Cada vuelta publicada se reparte en 230 sectores angulares de $2\pi/230 \approx 1.57°$. En cada sector se conserva la menor distancia medida. Los sectores sin medición o con lecturas fuera del intervalo de 0.15 m a 12 m quedan con valor infinito. El ángulo se invierte antes de asignar el sector, para que el barrido quede en la convención de ángulos positivos en sentido antihorario de ROS 2. El mensaje se publica en `/scan`, en el marco `laser_frame` y con el perfil de calidad de servicio para datos de sensores. El lanzamiento entrega también un parámetro de frecuencia de barrido que el nodo no declara, por lo que no tiene efecto.

La marca de tiempo de cada barrido corresponde al instante de publicación, posterior al cierre de la vuelta, y no al de su primera medición. El período del barrido se estima con el intervalo entre publicaciones consecutivas. Por ello, los nodos que consumen el tópico asocian cada barrido a una pose posterior a la real, desfase cuyo efecto crece con la velocidad del robot, en especial durante los giros. Su efecto sobre los mapas no se cuantifica.

ANEXO 3.5. Temporalidad y ciclo de la interfaz `DiffDriveArduino`

El *plugin* exporta, para cada rueda, una interfaz de comando de velocidad y dos interfaces de estado, de posición y de velocidad. En la fase de lectura, el conteo acumulado recibido por el enlace serial se convierte en posición angular mediante el factor $2\pi/N_e$ de la Ec. (3.9) y en velocidad angular por diferencias finitas, según la Ec. (3.10), empleando como denominador el intervalo Δt_k definido a continuación.

En la implementación, `time_` se inicializa al comienzo de `configure()`, antes de leer los parámetros y antes de que `arduino_.setup()` abra el puerto, espere

3000 ms y vacíe el búfer de entrada. Por eso el denominador Δt_k que divide las diferencias finitas no es un valor fijo: en la primera lectura (k = 0) es el tiempo transcurrido entre esa inicialización de `time_` y el comienzo de la primera llamada a `read()`, e incluye la espera serial de 3000 ms; desde la segunda lectura ($k \geq 1$) es el intervalo real medido entre dos llamadas consecutivas a `read()`. En consecuencia, la primera muestra de velocidad arrastra el tiempo de configuración, y la velocidad de estado entregada a `ros2_control` no utiliza un valor fijo 1/30 s.

La igualdad numérica de las frecuencias configuradas no implica sincronización temporal. El `controller_manager` y el parámetro `loop_rate` se configuran nominalmente en 30 Hz, mientras que el *firmware* programa un incremento entero de 33 ms sobre `nextPID` (`PID_INTERVAL`). Este intervalo corresponde a la planificación temporal interna del Arduino y no está sincronizado en reloj ni fase con el ciclo de `ros2_control`. En consecuencia, se distingue Δt_k para el intervalo entre lecturas de velocidad, f_{loop} para la conversión de consignas y $T_{PID,sched}$ para el intervalo programado del PID en el *firmware*.

En la fase de escritura de la interfaz, la velocidad angular comandada de la rueda, ω_j^{cmd} [rad/s], se convierte en una referencia entera de *ticks/frame* antes de enviar el comando serial. La conversión multiplica esa velocidad angular por un factor que combina la resolución efectiva del *encoder* N_e, la frecuencia nominal del lazo de control f_{loop} y el número de radianes de una vuelta de rueda; de forma equivalente, puede partirse de la consigna lineal de rueda, v_j^{cmd} [m/s], incorporando además el radio de rueda r. El resultado se trunca hacia cero al entero que admite el comando serial.

ANEXO 3.6. Identificadores de código de los parámetros del cuerpo

La Tabla A3.5 asocia los parámetros presentados en el Capítulo 4 con el identificador que reciben en el código y con el archivo que los define; la localización completa de cada archivo se indica en el ANEXO 9. El identificador de la resolución efectiva N_e se documenta en el ANEXO 4.1.

Tabla A3.5: Correspondencia entre parámetros del cuerpo e identificadores del código.

Parámetro	Identificador	Archivo
f_{ctrl} = 30 Hz	`controller_manager.update_rate`	`my_controllers.yaml`
f_{loop} = 30 Hz	`loop_rate`	`ros2_control.xacro`
$f_{PID,nom}$ = 30 Hz	`PID_RATE`	`ROSArduinoBridge.ino`
$T_{PID,sched}$ = 33 ms	`PID_INTERVAL`, incremento aplicado a `nextPID`	`ROSArduinoBridge.ino`
r = 0.0335 m	`wheel_radius`; en el *firmware*, `WHEEL_DIAMETER`= 0.067 m	`my_controllers.yaml`, `ROSArduinoBridge.ino`
b = 0.188 m	`wheel_separation`	`my_controllers.yaml`
Límites lineales $\pm$0.13 m/s	`linear.x.min_velocity`, `linear.x.max_velocity`	`my_controllers.yaml`
Límites angulares $\pm$0.35 rad/s	`angular.z.min_velocity`, `angular.z.max_velocity`	`my_controllers.yaml`
Publicación de `odom`→`base_link`	`enable_odom_tf`	`my_controllers.yaml`
Autodetención de seguridad	`AUTO_STOP_INTERVAL`= 2000 ms	`ROSArduinoBridge.ino`

ANEXO 4. *Firmware*, *encoders* y sintonización del controlador

Este anexo complementa el capítulo de *firmware* con datos experimentales y parámetros usados durante la sintonización del controlador de velocidad.

El *firmware* se distribuye en módulos independientes: `ROSArduinoBridge.ino` contiene el ciclo principal y el analizador serial; `commands.h`, los identificadores de comandos; `encoder_driver.h` y `encoder_driver.ino`, la lectura de *encoders*; `motor_driver.h` y `motor_driver.ino`, el accionamiento del L298N; y `diff_controller.h`, el controlador discreto y sus ganancias. Su localización se indica en el ANEXO 9.

ANEXO 4.1. *Ticks* por vuelta y ganancias finales

Los contadores `left_enc_pos` y `right_enc_pos` son variables `volatile long` actualizadas por interrupciones de cambio de pin. `readEncoder()` devuelve cada contador sin deshabilitar interrupciones ni utilizar una sección atómica. En el Arduino Nano AVR, la lectura de 32 bits no está garantizada como una operación atómica; por tanto, el conteo devuelto puede, en principio, ser modificado durante

su copia. Esta condición se conserva porque corresponde al *firmware* ejecutado.

Para caracterizar la resolución efectiva de los *encoders*, se hace girar cada rueda una vuelta completa y se repite la medición diez veces, cuyos resultados se presentan en la Tabla A4.1. Durante los ensayos, el *firmware* y la interfaz de *hardware* utilizan el valor unificado N_e = 1980 *ticks*/vuelta (Tabla 4.1); las mediciones experimentales permiten cuantificar la discrepancia de dicho valor respecto de los promedios obtenidos para cada rueda.

Tabla A4.1: Muestras del número de *ticks* por revolución.

N	Rueda derecha	Rueda izquierda
1	1966	1978
2	1960	1948
3	2002	1966
4	1980	1970
5	1972	1972
6	1964	1978
7	1980	1957
8	1980	1975
9	1980	1962
10	1978	1979

Los promedios de las diez repeticiones son $\bar{N}_R$ = 1976 y $\bar{N}_L$ = 1969 *ticks*/vuelta (ruedas derecha e izquierda), que difieren en menos del 0.4 %. El valor unificado N_e = 1980 *ticks*/vuelta introduce un error sistemático de aproximadamente 0.20 % respecto de la media derecha y 0.56 % respecto de la izquierda.

A partir del factor $2\pi/N_e$ de la Ec. (3.9), la constante `rads_per_count` vale, para N_e = 1980, $2\pi/1980 \approx 0.003173$ rad/*tick*. En el código, N_e corresponde a `ENC_TICKS_PER_REV` en el *firmware* y a `enc_counts_per_rev` en el *plugin* `DiffDriveArduino`.

ANEXO 4.2. Comparación ilustrativa entre PSO y sintonización Lambda

El *script* `SintonizacionLambda.py` vuelve a identificar un modelo FOPDT mediante un PSO sin semilla fija, por lo que sus parámetros pueden diferir de los generados por `SintonizacionAutomatica.py`, y simula las ganancias finales del *firmware* reutilizando la misma rutina de planta, referencia interpolada y cuantización descrita en el ANEXO 4.7, con banda muerta 2 para ambas ruedas. Además, el *script* recorta explícitamente `ITerm` a $\pm$12750, valor que resulta de multiplicar el PWM máximo (255) por el divisor de normalización de la salida (50); este límite

no existe en `diff_controller.h`; por tanto, no replica exactamente los tipos, las bandas muertas asimétricas ni todas las condiciones del *firmware*.

La referencia analítica Lambda se calcula a partir de esa nueva identificación. Como retardo efectivo, $L_{ef,j}$, se toma el mayor entre el retardo puro identificado para la rueda, $L_{d,j}$, y dos períodos de control (2/30 s); y como constante de diseño, λ_j, se fija el triple de la constante de tiempo identificada τ_j. A partir de ahí el *script* obtiene una forma serie, la convierte a la forma paralela estándar y la lleva a la escala de *firmware* mediante un factor de escalamiento f_λ; el detalle de estas conversiones se encuentra en `arduino_firmware/pid_config/SintonizacionLambda.py`.

Esa conversión asigna cada ganancia de la forma paralela a la ganancia del *firmware* con el mismo nombre y escala K_d sin dividir por el período de control, sin considerar que en la ley ejecutada esas ganancias cumplen otra función (Ec. (4.4)). Por ello, la curva Lambda no representa un controlador Lambda equivalente aplicado sobre la ley ejecutada.

La comparación es únicamente ilustrativa: además de las aproximaciones señaladas, el simulador aplica banda muerta cero al controlador Lambda. Por ello, las curvas y métricas corresponden al *script* comparativo ejecutado y no a una validación de controladores equivalentes en *hardware*.

ANEXO 4.3. Protocolo de comunicación serial

La comunicación entre la Raspberry Pi y el Arduino utiliza comandos textuales terminados mediante un carácter de retorno de carro (`\r`). La Tabla A4.2 resume los comandos implementados en el *firmware* y utilizados para el control, la configuración y el diagnóstico del robot.

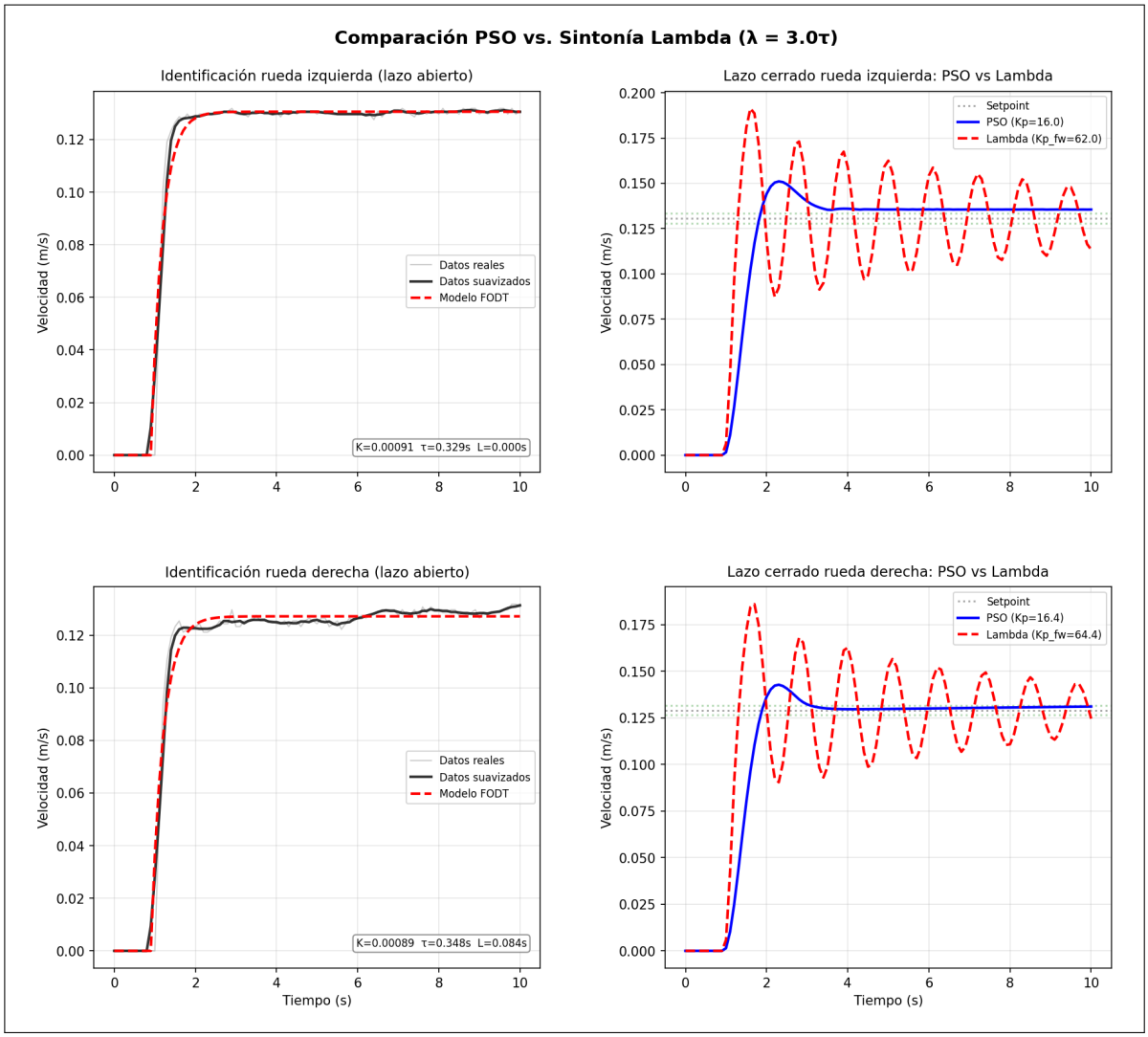


Figura A4.1: Identificación de los modelos FOPDT de ambas ruedas en lazo abierto y comparación simulada de las respuestas en lazo cerrado obtenidas con las ganancias finales asociadas al procedimiento PSO y con la sintonización Lambda.

Fuente: Elaboración propia.

Tabla A4.2: Comandos del protocolo serial implementado en el *firmware*.

Comando	Formato	Función	Respuesta
`b`	`b`	Consulta la velocidad de comunicación serial configurada en el *firmware*.	Tasa de baudios
`e`	`e`	Lee los conteos acumulados de los *encoders* izquierdo y derecho.	`L R`
`r`	`r`	Reinicia los conteos de ambos *encoders* y ejecuta `resetPID()`, que vuelve a cargar las ganancias predeterminadas y reinicia los estados internos.	`OK`
`o`	`o PWM_L PWM_R`	Ejecuta primero `resetPID()` y luego aplica directamente valores PWM a los motores izquierdo y derecho, sin control en lazo cerrado. Los valores se limitan al intervalo [–255, 255].	`OK`
`m`	`m M_L M_R`	Establece las referencias del controlador de velocidad en *ticks/frame* para las ruedas izquierda y derecha. Si ambas son cero, detiene los motores y ejecuta `resetPID()`.	`OK`
`k`	`k V_L V_R`	Establece las referencias de velocidad lineal de rueda en m/s. El *firmware* las convierte internamente a *ticks/frame*; si ambas son exactamente cero, detiene los motores y ejecuta `resetPID()`.	`OK`
`j`	`j`	Consulta `PrevInput` de cada controlador, es decir, la última medición entera en *ticks/frame* actualizada por el ciclo de control, y la convierte a m/s. El comando no fuerza una nueva actualización del controlador.	`V_L V_R`
`u`	`u Kp:Kd:Ki:Ko`	Carga un conjunto común de ganancias para ambas ruedas.	`OK`
`z`	`z` $\mathbf{K}_L : \mathbf{K}_R$	Carga ganancias independientes para las ruedas izquierda y derecha.	`OK`
`n`	`n`	Consulta las ganancias almacenadas. La respuesta conserva el orden izquierda K_p, K_d, K_i, K_o y luego derecha K_p, K_d, K_i, K_o.	Ocho valores

Nota: En el comando `z`, el formato serial completo es:

```
z KpL:KdL:KiL:KoL:KpR:KdR:KiR:KoR
```

donde L y R representan las ruedas izquierda y derecha, respectivamente; V_L y V_R son sus velocidades lineales [m/s], mientras que M_L y M_R son referencias expresadas en *ticks/frame*. Los comandos de movimiento actualizan el temporizador de seguridad del *firmware*; si no se recibe una nueva orden durante el intervalo de autodetención, definido en el *firmware* como `AUTO_STOP_INTERVAL`= 2000 ms, los motores se detienen automáticamente y la bandera interna `moving` vuelve a cero. Las ganancias cargadas mediante `u` o `z` modifican los valores almacenados sin reiniciar los estados internos; su vigencia se describe en la Sec. 4.4.3.

ANEXO 4.4. Ecuaciones y estados internos del PI incremental embebido

La implementación sigue la forma incremental de las Ecs. (4.1) a (4.3): en cada ciclo calcula un incremento, con un término calculado sobre la diferencia de la medición, y lo suma a la salida anterior. Además, usa variables enteras, truncamientos, banda muerta asimétrica, saturación y actualización condicional del integrador. Su idealización lineal, sin el estado adicional asociado a K_i ni las no linealidades anteriores, equivale al PI incremental de la Sec. 4.4.3, con las ganancias equivalentes de la Tabla 4.4. En el Arduino Nano basado en AVR, `int` es de 16 bits, `long` de 32 bits y `float`/`double` se representa con 32 bits.

En cada actualización se leen primero ambos *encoders*. El incremento de conteo de cada rueda, $c_j[k] – c_j[k – 1]$, se almacena en una variable de tipo `int` (16 bits), que conserva solo el valor representable por ese tipo. La referencia `TargetTicksPerFrame` es de tipo `double`, mientras que el error de control se almacena en un `long` con truncamiento hacia cero y, sobre él, se aplica una banda muerta asimétrica ($e_{db,R}$ = 1, $e_{db,L}$ = 2 *ticks/frame*). El incremento de salida se calcula en punto flotante a partir de los términos asociados a K_p, a K_d y al estado integral (dividido por K_o). Ese cociente se trunca hacia cero al asignarlo a un `long`, antes de sumarlo a la salida anterior; así se forma $u_j^{raw}[k]$, que se satura al rango $[–255, 255]$ para obtener la orden PWM aplicada $u_j[k]$.

El término integral se almacena en un `int`: la operación compuesta `ITerm += Ki*Perror` suma $K_{i,j}\, e_j^{db}[k]$ al valor anterior y vuelve a truncar el resultado hacia cero al tipo entero. El integrador solo se actualiza cuando la salida no está saturada ($–255 < u_j^{raw}[k] < 255$). En caso contrario conserva su valor, lo que implementa un mecanismo simple de *anti-windup* por integración condicional (Åström & Hägglund, 2006/2009, Sec. 3.5). Por el truncamiento entero, además, puede permanecer sin cambios cuando ese incremento no alcanza una unidad entera. El término I_j^{int} pertenece a la escala interna del numerador del controlador y no debe interpretarse directamente como una salida PWM.

Las ganancias definitivas se presentan en la Tabla 4.4. La secuencia completa de actualización se resume en el Alg. 4.2.

ANEXO 4.5. Origen de la ley de control y cambios respecto del *firmware* original

El *firmware* se adapta a partir de la bifurcación de `ros_arduino_bridge` mantenida por Josh Newans (Newans, 2022b), derivada a su vez del proyecto del Home Brew Robotics Club (Home Brew Robotics Club, s.f.). El encabezado de `diff_controller.h` en ambos repositorios indica que el controlador proviene en su mayor parte del código ArbotiX de Mike Ferguson. La función `doPID()` de ambos repositorios contiene la misma ley acumulativa de la Sec. 4.4.3. Por tanto, la estructura de la ley y el nombre de sus ganancias no se introducen en este trabajo, sino que se heredan.

Los comentarios del código original explican esa estructura. El término asociado a K_d se calcula sobre la medición para evitar saltos de la derivada ante cambios de la referencia, y el estado integral se guarda ya multiplicado por K_i para admitir cambios de ganancia en ejecución. Ambas mejoras corresponden a la forma posicional del PID, en la que la salida se calcula completa en cada ciclo (Åström & Hägglund, 2006/2009, Sec. 13.4). Al sumarse cada incremento a la salida anterior, cada término sube un orden de integración, tal como muestra la Ec. (4.4).

Respecto del original se conservan la estructura de la ley, la frecuencia de control de 30 Hz, la saturación a ± 255, la integración condicional, el reinicio de estados al detenerse, el intervalo de autodetención de 2000 ms y la rutina de lectura de *encoders*, incluida su lectura no atómica (ANEXO 4.1). Los cambios introducidos son cuatro. Primero, las ganancias dejan de ser constantes enteras compartidas por ambas ruedas, con $K_p = 20$, $K_d = 12$, $K_i = 0$ y $K_o = 50$ en el original, y pasan a ser valores de punto flotante propios de cada rueda, que `resetPID()` vuelve a cargar con los valores de la Tabla 4.4. Por ello, las ganancias cargadas por el enlace serial solo rigen hasta la siguiente detención o reinicio. Segundo, se agrega a `doPID()` una banda muerta sobre el error, de 1 *tick/frame* en la rueda derecha y 2 en la izquierda. Tercero, se definen el diámetro de rueda y la resolución de *encoders* del prototipo y se agregan comandos para fijar y leer velocidades en m/s, leer las ganancias y cargar ganancias distintas por rueda (Tabla A4.2), retirando los de servomotores y de verificación de conexión. Cuarto, en el accionamiento del L298N se intercambian los pines de avance y retroceso del motor izquierdo y se declaran los pines como salidas al iniciar.

El paso a ganancias de punto flotante tiene una consecuencia que el original no presenta. Con K_i entero, el producto $K_i e$ es siempre entero y el estado integral lo acumula sin pérdida. En este trabajo el estado integral se mantiene como entero de 16 bits, por lo que los valores fraccionarios $K_{i,L} = 0.075$ y $K_{i,R} = 0.001$ se truncan y su efecto queda limitado en la rueda izquierda y anulado en la derecha, como se analiza en la Sec. 5.2.

ANEXO 4.6. Procesamiento FOPDT usado por los *scripts*

Justificación del modelo. La estructura FOPDT es una de las reducciones de modelo más habituales antes de sintonizar un controlador, porque describe con pocos parámetros la respuesta al escalón (Maxim & De Keyser, 2023). Con $L_{d,j} = 0$ contiene el modelo de un motor DC en que se desprecia la inductancia de armadura (Iqbal, 2023), por lo que adoptarla no impone un retardo, ya que su valor lo determina la identificación. El retardo permite representar, además, los retardos que agrega la implementación digital, como el del muestreo de la medición (Åström & Hägglund, 2006/2009, Sec. 13.3). Como la respuesta se registra con el período de captura T_{cap}, el retardo se estima como un número entero de períodos (Maxim & De Keyser, 2023, Sec. 2.4). Esta resolución del procedimiento de ajuste debe distinguirse de una cota física del retardo del robot.

Procesamiento en los *scripts*. Para simular el modelo con datos muestreados cada T_{cap} segundos (el período de captura), el retardo puro se cuantiza a un número entero de muestras, $n_{d,j}$, dividiendo el retardo identificado $L_{d,j}$ por el período de captura y truncando el resultado. La entrada retardada, $u_{j,d}[k]$, se obtiene tomando la muestra de PWM correspondiente a $n_{d,j}$ pasos antes del instante actual, o la primera muestra disponible cuando ese índice resultaría negativo. El sistema de primer orden se integra por el método de Euler explícito: en cada paso, la velocidad estimada por el modelo $\widehat{v}_j[k]$, parte de su valor en el paso anterior y le suma una fracción de la diferencia entre la respuesta estática y esa misma estimación previa; la fracción es la razón entre el período de captura y la constante de tiempo τ_j, y la respuesta estática es la ganancia estática $K_{m,j}$ multiplicada por la entrada retardada. La estimación arranca en cero. Aquí $u_j[k]$ es la entrada PWM aplicada y $\widehat{v}_j[k]$ la velocidad lineal que el modelo estima para la rueda j (m/s). Esta formulación corresponde al procedimiento de identificación; las diferencias de la planta usada durante la sintonización se describen más adelante.

La velocidad medida $v_j^{med}[k]$ se procesa fuera de línea con `scipy.ndimage.uniform_filter1d` (`size=5`, modo `reflect`): un filtro uniforme centrado de cinco muestras; no causal ya que usa muestras anteriores y posteriores, del que se obtiene la velocidad filtrada $v_j^{filt}[k]$.

La función de costo de identificación ejecutada por el *script* es el error cuadrático medio entre la velocidad filtrada $v_j^{filt}[k]$ y la estimada por el modelo $\widehat{v}_j[k]$, promediado sobre las N muestras. Si $\tau_j < 0.01$ s o $\widehat{v}_j$ contiene valores no finitos, el costo se fija en 10^6.

Retardos identificados. Con esta cuantización, cualquier retardo que cumple $0 \leq L_{d,j} < T_{cap}$ produce $n_{d,j} = 0$ y el mismo costo de identificación, por lo que el optimizador puede detenerse en cualquier valor de ese intervalo. Eso explica que,

para la rueda derecha, el *script* de sintonización entregue 0.0692 s y el de comparación Lambda 0.0845 s, con los mismos $K_{m,R}$ y τ_R. Los retardos identificados, $L_{d,L}$ = 0.0001 s y $L_{d,R}$ = 0.0692 s (Tabla A4.4), son salidas numéricas indistinguibles dentro de ese intervalo; no establecen que el retardo físico esté acotado por T_{cap}. Con estos datos y esta discretización, el modelo FOPDT y uno de primer orden sin retardo producen el mismo ajuste, y la planta usada en la sintonización tampoco aplica el retardo (ANEXO 4.7). La estructura FOPDT se conserva porque contiene al modelo de primer orden y permite estudiar los escenarios de retardo de 0 y 0.1 s (ANEXO 4.12). La estimación precisa de retardos menores que 0.1 s requiere un muestreo más rápido y queda como trabajo futuro.

ANEXO 4.7. Planta utilizada durante la sintonización del controlador

Con los parámetros identificados, el *script* simula una planta discreta FOPDT sin aplicar el retardo $L_{d,j}$ (aunque calcula una variable local `delay_steps`). La consigna de cada rueda no se fija en 0.13 m/s: se obtiene del promedio de la respuesta experimental en lazo abierto para $t \geq 7$ s, tabulada cada 0.1 s e interpolada linealmente (`np.interp`) al período de simulación T_{sim}. La simulación cuantiza a *ticks/frame* solo la medición que ingresa al controlador (redondeo al par más cercano), mientras que sus estados internos permanecen en punto flotante. Durante la optimización se usa una banda muerta común de 2 *ticks/frame* para ambas ruedas, a diferencia del *firmware* final (1 *tick/frame* en la derecha y 2 en la izquierda); por ello la simulación conserva la estructura acumulativa del controlador pero no replica exactamente sus tipos de datos ni sus condiciones por rueda.

ANEXO 4.8. Particularidades de la implementación PSO

Después de sumar la velocidad a la posición, el núcleo aplica únicamente una cota inferior de 10^{-4} a cada componente del estado almacenado $\xi_{i,d}$; no aplica ninguna cota superior. Para evaluar cada partícula, sus componentes se recortan a $[10^{-4}, 1]$ y se escalan a los parámetros físicos θ_i^{eval} según la etapa: (K_m, τ, L_d) en la identificación FOPDT y (K_p, K_d, K_i) en la sintonización PID. El recorte superior a 1 existe solo dentro de la función de costo PID.

La función de costo PID, asignada al candidato evaluado para la rueda j, toma como objetivo el máximo de la señal de referencia v_j^{ref} y combina dos términos: el error cuadrático medio respecto de esa referencia, ponderado por w_{mse}, más una penalización por sobreimpulso; el cuadrado de cuánto excede la respuesta a la referencia, cuando la excede, ponderada por w_{OS}. Si la simulación genera una excepción o la respuesta contiene valores no finitos, se asigna en su

lugar una penalización fija de 10^6. Ambos pesos no se calculan: son constantes de diseño definidas localmente en la función `costFunction_PID()` de `arduino_firmware/pid_config/SintonizacionAutomatica.py`, con w_{mse} = 1.0 actuando como escala identidad del MSE y w_{OS} = 3.0 triplicando el peso relativo de cualquier excursión sobre la referencia, para orientar la búsqueda del enjambre hacia respuestas de bajo sobreimpulso. La función no penaliza el tiempo de subida ni el de establecimiento. Los indicadores de la Tabla 5.3 corresponden a la captura física descrita en la Sec. 5.2; no se confunden con las predicciones de la simulación usada por el optimizador.

La actualización de los mejores valores sigue la clase `pso` ejecutada. Las particularidades de la posición global y la salida final escalada se desarrollan a continuación.

Con los coeficientes de la Tabla A4.3, el factor de constricción está fijado en el *script* como la constante 0.7298, que es el valor que entrega la expresión de Clerc y Kennedy (Clerc & Kennedy, 2002, Ec. 5.3) cuando su parámetro de ajuste vale 1 y la suma de los coeficientes cognitivo y social es $\psi = c_1 + c_2 = 4.1$. El error que emplea el optimizador se define sobre la velocidad: es la diferencia entre la velocidad de referencia v_j^{ref} y la velocidad estimada por el modelo $\widehat{v}_j[k]$ para la rueda j, denotada $\varepsilon_j[k]$, y se resume por su norma-2 y su error cuadrático medio; no corresponde al error entero e_j^{long} que consume el *firmware*.

La mejor posición personal P_i se actualiza cuando el costo actual es menor que su costo histórico. El almacenamiento global presenta una particularidad: el costo global J^* se toma como el mínimo de los costos históricos J_i^{best}, pero la posición global G se copia desde la posición actual de la partícula de menor costo histórico (no desde su posición personal P_{i^*}). Esta diferencia se mantiene porque representa el código ejecutado. Su efecto sobre el resultado es nulo, porque el mejor global solo cambia cuando una partícula acaba de mejorar su costo, y en ese instante su posición actual y su mejor posición personal coinciden.

Al finalizar, los parámetros informados se obtienen escalando la posición global final G^{final} componente a componente, sin aplicar un nuevo recorte superior. En la etapa de identificación FOPDT, las tres componentes se multiplican por 0.01, 2.0 y 0.5 (para K_m, τ y L_d, respectivamente); en la etapa de sintonización PID, se multiplican por 40, 25 y 0.05 (para K_p, K_d y K_i). Como en este paso no hay recorte superior, en la etapa PID la salida puede superar los valores de escala 40, 25 y 0.05 aunque el costo se haya evaluado con una copia recortada de la posición; por tanto, la salida escalada no necesariamente coincide con el candidato que produce el costo almacenado. En la ejecución de la Tabla A4.4, la componente de G asociada a K_d termina en 1.39 y 1.47, de modo que el costo evalúa K_d = 25 en ambas ruedas y el *script* informa 34.69 y 36.70.

ANEXO 4.9. Protocolos de captura, identificación y sintonización

Tabla A4.3: Configuración de la sintonización por PSO (qué se optimiza y con qué hiperparámetros).

Parámetro	**Etapa 1: Identificación FOPDT**	**Etapa 2: Sintonización PID**
Variables optimizadas	(K_m, τ, L_d)	(K_p, K_d, K_i)
Dimensión del espacio	3	3
Función de costo	MSE de identificación (ANEXO 4.6)	MSE + penalización por sobreimpulso (ANEXO 4.8)
Vector de escala s_{esc}	(0.01, 2.0, 0.5)	(40, 25, 0.05)
Cota superior del estado ξ	No aplicada	No aplicada
Recorte superior durante la evaluación	No aplicado	Solo dentro de la función de costo PID (copia evaluada recortada a $[10^{-4}, 1]$)
Hiperparámetros comunes del enjambre		
Número de partículas del enjambre N_p	50	
Generaciones $g_{máx}$	150	
Coeficientes $c_1 = c_2$	2.05	
Factor de constricción ζ	0.7298	
Cota inferior $\xi_{mín}$	10^{-4}	
Tratamiento del costo	MSE con penalización 10^6 si $\tau < 0.01$ s o la respuesta contiene valores no finitos	MSE y sobreimpulso con $(w_{mse}, w_{OS}) = (1.0, 3.0)$; penalización 10^6 si la simulación genera una excepción o la respuesta contiene valores no finitos
Entrada o referencia	Escalón PWM = 143	Referencia obtenida del promedio de la respuesta en lazo abierto ($t \geq 7$ s), tabulada a 0.1 s e interpolada linealmente a T_{sim}

Combinando las Ecs. (3.9) y (3.10) con el período nominal de captura T_{cap}, el *script* calcula directamente la velocidad lineal medida de cada rueda, $v_j^{med}[k]$ [m/s]. Para ello toma el incremento de conteo del *encoder* en el intervalo —la

diferencia entre el conteo actual y el anterior, $\Delta c_j[k]$—, lo divide por la resolución efectiva del *encoder* N_e y lo multiplica por el cociente entre el perímetro de la rueda y el período de captura. En el *script* de captura `arduino_firmware/pid_config/ControlLazoAbierto.py`, T_{cap} = 0.1 s es un valor nominal fijo: la velocidad se calcula dividiendo por ese período y no por el tiempo real transcurrido entre lecturas. El tiempo medido mediante `time.time()` solo se utiliza para dormir el remanente del ciclo; si la comunicación serial consume más de 0.1 s, el *script* no corrige la escala temporal de la muestra.

ANEXO 4.10. Resultados de la ejecución PSO empleada

Tabla A4.4: Salida registrada de una ejecución de `SintonizacionAutomatica.py`: modelo FOPDT identificado y ganancias sugeridas por PSO.

Resultado	Rueda izq.	Rueda der.
Modelo FOPDT identificado (Etapa 1)		
K_m [(m/s)/PWM]	0.000913	0.000890
τ [s]	0.3291	0.3485
L_d [s][a]	0.0001	0.0692
Retardo entero aplicado n_d [muestras]	0	0
Ganancias sugeridas por PSO (Etapa 2)		
K_p	12.48	10.10
K_d informado y cargado[b]	34.69	36.70
K_d evaluado por el costo[b]	25.00	25.00
K_i	0.00001	0.00001

[a] Valor que entrega el optimizador. Como el retardo se cuantiza en muestras de T_{cap} = 0.1 s, cualquier L_d menor que ese período produce n_d = 0 y el mismo costo, por lo que el valor indica que el retardo es menor que T_{cap}, sin precisar su magnitud.

[b] La salida del *script* escala la posición global sin recortarla, mientras que el costo evalúa una copia recortada a la cota de 25.

Las ganancias finales, tras el refinamiento experimental, se reportan en la Tabla 4.4, y el desempeño del lazo cerrado, en la Tabla 5.3.

La clase PSO no fija una semilla pseudoaleatoria, por lo que estos parámetros corresponden a la ejecución que genera el archivo `resultados_pso.tex` y pueden variar al repetir el *script*. Debido a que el período de captura utilizado en la identificación es T_{cap} = 0.1 s, los retardos numéricos obtenidos cumplen $L_{d,j} < T_{cap}$ y producen, en ambos casos, cero muestras de retardo ($n_{d,j}$ = 0) en la cuantización descrita en el ANEXO 4.6. Por tanto, la discretización implementada no permite

identificar con precisión retardos inferiores a un período de captura; los valores informados corresponden a la salida numérica del optimizador, pero no introducen un desplazamiento temporal efectivo en el modelo discreto.

Aporte del PSO a las ganancias finales. Este párrafo desarrolla el análisis resumido en la Sec. 5.2. Gökçe et al. (2021) emplean un costo que combina error estacionario, tiempo de subida y sobreimpulso, basado como el de este trabajo en la respuesta al escalón, pero validan en simulación las ganancias que obtienen por PSO. En este trabajo, esas ganancias se cargan en el robot, y el refinamiento aumenta K_p en 28 % y 63 %, reduce K_d en 41 % y 45 % respecto de los valores informados y aumenta K_i en varios órdenes de magnitud. El ajuste se explica por tres factores. El primero es el modelo, porque la identificación usa un único escalón de 143 niveles de PWM y una estructura lineal que no representa la zona muerta del *driver*, y la simulación del costo calcula el controlador en punto flotante, por lo que no reproduce la banda de insensibilidad de hasta 4 *ticks/frame* de las ganancias sugeridas (ANEXO 4.12). El segundo es la función objetivo, que evalúa un único escalón, no penaliza el esfuerzo de control y deja la componente asociada a K_d sobre la frontera del espacio de búsqueda. El tercero es la implementación del optimizador, que no recorta su salida ni fija una semilla (ANEXO 4.8), por lo que los K_d informados, 34.69 y 36.70, no son los que evalúa el costo, aunque según la misma rutina ambos producen la misma subida (ANEXO 4.12). Aun así, el PSO entrega un punto de partida estable, con más margen que las ganancias definitivas y que las ganancias por defecto del *firmware* original (ANEXO 4.12), del mismo orden de magnitud que las definitivas y obtenido sobre la estructura de la ley ejecutada, sin traducir ganancias de una regla analítica, como sí lo exige la sintonía Lambda del ANEXO 4.2. Por tanto, su aporte es el de una semilla de búsqueda y no el de un ajuste final validado por sí solo.

ANEXO 4.11. Captura de la respuesta en lazo cerrado

La respuesta experimental se adquiere mediante el *script* `arduino_firmware/pid_config/ControlLazoCerrado.py`. El *script* envía una referencia de 0.13 m/s mediante el comando `k`, consulta la velocidad informada por el *firmware* mediante `j` y almacena muestras con un período nominal de 0.1 s. La lectura serial se ejecuta en un hilo asíncrono y, después de solicitar `j`, el bucle copia inmediatamente el último par numérico disponible en `rawData`. Por ello, cada registro corresponde a la medición más reciente recibida, sin comprobar que haya sido actualizada específicamente para esa iteración.

ANEXO 4.12. Análisis de estabilidad del lazo de velocidad

Este anexo desarrolla el análisis de estabilidad resumido en la Sec. 4.4.3. Los cálculos se ejecutan con el *script* `AnalisisEstabilidadPID.py` (ANEXO 9), que usa solo los parámetros documentados en las Tablas 4.1, 4.4 y A4.4.

Forma incremental del PID. En la forma incremental, también llamada algoritmo de velocidad, cada muestra calcula el cambio de la señal de control y lo suma a la salida anterior (Åström & Hägglund, 2006/2009, Sec. 13.4). Con una ponderación b_{sp} del punto de consigna en la acción proporcional, la acción derivativa calculada sobre la medición y las acciones integral y derivativa aproximadas por diferencias hacia atrás, el incremento del PID es el de la Ec. (A4.1) (Åström & Hägglund, 2006/2009, Secs. 3.4, 13.3 y 13.4):

$$\Delta u[k] = K_c\left(b_{sp}\,\Delta m^{ref}[k] - \Delta m[k]\right) + \frac{K_c\,T_m}{T_i}\,e[k] - \frac{K_c\,T_d}{T_m}\,\Delta^2 m[k], \qquad u[k] = u[k-1] + \Delta u[k], \tag{A4.1}$$

donde K_c es la ganancia proporcional, T_m el período de muestreo, T_i y T_d los tiempos integral y derivativo, m^{ref} la referencia, m la medición, $e = m^{ref} - m$ el error, y Δ y Δ^2 la primera y la segunda diferencia entre muestras consecutivas. Con $b_{sp} = 0$ la referencia solo entra en el término integral, lo que evita saltos de la señal de control (Åström & Hägglund, 2006/2009, Sec. 3.4), y con $T_d = 0$ el algoritmo se reduce a un PI incremental. En la ley ejecutada, sin el término asociado a $K_{i,j}$, $K_{d,j}/K_o$ cumple el papel de K_c y $K_{p,j}/K_o$ el de $K_c T_m/T_i$. Con las ganancias definitivas, la ganancia proporcional equivalente es de 0.41 y 0.40 PWM por *tick/frame*, y el tiempo integral equivalente, $T_i = T_m K_{d,j}/K_{p,j}$, es de 42 y 40 ms en las ruedas izquierda y derecha, respectivamente.

Criterios de estabilidad. El lazo discreto es estable cuando todas las raíces de su ecuación característica tienen magnitud menor que uno. Para estimar cuánto puede diferir la planta real del modelo, se calculan además el margen de ganancia, que indica cuántas veces puede crecer la ganancia del lazo, el margen de fase, que indica cuánto atraso adicional tolera, y el margen de retardo, que indica el máximo retardo adicional que admite antes de perder la estabilidad. La sensibilidad máxima M_s resume la robustez en un solo número, y sus valores típicos se encuentran entre 1.2 y 2.0 (Wittenmark et al., 2002, Sec. 5).

Planteamiento. El análisis se aplica al lazo discreto que forman las Ecs. (4.1) a (4.3) con el modelo FOPDT identificado de cada rueda, discretizado de forma exacta con el período $T_{PID,sched} = 33$ ms. El modelo discreto incluye el retardo $L_{d,j}$, el retenedor de orden cero del PWM y la medición del *firmware*, que entrega la

velocidad media del período anterior. Como el período de captura no resuelve el retardo (ANEXO 4.6), el análisis se realiza sin retardo, que es el modelo efectivamente ajustado, y en un escenario de sensibilidad con $L_{d,j}$ = 0.1 s. Ese segundo valor no se presenta como una cota física medida. El barrido del *script* explora el intervalo entre ambos escenarios, sin convertirlo en un rango experimental de incertidumbre. El análisis es lineal, por lo que no incluye la saturación, la zona muerta del *driver* ni los truncamientos enteros, que se comentan al final de este anexo.

Resultados. La Tabla A4.5 resume los resultados con las ganancias definitivas. En la rueda izquierda se informan dos casos, sin el término asociado a $K_{i,L}$, que corresponde a la operación cerca del equilibrio, y con ese término tratado como lineal, que es el caso más desfavorable. En la rueda derecha ese término no tiene efecto, porque su estado integral permanece en cero.

Tabla A4.5: Estabilidad y robustez de los modelos lineales del lazo de velocidad con las ganancias definitivas en los escenarios de retardo de 0 y 0.1 s.

Indicador	**Izquierda sin $K_{i,L}$**	**Izquierda con $K_{i,L}$**	**Derecha**
Sin retardo, $L_{d,j}$ = 0 *(modelo efectivamente ajustado)*			
Mayor magnitud de los polos de lazo cerrado	0.947	0.995	0.950
Margen de fase	56.9°	53.4°	55.5°
Margen de ganancia	38.9 dB	38.9 dB	39.4 dB
Margen de retardo	0.45 s	0.42 s	0.44 s
Sensibilidad máxima M_s	1.35	1.39	1.38
Escenario de sensibilidad, $L_{d,j}$ = 0.1 *s*			
Mayor magnitud de los polos de lazo cerrado	0.961	0.995	0.964
Margen de fase	44.1°	40.7°	42.9°
Margen de ganancia	13.4 dB	13.1 dB	13.3 dB
Margen de retardo	0.35 s	0.32 s	0.34 s
Sensibilidad máxima M_s	1.78	1.85	1.81

En los tres casos, todas las raíces quedan dentro del círculo unitario, tanto sin retardo como en el escenario de 0.1 s, y la sensibilidad máxima, de 1.35 a 1.85, queda dentro del intervalo típico. Los márgenes de retardo adicional calculados son de 0.42 a 0.45 s para el modelo sin retardo y de 0.32 a 0.35 s para el escenario de 0.1 s. Son propiedades de los modelos lineales con sus parámetros fijados; no determinan el retardo físico del robot ni prueban la estabilidad global del controlador no lineal. Un barrido de 625 combinaciones de la ganancia de la

planta, entre 0.1 y 10 veces la identificada, y de su constante de tiempo, entre 0.05 y 3 s, muestra además que sin retardo el lazo es estable en todas ellas, y que con el retardo de 0.1 s solo pierde la estabilidad con ganancias de planta mayores que 4.4 veces la identificada, o que 2.8 veces en la rueda izquierda si el término asociado a $K_{i,L}$ se trata como lineal. El barrido describe sensibilidad en esa rejilla de parámetros; no identifica el rango real de variación de la planta.

Condiciones de operación. En operación, `diff_drive_controller` limita la base a 0.13 m/s y 0.35 rad/s (Tabla A3.5), por lo que cada rueda opera hasta unos 0.16 m/s, es decir, unos 51 *ticks/frame*. Según el modelo, esa velocidad requiere en régimen permanente unos 180 de los 255 niveles de PWM, por lo que la salida no se satura fuera de los transitorios. El ensayo a 0.13 m/s corresponde al límite de velocidad lineal, y su escalón es mayor que los cambios de referencia de la navegación normal, en la que DWB limita la aceleración lineal a 0.25 m/s^2 (Tabla A5.1). Como el modelo es lineal, los márgenes no dependen de la velocidad de operación, y el barrido anterior explora variaciones hipotéticas de la ganancia y de la constante de tiempo. La caracterización de esas variaciones entre puntos de operación reales queda como trabajo futuro.

Efectos no incluidos en el análisis lineal. Como el incremento de la Ec. (4.1) se trunca a un entero, en estado estacionario la salida no cambia mientras se cumpla $|K_{p,j}e_j^{db}[k] + I_j^{int}[k]| < K_o$, suponiendo medición constante. Con $I_j^{int} = 0$, esta banda de insensibilidad deja sin corregir errores de hasta 3 *ticks/frame* con las ganancias definitivas y de hasta 4 con las sugeridas por PSO, es decir, cerca de 0.01 m/s, y el error estacionario medido, de 0.002 y 0.000 m/s (Tabla 5.3), queda dentro de esa banda calculada sin el estado adicional. Si dicho estado está cargado, el umbral cambia y puede ser asimétrico; por ejemplo, con $K_{p,L} = 16$, error 3 e $I_L^{int} = 2$, el cociente es $(16 \cdot 3 + 2)/50 = 1$ y la salida sí cambia. Asimismo, un estado cargado puede disminuir con errores de magnitud menor que 14 por el truncado de la suma completa. El análisis tampoco cubre las velocidades bajas. En un giro sobre el propio eje a 0.35 rad/s, cada rueda opera cerca de 10 *ticks/frame*, donde esa banda representa cerca de un 30 % de la referencia y la zona muerta del *driver* tiene mayor peso.

Ganancias sugeridas por PSO. Con el mismo modelo, las ganancias sugeridas por PSO dan márgenes de fase de 64° a 71° sin retardo y de 53° a 62° en el escenario de retardo de 0.1 s, y una sensibilidad máxima de 1.15 a 1.52, por lo que son más conservadoras que las definitivas. Las ganancias por defecto del *firmware* original ($K_p = 20$, $K_d = 12$ y $K_i = 0$, ANEXO 4.5) dan, en cambio, márgenes de fase de 49° a 50° sin retardo y de 35° en el escenario de 0.1 s, con una

sensibilidad máxima de hasta 2.18, fuera del intervalo típico. La rutina de simulación del *script* de sintonización, que no aplica el retardo (ANEXO 4.7), predice con las ganancias del PSO una subida de 0.8 y 1.1 s, igual con el K_d informado y con el evaluado por el costo. En el robot, en cambio, la rueda izquierda se detiene en 0.12 m/s y la derecha recién sostiene la referencia después de los 6 s (Fig. 5.3). Con las ganancias definitivas, la misma rutina predice sobreimpulsos de 10 % a 16 %, que no aparecen en el robot. El modelo, por tanto, sobrestima la rapidez del lazo real.

ANEXO 5. Configuración complementaria de SLAM, Nav2 y exploración

Las configuraciones fuente se mantienen en `articubot_one/config/`. En simulación se utilizan `nav2_params_sim.yaml` y `mapper_params_online_async_sim.yaml`, con `use_sim_time: true`; en *hardware* real se utilizan `nav2_params.yaml` y `mapper_params_online_async.yaml`, con `use_sim_time: false`.

Los identificadores de *plugin* cargados por los componentes resumidos en la Tabla 4.5 son `nav2_navfn_planner/NavfnPlanner` para el planificador global y `dwb_core::DWBLocalPlanner` para el controlador local. Los parámetros principales de ambos se consolidan en la Tabla A5.1.

Tabla A5.1: Parámetros principales de Nav2 y DWB.

Parámetro	*Hardware*	Simulación
`controller_frequency`	8.0 Hz	20.0 Hz
`max_vel_x` (DWB)	0.15 m/s	0.26 m/s
`max_vel_theta`	0.35 rad/s	1.0 rad/s
`acc_lim_x` (DWB)	0.25 m/s^2	2.5 m/s^2
`xy_goal_tolerance`	0.10 m	0.25 m
`yaw_goal_tolerance`	3.14 rad	0.25 rad
`min_vel_x` (DWB)	–0.15 m/s	0.0 m/s
Verificador de progreso	0.05 m en 6.0 s	0.5 m en 10.0 s
`use_astar`	`true`	`true`

En *hardware* real, DWB evalúa candidatos de hasta 0.15 m/s hacia adelante y hacia atrás, mientras que `diff_drive_controller` limita la velocidad lineal a 0.13 m/s. Por ello, las consignas superiores a este último valor se limitan en la etapa posterior. La tolerancia angular de 3.14 rad equivale a no exigir una orientación final, de modo que una meta se considera alcanzada solo por su posición. El verificador de progreso, configurado con `required_movement_radius` y

movement_time_allowance, declara una falla del controlador cuando el robot no se desplaza 0.05 m en 6.0 s.

ANEXO 5.1. Lanzamiento de la localización con AMCL

El lanzamiento de localización activa el servidor de mapas (nav2_map_server) y el nodo amcl (nav2_amcl), gestionados como nodos de ciclo de vida por nav2_lifecycle_manager. AMCL estima la pose a partir del mapa de ocupación almacenado, la odometría, los barridos del tópico /scan y las transformaciones del robot. Este lanzamiento es independiente del de slam_toolbox y no se ejecuta durante las pruebas de mapeo y exploración descritas en la Sec. 5.3.

ANEXO 5.2. Geometría del *costmap* e inflación

Los mapas de costo real utilizan un *footprint* rectangular respecto de base_link, con vértices en (0.20, 0.13), (–0.08, 0.13), (–0.08, –0.13) y (0.20, –0.13) m. La distancia mínima desde el origen a sus bordes es aproximadamente 0.08 m, correspondiente al borde posterior. Con radio de inflación $r_{inflacion}$ = 0.15 m y factor de decaimiento β = 5.0 m^{-1} (cost_scaling_factor), la capa asigna un costo entero de 254 en la celda del obstáculo (letal), 253 dentro del radio inscrito, un valor con decaimiento exponencial en la distancia d_o entre el radio inscrito y el de inflación, y 0 más allá del radio de inflación.

La esquina delantera del *footprint* queda a unos 0.24 m de base_link, por lo que el radio circunscrito supera al radio de inflación. Por eso, las celdas situadas entre 0.15 m y 0.24 m de un obstáculo tienen costo nulo aunque la esquina del robot pueda alcanzarlas. El planificador global trata al robot como un punto sobre el mapa de costos, y el crítico BaseObstacle del controlador DWB evalúa solo la celda bajo el centro del robot. Ninguno de los dos comprueba el contorno completo, así que la configuración no garantiza que la ruta deje espacio para la huella rotada del robot en pasos estrechos. Su efecto en las ejecuciones informadas no se cuantifica.

ANEXO 5.3. Acciones de recuperación de Nav2

Tabla A5.2: Acciones de recuperación configuradas en Nav2.

Acción	Condición o alcance	Parámetro configurado
Clear global costmap	Fallo del planificador o recuperación global	Radio de limpieza de 1.5 m
Clear local costmap	Fallo del seguimiento o recuperación global	Radio de limpieza de 1.0 m
BackUp	Secuencia de recuperación global	0.15 m a 0.05 m/s; máximo 8.0 s
Spin	Secuencia de recuperación global	Giro de 0.8 rad
Wait	Fin de la secuencia antes del reintento	2 s

ANEXO 5.4. Parámetros principales de SLAM en *hardware* real

En el robot real se ejecuta `async_slam_toolbox_node`, con el archivo `mapper_params_online_async.yaml` del directorio de configuración (ANEXO 9). La Tabla A5.3 resume los marcos de referencia, la entrada sensorial, la resolución, los umbrales de movimiento y los intervalos configurados. El nombre del nodo y el archivo identifican cómo se inicia el sistema; no son parámetros del algoritmo.

Tabla A5.3: Parámetros principales de `slam_toolbox` cargados en *hardware* real.

Parámetro	Valor	Función
`solver_plugin`	`solver_plugins::CeresSolver`	Solucionador numérico para optimizar el grafo de poses.
`mode`	`mapping`	Construcción del mapa durante el desplazamiento.
`odom_frame`	`odom`	Marco local continuo de la odometría.
`map_frame`	`map`	Marco global en el que se expresa el mapa.
`base_frame`	`base_link`	Marco del robot usado para transformar los barridos.
`scan_topic`	`/scan`	Tópico de entrada de los barridos del LIDAR.
`resolution`	0.05 m/celda	Tamaño de celda del mapa de ocupación; no indica su exactitud.
`max_laser_range`	10.0 m	Alcance máximo de rasterización del mapa de ocupación.
`use_scan_matching`	`true`	Refina la pose odométrica mediante correspondencia de barridos.
`minimum_travel_distance`	0.05 m	Umbral de traslación configurado para incorporar nuevas observaciones.
`minimum_travel_heading`	0.05 rad	Umbral angular configurado para incorporar nuevas observaciones.
`map_update_interval`	1.0 s	Intervalo configurado de actualización del mapa de ocupación.
`transform_publish_period`	0.02 s	Período configurado de publicación de la transformación `map`→`odom`.
`do_loop_closing`	`true`	Habilita la búsqueda de cierres de lazo en el grafo.

Nota: Los intervalos son valores de configuración, no mediciones de frecuencia efectiva. Habilitar el cierre de lazo no demuestra que ocurra en los ensayos; su detección automática no forma parte de los indicadores registrados (Sec. 5.3).

ANEXO 5.5. Parámetros y flujo operativo de exploración

Tabla A5.4: Parámetros principales del explorador de fronteras.

Parámetro	Valor	Función
`robot_base_frame`	`base_link`	Marco base del robot
`costmap_topic`	`/global_costmap/costmap`	*Costmap* usado para detectar fronteras
`planner_frequency`	0.3 Hz	Valor configurado; período efectivo de 3333 ms ($\approx$ 0.30003 Hz)
`progress_timeout`	10.0 s	Tiempo máximo sin progreso
`gain_scale`	1.0	Peso de ganancia de información en el costo de frontera (ANEXO 5.6)
`potential_scale`	3.0	Peso de distancia a la frontera en el costo de frontera (ANEXO 5.6)
`orientation_scale`	0.0	Parámetro cargado, pero no utilizado por `FrontierSearch` en el código ejecutado
`min_frontier_size`	0.05 m	Tamaño mínimo de una frontera válida
`return_to_init`	`false` (valor por defecto)	No se carga en `explore.yaml`; no se ejecuta el retorno a la pose inicial
`resolution` (*costmap*)	0.05 m/celda	Resolución del *costmap*, ρ_{map} en el costo de frontera (ANEXO 5.6)

ANEXO 5.6. Particularidades del explorador de fronteras

El detector inicia la búsqueda desde una celda libre próxima al robot. Al expandir una celda de la BFS, el código incorpora un vecino de cuatro conexiones únicamente cuando todavía no se ha visitado y se cumple `map_[nbr] <= map_[idx]`; si esa condición no se cumple, evalúa el vecino como posible celda de frontera. Una celda desconocida constituye una celda de frontera cuando posee al menos un vecino de cuatro conexiones clasificado como espacio libre. Las celdas de frontera contiguas mediante ocho conexiones se agrupan en una misma frontera.

La construcción implementada inicializa el tamaño de la frontera en uno para representar la celda inicial p_0, pero esa celda no se incorpora a la suma que calcula el punto denominado `centroid`: el punto enviado como centroide es la suma de las coordenadas de las celdas añadidas después de p_0 dividida por $1 + |F_{add}|$, donde $|F_{add}|$ es el número de esas celdas añadidas. Por incluir la celda

inicial en el divisor pero no en la suma, el punto almacenado p_F^{impl} no es el centroide geométrico convencional de todas las celdas de la frontera. La misma celda inicial tampoco participa en el cálculo de `min_distance`.

La variable `frontier.min_distance` se calcula después de convertir las celdas añadidas a coordenadas del mundo, por lo que ya se encuentra en metros. Aun así, la función de costo implementada asigna a cada frontera F un puntaje U(F) restando a un término de distancia un término de ganancia: toma esa distancia mínima ya métrica $d_{min}^{world}(F)$ [m], la vuelve a multiplicar por la resolución del *costmap* ρ_{map} [m/celda] y la pondera por w_{pot}; y le resta el número de celdas almacenado |F| multiplicado también por ρ_{map} y ponderado por w_{gain}. Los valores configurados de ambos pesos y del tamaño mínimo de frontera corresponden a la Tabla A5.4. Como la distancia ya está en metros, esa segunda multiplicación por ρ_{map} deja el primer término escalado en m^2/celda y el segundo en metros, de modo que U(F) es un puntaje heurístico no homogéneo. Esta diferencia se mantiene porque corresponde al código ejecutado.

El ordenamiento, la selección y el envío de la meta se formalizan en el Alg. 4.4. Con resolución 0.05 m/celda y `min_frontier_size`=0.05 m, el criterio $|F|\rho_{map} \geq 0.05$ admite desde un tamaño almacenado de una celda. Si una frontera no incorpora vecinos después de la celda inicial, conserva |F| = 1, $p_F^{impl} = (0, 0)$ y $d_{min}^{world} = \infty$, porque la celda inicial no se suma ni se utiliza para actualizar la distancia. Por tanto, su costo es infinito; aun así, puede permanecer como candidata si no existe otra frontera seleccionable. Dos objetivos se consideran iguales cuando su distancia euclidiana es inferior a 0.01 m.

El parámetro configurado es f_{exp} = 0.3 Hz, pero el temporizador se construye por conversión a milisegundos enteros sin redondeo, lo que da un período de 3333 ms, equivalente a una frecuencia efectiva de aproximadamente 0.30003 Hz. La gestión de la lista negra se formaliza en el Alg. 4.4; como particularidad de la implementación, la comprobación de pertenencia utiliza tolerancias independientes en x e y iguales a cinco celdas del *costmap*.

ANEXO 6. Detalle de validación de odometría

Este anexo recoge el detalle por repetición de las pruebas resumidas en la Sec. 5.1: trayectoria rectilínea de 2 m (medias en la Tabla 5.1) y protocolo angular con referencia nominal de 360° (medias en la Tabla 5.2). Se presentan las cinco repeticiones por entorno.

Los *scripts* de la prueba angular publican una consigna de 0.4 rad/s durante 2π/0.4 s directamente sobre el tópico de entrada del controlador diferencial, lo que corresponde a la referencia nominal de 360° empleada por los *scripts* de análisis. Sin embargo, la configuración del controlador satura la velocidad angular

en 0.35 rad/s (Tabla 4.3); bajo ese límite, mantenido de forma ideal y constante, el giro teórico alcanzable es $0.35\,(2\pi/0.4)$ rad, equivalente a 315°. Sin embargo, todos los giros registrados superan ese valor, tanto el giro acumulado de `/odom` en simulación (Tabla A6.3) como el giro físico medido en *hardware* (Tabla A6.4). Esto indica que la velocidad angular efectiva no queda limitada de forma ideal a 0.35 rad/s durante los ensayos. Los registros *rosbag* de cada prueba contienen `/odom`, cuya velocidad angular permitiría verificar la velocidad efectivamente aplicada sin repetir los ensayos.

En simulación, la interpretación del error rectilíneo y la condición del protocolo angular corresponden a las definidas en la Sec. 5.1. Como tratamiento exacto de la orientación, `prueba_giro_sim.py` reemplaza la diferencia normalizada de orientación de Gazebo Classic por el giro acumulado de `/odom` cuando aquella es menor que 10°; por ello, la tabla simulada no contiene una verdad de terreno angular independiente.

ANEXO 6.1. Trayectoria rectilínea de 2 m

La Tabla A6.1 presenta los resultados en simulación y la Tabla A6.2 los obtenidos sobre *hardware* real.

Tabla A6.1: Desviación de la odometría simulada respecto de la referencia nominal de 2 m (Gazebo Classic). La pose verdadera del simulador no queda registrada.

Prueba	Dist. teórica (m)	Dist. odom. (m)	Desviación (%)
1	2.00	1.866	6.69 %
2	2.00	1.954	2.28 %
3	2.00	1.946	2.72 %
4	2.00	1.923	3.83 %
5	2.00	1.972	1.40 %

Tabla A6.2: Error de odometría en trayectoria rectilínea de 2 m en *hardware* real. El error corresponde a la discrepancia entre la odometría de *encoders* y la distancia física medida.

Prueba	Dist. teórica (m)	Dist. odom. (m)	Dist. real (m)	E_{odom} (%)
1	2.00	1.920	1.846	4.01 %
2	2.00	1.922	1.868	2.88 %
3	2.00	1.923	1.851	3.87 %
4	2.00	1.926	1.856	3.78 %
5	2.00	1.928	1.861	3.61 %

ANEXO 6.2. Protocolo angular con referencia nominal de 360°

Tabla A6.3: Desviación del giro acumulado de `/odom` respecto de la referencia nominal de 360° en simulación.

Prueba	Ref. nominal (°)	Giro acumulado `/odom` (°)	Desviación (°)	$D_{\theta,360}$ (%)
1	360.0	335.86	-24.14	6.71 %
2	360.0	347.40	-12.60	3.50 %
3	360.0	346.62	-13.38	3.72 %
4	360.0	347.40	-12.60	3.50 %
5	360.0	337.41	-22.59	6.28 %

Tabla A6.4: Desviación angular física respecto de la referencia nominal de 360° usada por el *script* en *hardware* real. Las desviaciones son bidireccionales (sobre- y subgiros).

Prueba	Ref. nominal (°)	Giro físico medido (°)	Desviación (°)
1	360.0	355.7	-4.3
2	360.0	340.0	-20.0
3	360.0	380.0	+20.0
4	360.0	370.0	+10.0
5	360.0	356.0	-4.0

ANEXO 7. Resultados complementarios de exploración y mapeo

Este anexo presenta los registros por iteración de la exploración autónoma en *hardware* real obtenidos mediante el *framework* `sesiones_mapeo`. Cada ejecución parte de un reinicio de SLAM y Nav2. La definición y la interpretación de la cobertura y del área libre mapeada son las establecidas en la Sec. 5.3; aquí se conservan los valores desagregados de la sesión realizada el 22 de junio de 2026.

Se ejecutan seis iteraciones. La iteración 02 se excluye del análisis. Durante esa ejecución, que se extiende por 5.09 min, se constata que el robot no se desplaza de su posición inicial, mientras que la odometría de *encoders* acumula 3.851 m. La cercanía de ese valor con los 3.850 m de la iteración 5 no proviene de una mezcla de registros, porque cada distancia se calcula sobre el *rosbag* de su propia carpeta, con duraciones de 5.09 y 2.50 min. El explorador despacha 14 metas de navegación, de las cuales 13 terminan en aborto y una queda sin

estado terminal, y se registran 13 avisos de lista de fronteras agotada. La imagen de ocupación tampoco queda almacenada al término de la sesión, por lo que no es posible calcular la cobertura ni el área libre mapeada. La causa de la inmovilidad no se determina, y la iteración no se incorpora al recuento de metas ni a los promedios de la Tabla 5.4.

Este episodio ilustra de forma directa la limitación de una odometría basada únicamente en *encoders*, que mide el giro de las ruedas y no el desplazamiento efectivo del vehículo. En las cinco iteraciones conservadas, en cambio, el desplazamiento queda respaldado por evidencia independiente de los *encoders*, porque el mapa crece hasta coberturas de entre 53.4 % y 82.9 % y ese crecimiento exige que el sensor láser observe geometría nueva desde posiciones distintas. Del mismo modo, el error odométrico informado en la Sec. 5.1 se calcula contra una distancia medida con cinta métrica y no contra la propia odometría.

ANEXO 7.1. Exploración autónoma por iteración

Tabla A7.1: Resultado de las metas de navegación de la exploración, reconstruidas por identificador único a partir del flujo de estado de la acción `NavigateToPose` registrado en cada sesión. Las cinco iteraciones terminan al no quedar fronteras.

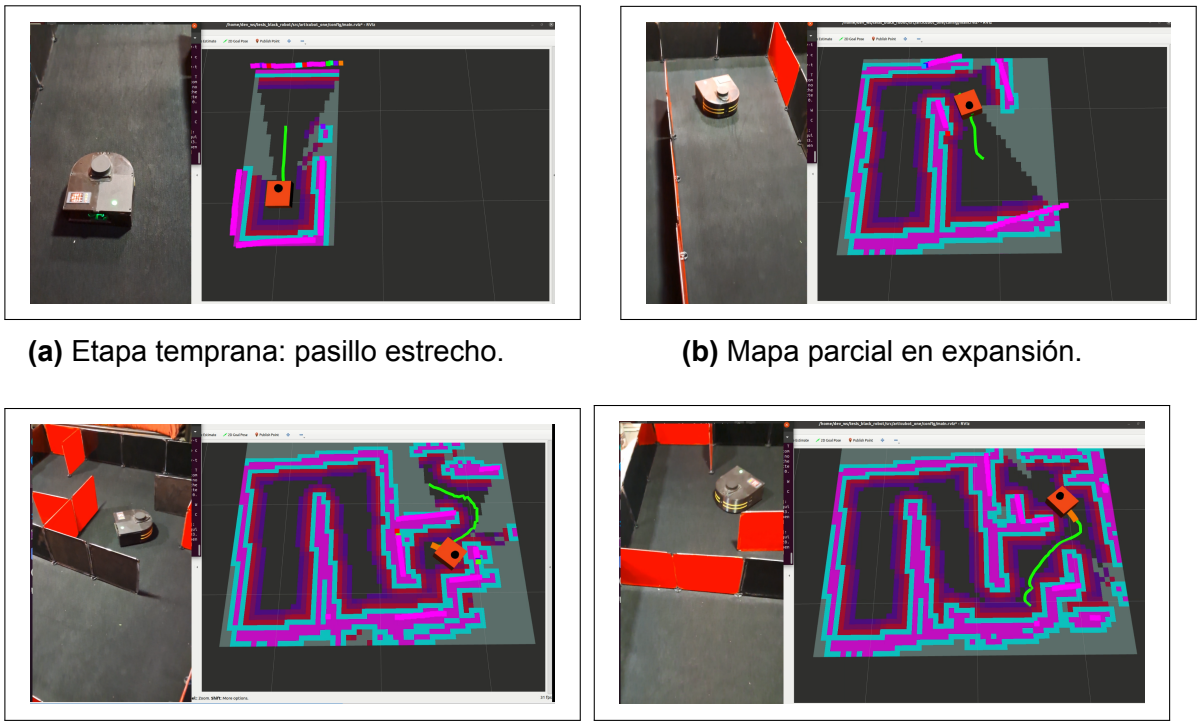
Iter.	Metas generadas	Abortadas	Abortos asociados a reemplazos	Exitosas	Sin estado terminal	Mediana entre metas (s)
1	18	17	16	0	1	3.3
3	9	8	8	0	1	3.3
4	12	11	11	0	1	3.3
5	11	9	9	1	1	3.3
6	10	9	9	0	1	3.3
Total	**60**	**54**	**53**	**1**	**5**	—

De las 60 metas, 59 alcanzan el estado de ejecución. La restante es la primera meta de la iteración 1, que ya figura abortada en el primer mensaje registrado y por eso tampoco cumple el criterio temporal de asociación a reemplazo. Las metas sin estado terminal son la última de cada ejecución. Al no encontrar fronteras, el explorador solicita automáticamente su cancelación, y el *script* de ensayo cierra el registro antes de que esa cancelación termine. El intervalo informado se calcula entre la aceptación de una meta y la de la siguiente en los casos asociados a reemplazos; no es una medición independiente del instante de finalización de la anterior. Su máximo es de 16.7 s en la iteración 1 y de 6.7 s en las demás. La reconstrucción se obtiene agrupando los mensajes de estado de la acción por el identificador de meta de 16 bytes.

Los estadísticos agregados por iteración se presentan en la Tabla 5.4 y la interpretación de los contadores en la Sec. 5.4. Las intervenciones manuales, registradas por el operador, son nulas en las cinco iteraciones válidas.

La Fig. A7.1 muestra la secuencia completa de una ejecución representativa, desde el mapeo inicial de un pasillo estrecho hasta la mayor cobertura alcanzada, junto con la ruta planificada por Nav2 hacia las fronteras seleccionadas.

El video de la ejecución se cita en la Sec. 5.4.



(a) Etapa temprana: pasillo estrecho.

(b) Mapa parcial en expansión.

(c) Cobertura amplia con ruta planificada activa.

(d) Mayor cobertura alcanzada al cierre de la exploración.

Figura A7.1: Secuencia de una ejecución representativa de exploración por fronteras y navegación autónoma sobre *hardware* real (vista de RViz y cámara cenital del recinto).

Fuente: Elaboración propia.

ANEXO 8. Procedencia y limitaciones de las métricas complementarias

ANEXO 8.1. Procedencia de los indicadores de mapeo

La clasificación de celdas empleada en la Ec. (5.4) se realiza sobre la imagen PGM del mapa de ocupación. Siendo M el conjunto de intensidades p de las celdas de dicha imagen PGM, los conteos empleados en la Ec. (5.4) se obtienen con la Ec. (A8.1), mediante la función indicadora $\mathbf{1}[\cdot]$, que vale 1 cuando la condición entre corchetes se cumple y 0 en caso contrario:

$$N_{libre} = \sum_{p \in M} \mathbf{1}\left[p \geq 230\right], \qquad N_{desconocido} = \sum_{p \in M} \mathbf{1}\left[50 < p < 230\right]. \tag{A8.1}$$

Las celdas restantes ($p \leq 50$) se clasifican como ocupadas y no participan en el cálculo del índice de cobertura.

Las diferencias de lectura y redondeo entre los distintos *scripts* de análisis se originan en esta clasificación. El *script* de análisis `test/slam/analizar_slam.py` calcula a partir del *rosbag* y de la imagen PGM el tiempo, la distancia odométrica, la cobertura y el área libre. En cambio, el número de nodos se suministra mediante el argumento `--nodos`, mientras que el cierre de lazo y la alineación se incorporan mediante `--cierre` y `--alineacion` o por ingreso manual. Por ello, el número de nodos informado corresponde al dato suministrado durante el análisis y la consistencia global indicada en la tabla es una valoración visual; no se presenta como una detección automática de cierre de lazo. Para el área libre, `analizar_slam.py` lee la resolución desde el YAML del mapa, mientras `run_explore.sh` utiliza directamente ρ_{map} = 0.05 m/celda y redondea el resultado a dos decimales.

ANEXO 8.2. Comparación cualitativa de mapas

Tabla A8.1: Comparación cualitativa entre mapas de simulación y *hardware* real.

Aspecto	**Simulación**	***Hardware* real**
Nitidez de paredes	Alta	Media–alta
Ruido en espacio libre	Nulo	Bajo–moderado
Consistencia global	Alta	Media–alta
Obstáculos pequeños	Detectados	Parcialmente detectados
Zonas de sombra láser	Mínimas	Presentes

ANEXO 8.3. Mediciones no incluidas en los ensayos

Los registros entregados no contienen mediciones de tensión, temperatura, uso de CPU, memoria ni autonomía energética. Esta es la razón por la que la Sec. 5.5 deja la caracterización cuantitativa del consumo energético y de la carga computacional fuera de los ensayos realizados y la plantea como trabajo futuro.

El repositorio incluye además *scripts* preparados para ensayos que no forman parte de la evidencia del Capítulo 5, como deslizamiento de ruedas, cierre de lazo, carga de CPU, monitoreo de recursos, bloqueo y calidad del LIDAR y navegación con metas fijas. Sus directorios no contienen registros de salida, por lo que ninguno aporta mediciones a este trabajo. El *script* de navegación con metas fijas define tres escenarios con cinco repeticiones y clasifica cada repetición según el resultado de la acción `NavigateToPose` y su duración, sin medir el error de posición final. Además, sus tres poses objetivo solo están definidas en coordenadas del mundo simulado, sin un conjunto propio para el recinto real. La tolerancia que informa para el modo real, 0.22 m, no coincide con el valor de 0.10 m configurado en Nav2 (Tabla A5.1). Ese valor solo se imprime en sus salidas y no interviene en la clasificación.

Un protocolo de navegación con metas fijas requiere objetivos definidos, referencias físicas medidas y localización coherente en el marco del mapa. Esta localización puede provenir de SLAM durante el mapeo o de AMCL sobre un mapa almacenado; Nav2 admite el envío de metas mientras SLAM publica el mapa y la transformación `map`→`odom` (Nav2 Maintainers, s.f.). El lanzador de Nav2 para Foxy selecciona SLAM o localización sobre un mapa y mantiene el arranque de la navegación. El protocolo debería informar, para cada meta y repetición, el resultado de la acción, el error de posición final respecto de la marca, el tiempo hasta la meta y la longitud recorrida. Ngoc et al. (2023) miden el tiempo de llegada y el error euclidiano entre la meta y la posición final del robot en simulación, y Francis et al. (2025) describen, entre las métricas de navegación, la tasa de éxito, el tiempo hasta la meta y la tasa de éxito ponderada por la longitud del camino. Con las tres metas y cinco repeticiones que define el *script* preparado, 15 éxitos en 15 intentos permitirían afirmar con 95 % de confianza que la tasa de éxito supera el 78 %, según el intervalo de Clopper y Pearson, uno de los más usados para tasas de éxito en la evaluación de robots (Vincent et al., 2024).

ANEXO 8.4. Limitaciones de los contadores de exploración

Los contadores de texto de `run_explore.sh` no permiten calcular una tasa de éxito, por dos motivos distintos. El contador de metas generadas se alimenta de coincidencias de texto en el registro de `explore_server` y del eco del tópico de meta de la acción `NavigateToPose`. La primera fuente no produce coincidencias porque el mensaje de envío de meta está declarado en nivel *debug* en

`explore.cpp` y el nodo se ejecuta en nivel informativo. La segunda tampoco, porque ese tópico no entrega datos y el propio registro conserva la advertencia de tópico no publicado. Por eso el contador de metas generadas queda en cero. Los contadores de éxito y de fallo sí se incrementan, pero lo hacen una vez por cada línea del flujo de estado de la acción, sin conservar ni deduplicar identificadores de meta, de modo que cuentan mensajes de estado y no metas únicas.

Para obtener una medida fiable se descartan esos contadores y se reconstruye el resultado de cada meta a partir del flujo de estado de la acción `NavigateToPose`, que queda registrado íntegro en cada sesión. Cada meta se identifica por su identificador único de 16 bytes. El clasificador conserva el primer estado terminal observado de cada identificador y deja sin estado terminal a los que no lo presentan. El indicador de metas ejecutadas registra si una meta alcanza ese estado en algún momento del registro. El resultado agregado se presenta en la Tabla A7.1.

Para asociar temporalmente los abortos a posibles reemplazos del explorador se usa el orden de los mensajes de estado. Una meta abortada cumple el criterio cuando existe un mensaje en el que la meta siguiente ya figura y la anterior sigue aceptada o en ejecución. El estado de la nueva meta no se restringe a ejecución. La coexistencia es compatible con el mecanismo de reemplazo, pero no determina por sí sola la causa del aborto ni descarta fallos concurrentes de Nav2. No se utiliza la hora escrita en cada línea del registro, porque tiene resolución de un segundo y corresponde al momento de escritura y no al de publicación del mensaje. Los tiempos se calculan con el sello de aceptación que cada meta incluye en el mensaje. El procedimiento está implementado en el *script* `clasificar_metas_exploracion.py`. El comportamiento de Nav2 ante una meta nueva se verifica en los archivos `simple_action_server.hpp` y `bt_navigator.cpp` de la rama `foxy-devel` de Nav2 (Nav2 Maintainers, 2021).

ANEXO 8.5. Comparación de los mapas con el plano del recinto

Para evaluar la exactitud de los mapas de la Sec. 5.3, se levanta un plano del recinto con las distancias entre muros medidas manualmente con huincha de medir, que se presenta en la Fig. A8.1. Sus muros tienen menos de 2 cm de espesor y se representan como líneas. El largo del tabique corto unido al muro inferior no se mide, por lo que la Tabla A8.3 solo incluye su posición.

La comparación usa los cinco mapas válidos, en las imágenes de cobertura que genera `analizar_slam.py` a partir de los mapas registrados en ROS 2, de las que se recupera la clase de cada celda de 0.05 m con los umbrales de la Ec. (A8.1). Los conteos recuperados reproducen el índice de cobertura y el área libre de la Tabla 5.4, con una diferencia máxima de dos celdas. Cada mapa se

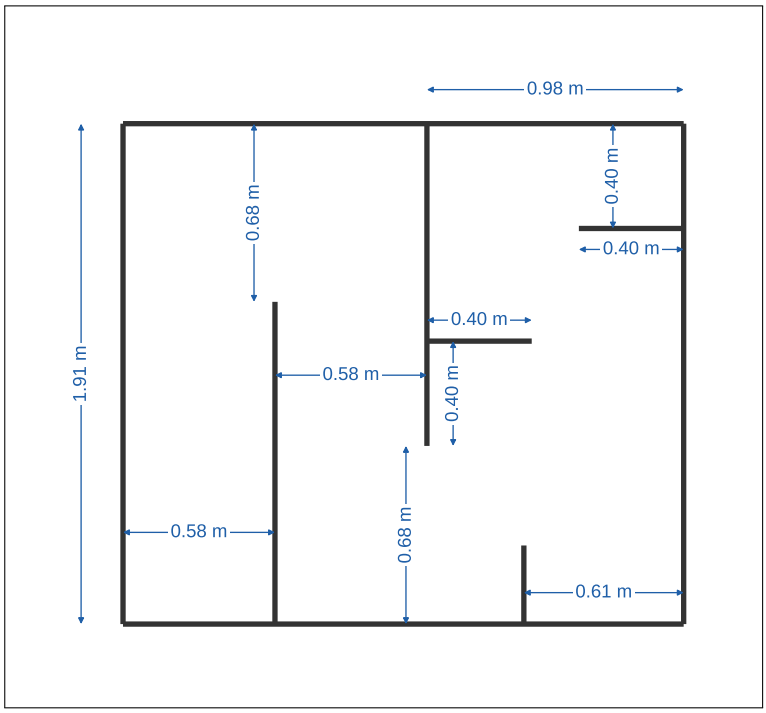


Figura A8.1: Plano del recinto de ensayo, con las distancias entre muros medidas con huincha de medir.

Fuente: Elaboración propia.

alinea con el plano mediante una rotación y una traslación, sin ajustar su escala, de modo que un error de escala del mapa queda reflejado en las distancias. Con el plano alineado, se toma un punto cada 1 cm a lo largo de los muros y se calcula su distancia a la celda ocupada más cercana del mapa, que es la métrica de exactitud que emplean Trejos et al. (2022). Solo se consideran los puntos que el robot observa, es decir, los que tienen una celda libre a menos de 0.10 m, que son entre el 92 % y el 93 % del total.

La distancia media entre el plano y el mapa es de 3.4 cm, con 3.55, 3.19, 3.72, 3.53 y 3.17 cm en las iteraciones 1, 3, 4, 5 y 6, del orden de la discretización de 0.05 m/celda. La resolución no establece una cota del error geométrico de los muros. La Tabla A8.3 compara las distancias del plano con las medidas en los mapas, y la Fig. A8.2 muestra cada mapa alineado con el plano. El alto del recinto, los dos primeros pasillos y los pasos que dejan los muros largos coinciden con diferencias de –2 a +4 cm, menores que una celda, y el tercer pasillo resulta 5 cm más ancho. En total, los tres pasillos suman 12 cm más que en el plano. Como el plano registra distancias libres entre muros, el espesor de los dos muros largos explica a lo sumo 4 cm de esa diferencia, y el resto puede provenir de la medición manual o de una distorsión del mapa en esa dirección, causas que estos datos

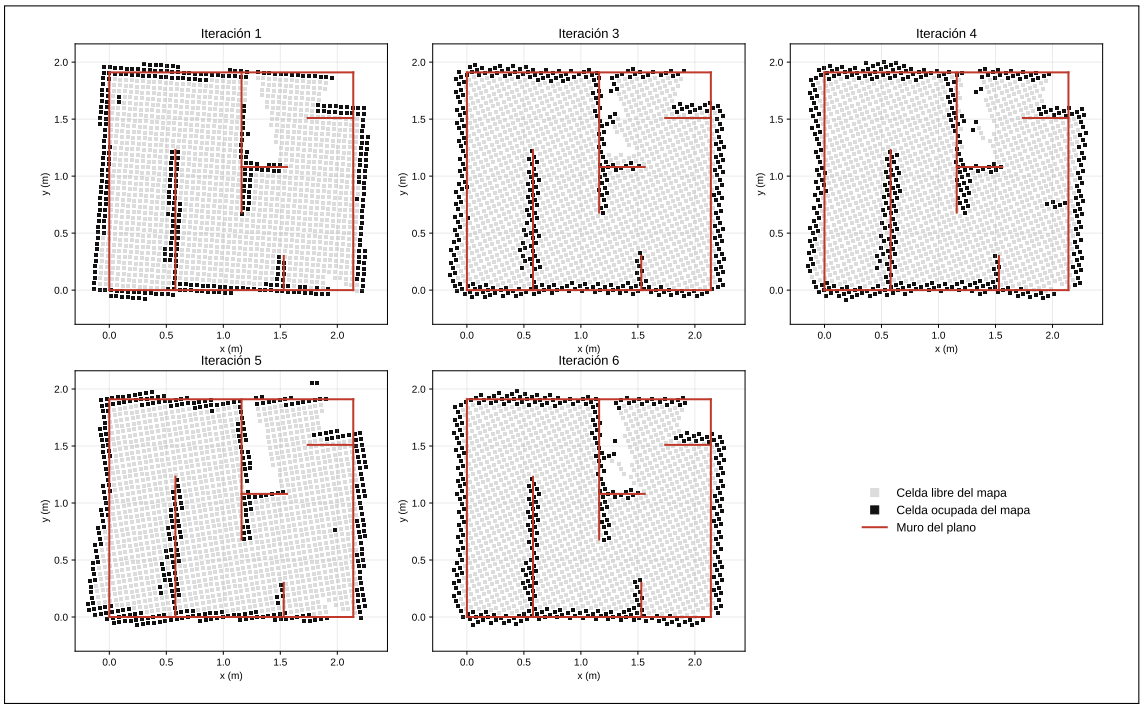


Figura A8.2: Mapas de las cinco iteraciones válidas alineados con el plano del recinto.

Fuente: Elaboración propia.

no permiten separar. Las mayores diferencias, de 6 a 9 cm, corresponden a los tabiques cortos, que quedan representados por pocas celdas ocupadas.

El informe existente `resultados_comparacion_plano.txt`, generado por `comparar_mapa_plano.py` (ANEXO 9), también registra distancias en el sentido inverso y percentiles. La Tabla A8.2 transcribe esos resultados de las mismas cinco ejecuciones; no añade ensayos ni una nueva adquisición de mapas.

Tabla A8.2: Distancias geométricas en ambos sentidos y percentil 95, transcritos del informe de comparación existente.

Iter.	Plano → mapa Media (cm)	Plano → mapa p95 (cm)	Mapa → plano Media (cm)	Mapa → plano p95 (cm)
1	3.55	9.3	4.23	10.4
3	3.19	7.3	4.30	10.6
4	3.72	9.6	4.94	12.2
5	3.53	9.3	4.30	11.0
6	3.17	6.8	4.09	9.7

El percentil 95 conjunto plano→mapa registrado es de 8.4 cm; corresponde a las distancias de los puntos incluidos, no al promedio de los percentiles por mapa. El sentido inverso utiliza las celdas ocupadas dentro de la franja definida por `zona_evaluada` alrededor de los muros de referencia, por lo que no evalúa toda ocupación fuera del recinto. Los percentiles describen la distribución espacial de distancias y no son intervalos de confianza. Para el intervalo entre ejecuciones se mantiene n = 5, como en la Tabla A8.4; ni ese intervalo ni los percentiles incluyen automáticamente la incertidumbre de la huincha o del alineamiento.

Tabla A8.3: Distancias del plano del recinto y las mismas distancias medidas en los mapas de las cinco iteraciones válidas.

Distancia	Plano (m)	Media de los mapas (m)	Rango de los mapas (m)	Diferencia media (cm)
Alto interior del recinto	1.91	1.89	1.89 a 1.90	–1.7
Ancho del primer pasillo	0.58	0.62	0.61 a 0.64	+4.1
Ancho del segundo pasillo	0.58	0.60	0.59 a 0.61	+2.4
Ancho del tercer pasillo	0.98	1.03	1.02 a 1.05	+5.2
Paso sobre el primer muro largo	0.68	0.69	0.68 a 0.69	+0.5
Paso bajo el segundo muro largo	0.68	0.67	0.67 a 0.68	–0.8
Largo del tabique unido al segundo muro largo	0.40	0.34	0.32 a 0.38	–6.1
Altura de ese tabique sobre el extremo del muro	0.40	0.39	0.38 a 0.40	–0.5
Distancia del tabique superior del tercer pasillo al muro superior	0.40	0.32	0.31 a 0.34	–7.8
Largo del tabique superior del tercer pasillo	0.40	0.38	0.34 a 0.41	–2.3
Distancia del tabique corto inferior al muro derecho	0.61	0.70	0.69 a 0.71	+9.3

Las distancias de los mapas se miden entre las celdas ocupadas de cada muro, después de alinear el plano con cada mapa. La resolución del mapa es de 0.05 m/celda.

ANEXO 8.6. Intervalos de confianza de los indicadores

Este anexo desarrolla el alcance estadístico resumido en la Sec. 5.6. La Tabla A8.4 presenta el intervalo de confianza al 95 % de la media de cada indicador, calculado con la Ec. (A8.2), que usa la distribución t de Student (Devore et al., 2021, Sec. 8.2):

$$\mu \pm t_{0.975,\,n-1}\,\frac{\sigma}{\sqrt{n}}, \tag{A8.2}$$

donde μ y σ son la media y la desviación estándar muestrales de las n repeticiones y $t_{0.975,\,n-1}$ el cuantil de la distribución t, igual a 2.776 para n = 5. Para la proporción de exploraciones válidas se usa el intervalo de Clopper y Pearson, uno de los más usados para tasas de éxito en la evaluación de robots (Vincent et al., 2024).

Tabla A8.4: Intervalos de confianza al 95 % de los indicadores obtenidos sobre *hardware* real.

Indicador	n	**Media**	**Intervalo al 95 %**
Error odométrico E_{odom} (%)	5	3.63	3.08 a 4.18
Error de seguimiento E_{seg} (%)	5	7.18	6.65 a 7.71
Desviación angular respecto de 360° (°)	5	11.7	1.8 a 21.6
Índice de cobertura libre C_{libre} (%)	5	62.2	46.2 a 78.3
Área libre mapeada A_{libre} (m^2)	5	3.50	3.46 a 3.53
Distancia media del plano del recinto al mapa (cm)	5	3.4	3.1 a 3.7
Exploraciones válidas (%)	6	83.3	35.9 a 99.6

Intervalos de la media con la distribución t de Student (Ec. (A8.2)), salvo la proporción de exploraciones válidas, que usa el intervalo de Clopper y Pearson. Los intervalos se calculan con el *script* del ANEXO 4.12, salvo el de la distancia al plano, que usa los valores por iteración del ANEXO 8.5.

Los indicadores con baja dispersión, como el error odométrico, el área libre mapeada y la distancia al plano, quedan bien acotados aun con cinco repeticiones, mientras que la cobertura libre y la desviación angular tienen intervalos amplios. Con seis ejecuciones, la proporción de exploraciones válidas tiene un intervalo amplio, de 35.9 % a 99.6 %. Como contraste, para cinco éxitos en seis ejecuciones bajo la hipótesis nula de una proporción de éxito de 50 %, la prueba binomial exacta unilateral entrega p = 7/64 = 0.109375 (Devore et al., 2021, Cap. 9), por lo que al nivel de 0.05 no se rechaza esa hipótesis. Con seis ejecuciones, el único resultado que permitiría rechazarla es el de seis éxitos, con p = 1/64, de modo que una sola falla basta para perder la significancia. Por eso el número de ensayos se considera bajo, lo que, junto con la amplitud de los intervalos y el alcance experimental, motiva ampliar la campaña.

Se considera admisible un mínimo de 6 a 10 repeticiones por ensayo, con 10 por condición como objetivo de partida. Ese criterio es coherente con la guía Eurachem de validación de métodos analíticos, que considera necesarias entre 6 y 15 repeticiones para estimar la desviación estándar y señala que a menudo se recomiendan 10 (Cantwell, 2025, Sec. 5.3.2), aunque no se refiere a ensayos

robóticos. El total final depende de la dispersión y de la precisión buscada. Con diez repeticiones y la misma dispersión, la semiamplitud de los intervalos de la Ec. (A8.2) se reduciría en cerca de un 42 %, porque el factor $t_{0.975,\,n-1}/\sqrt{n}$ baja de 1.24 a 0.72. Metas de precisión más exigentes requieren más ensayos. Reducir a la mitad la semiamplitud del intervalo del error odométrico requeriría unas 13 repeticiones, y afirmar una tasa de éxito superior al 80 % requeriría al menos 17 ejecuciones consecutivas sin fallas. En la literatura, Liu et al. (2025) realizan diez ejecuciones por algoritmo en su ensayo real, Placed et al. (2023) señalan que faltan métricas y bancos de prueba estandarizados para comparar sistemas de exploración, y Trejos et al. (2022) acompañan la comparación de algoritmos de SLAM con pruebas estadísticas y niveles de confianza.

ANEXO 9. Convención de rutas y localización del *software*

Las rutas de código citadas en la tesis son relativas al directorio raíz `src/` del repositorio. En el cuerpo del documento se indica la ruta completa en la primera mención relevante y, en las menciones posteriores, se conserva solo el nombre del archivo cuando no existe ambigüedad. Las copias almacenadas bajo `test/sesiones_mapeo/` corresponden a instantáneas de cada ensayo; salvo indicación expresa, la configuración fuente es la ubicada en `articubot_one/config/`. La Tabla A9.1 consolida la localización y la función de todos los archivos de código citados en el documento.

Tabla A9.1: Localización y función de los archivos citados en el documento.

Archivo	**Directorio padre relativo a `src/`**	**Contexto de uso**
`robot.urdf.xacro`	`articubot_one/description/`	Modelo principal URDF/`xacro`.
`robot_core.xacro`	`articubot_one/description/`	Geometría, eslabones y articulaciones.
`lidar.xacro`, `lidar_sim.xacro`	`articubot_one/description/`	Sensor real y modelo simulado del LIDAR.
`rplidar.launch.py`	`articubot_one/launch/`	Arranque del nodo de adquisición del LIDAR en *hardware* real.
`rplidar_serial_publisher.py`	`articubot_one/scripts/`	Nodo propio de lectura serie del RPLidar A1 y publicación de `/scan`.

Archivo	Directorio padre relativo a `src/`	Contexto de uso
`gazebo_control.xacro`	`articubot_one/description/`	*Backend* alternativo no seleccionado.
`ros2_control.xacro`	`articubot_one/description/`	Selección del *backend* de `ros2_control` y parámetros del enlace serial.
`launch_robot.launch.py`	`articubot_one/launch/`	Arranque del robot físico y controladores.
`launch_sim.launch.py`	`articubot_one/launch/`	Arranque del gemelo digital en Gazebo Classic.
`my_controllers.yaml`	`articubot_one/config/`	Geometría y límites del controlador diferencial.
`nav2_params.yaml`, `nav2_params_sim.yaml`	`articubot_one/config/`	Configuración Nav2 real y simulada.
`mapper_params_online_async.yaml`, `mapper_params_online_async_sim.yaml`	`articubot_one/config/`	Configuración de `slam_toolbox` real y simulada.
`explore.yaml`	`articubot_one/config/`	Parámetros del explorador de fronteras.
`obstacles.world`	`articubot_one/worlds/`	Mundo empleado en las pruebas de mapeo simulado.
`ROSArduinoBridge.ino`	`arduino_firmware/ROSArduinoBridge/`	Ciclo principal y protocolo serial del *firmware*.
`commands.h`	`arduino_firmware/ROSArduinoBridge/`	Identificadores de comandos seriales.
`diff_controller.h`	`arduino_firmware/ROSArduinoBridge/`	Controlador discreto y ganancias finales.
`encoder_driver.h`, `encoder_driver.ino`	`arduino_firmware/ROSArduinoBridge/`	Lectura de *encoders*.
`motor_driver.h`, `motor_driver.ino`	`arduino_firmware/ROSArduinoBridge/`	Accionamiento del L298N.
`ControlLazoAbierto.py`	`arduino_firmware/pid_config/`	Captura de la respuesta en lazo abierto.
`ControlLazoCerrado.py`	`arduino_firmware/pid_config/`	Captura experimental en lazo cerrado.
`SintonizacionAutomatica.py`	`arduino_firmware/pid_config/`	Identificación FOPDT y búsqueda PSO inicial.
`SintonizacionLambda.py`	`arduino_firmware/pid_config/`	Comparación simulada con sintonía Lambda.
`pyPso.py`	`arduino_firmware/pid_config/`	Implementación de la clase PSO.

Archivo	Directorio padre relativo a `src/`	Contexto de uso
`AnalisisEstabilidad PID.py`	`arduino_firmware/ pid_config/`	Análisis de estabilidad y robustez del lazo de velocidad e intervalos de confianza de los ensayos.
`resultados_pso.tex`, `resultados_lambda.tex`	`arduino_firmware/ pid_config/`	Salidas LaTeX generadas por los *scripts* de sintonización.
`prueba_giro.py`	`test/prueba_giro/`	Ejecución de la prueba angular en *hardware*.
`prueba_giro_sim.py`	`test/prueba_giro_ sim/`	Ejecución de la prueba angular en simulación.
`run_explore.sh`	`test/exploracion/`	Ejecución y registro de exploración.
`clasificar_metas_ exploracion.py`	`test/exploracion/`	Clasificación por UUID y asociación temporal de abortos a reemplazos.
`run_prueba_nav.sh`	`test/navegacion/`	Protocolo preparado de navegación con metas fijas, no ejecutado (ANEXO 8).
`analizar_slam.py`	`test/slam/`	Cálculo de métricas de mapeo.
`comparar_mapa_plano.py`	`test/mapas_vs_ plano/`	Comparación geométrica de los mapas con el plano medido.
`resultados_ comparacion_plano. txt`	`test/mapas_vs_ plano/`	Informe existente con distancias en ambos sentidos y percentiles.

ANEXO 10. Nomenclatura

Símbolo	Descripción	Unidad
Cinemática y odometría		
r	Radio de rueda	m
D = 2r	Diámetro de rueda (`WHEEL_DIAMETER`)	m
b	Separación entre ruedas (`wheel_separation`)	m
$q_k = (x_k, y_k, \theta_k)$	Pose del robot en el instante k	m, m, rad
$R(\theta)$	Matriz de rotación plana (Ec. (3.1))	adim.

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
$v[k]$	Velocidad lineal del robot	m/s
$\omega[k]$	Velocidad angular del robot	rad/s
$v^{cmd}[k]$	Consigna lineal del robot (Ec. (3.5))	m/s
$\omega^{cmd}[k]$	Consigna angular del robot (Ec. (3.5))	rad/s
$\omega_j[k]$	Velocidad angular de rueda $j \in \{R, L\}$	rad/s
$\widehat{\omega}_j[k]$	Velocidad angular exportada por `DiffDriveArduino`, calculada con el intervalo real Δt_k	rad/s
$\varphi_j[k]$	Ángulo acumulado de rueda j	rad
$c_j[k]$	*Ticks* acumulados del *encoder* de rueda j	*ticks*
$\Delta c_j[k]$	Incremento de *ticks* del *encoder* en el intervalo	*ticks*
N_e	Resolución efectiva del *encoder* (`ENC_TICKS_PER_REV`)	*ticks*/vuelta
N_e^{teo}	Resolución teórica del *encoder* ($N_e^{teo} = N_{Hall}\, c_q\, N_g$)	*ticks*/vuelta
N_{Hall}	Pulsos por vuelta del sensor Hall en el eje del motor	pulsos/vuelta
c_q	Factor de decodificación en cuadratura	adim.
N_g	Relación de reducción del motorreductor (nominal 46:1)	adim.
$\bar{N}_R, \bar{N}_L$	Promedio experimental de *ticks* por vuelta (rueda derecha/izquierda)	*ticks*/vuelta
f_{ctrl}	Frecuencia nominal del ciclo `read()`–`update()`–`write()`, configurada mediante `controller_manager.update_rate`	Hz
f_{loop}	Frecuencia nominal usada por `DiffDriveArduino` en la conversión de rad/s a *ticks/frame*, configurada mediante `loop_rate`	Hz
$f_{PID,nom}$	Frecuencia nominal declarada por `PID_RATE` en el *firmware*	Hz
$T_{PID,sched}$	Incremento del programador del *firmware*, igual a 33 ms por división entera	s
T_{cap}	Período de captura en lazo abierto	s
Δt_k	Intervalo usado por `DiffDriveArduino`: en la primera lectura se mide desde la asignación de `time_` al inicio de `configure()`, antes de la espera serial de 3000 ms; desde la segunda, entre llamadas consecutivas a `read()`	s
T_k	Intervalo entre las muestras k y k + 1 del modelo cinemático general (Ecs. (3.6)–(3.8)); distinto del intervalo ejecutado Δt_k	s

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
Conversión y control de firmware		
$m_j^{int}[k]$	Conversión a `int` del incremento $c_j[k] - c_j[k-1]$, almacenada en `PrevInput`	*ticks/frame*
$m_j^{cmd}[k]$	Referencia entera en *ticks/frame* enviada al *firmware*; `DiffDriveArduino` la genera al convertir ω_j^{cmd} o v_j^{cmd} (conversión descrita en el ANEXO 3.5)	*ticks/frame*
$v_j^{cmd}[k]$	Velocidad lineal de rueda comandada	m/s
$\omega_j^{cmd}[k]$	Velocidad angular de rueda comandada	rad/s
$u_j[k]$	PWM aplicado al motor j: salida del controlador en lazo cerrado; entrada de planta (escalón) en identificación FOPDT	PWM
$e_j^{long}[k]$	Error de control después de la conversión por asignación al tipo `long`	*ticks/frame*
$e_j^{db}[k]$	Error de control después de aplicar la banda muerta	*ticks/frame*
$\Delta u_j^{long}[k]$	Incremento de la salida calculado por el controlador discreto y asignado al tipo `long`	PWM
$u_j^{raw}[k]$	Salida acumulada antes de aplicar la saturación	PWM
$trunc_0$	Conversión entera con truncamiento hacia cero, dentro del rango representable del tipo de destino	No aplica
$I_j^{int}[k]$	Término integral acumulado de la rueda j, almacenado como `int`; posee la misma escala interna que $K_{p,j}e_j^{db}[k]$	esc. fw
$K_{p,j}, K_{d,j}, K_{i,j}$	Ganancias PID de *firmware* implementadas (escala interna del algoritmo; las ganancias son `float`, pero sus productos se asignan a estados enteros). Obtenidas mediante PSO y refinamiento experimental	esc. fw
$K_{p,j}^{cont}, K_{i,j}^{cont}, K_{d,j}^{cont}$	Ganancias PID continuas ideales (forma ilustrativa, no cuantificadas)	PWM · s/m, PWM/m, PWM · s^2/m
K_o	Divisor de normalización de la salida PID (común a ambas ruedas, = 50)	adim.
κ_{sim}	Factor de conversión m/s $\rightarrow$ *ticks/frame*, igual a $N_e/(2\pi r f_{PID,nom})$ ($\approx$ 313.56); empleado por `SintonizacionAutomatica.py`, `SintonizacionLambda.py` y por el *firmware* en el comando `k`	$\frac{ticks/frame}{m/s}$
T_{sim}	Período de la recurrencia de simulación del PID ($T_{sim} = 1/f_{PID,nom} = 1/30$ s)	s

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
$S_j^e[k]$	Error acumulado desde el último reinicio, usado en la forma no incremental de la ley (Ec. (4.4))	*ticks/frame*
$S_j^I[k]$	Acumulación del estado integral desde el último reinicio (Ec. (4.4))	esc. fw
Forma incremental y estabilidad del lazo de velocidad		
T_m	Período de muestreo del controlador discreto. En el *firmware*, $T_m = T_{PID,sched}$	s
K_c	Ganancia proporcional de la forma incremental del PID (Ec. (A4.1))	$\frac{\text{PWM}}{\textit{ticks/frame}}$
T_i, T_d	Tiempos integral y derivativo de la forma incremental del PID	s
b_{sp}	Ponderación del punto de consigna en la acción proporcional	adim.
$m^{ref}[k]$, $m[k]$, $e[k]$	Referencia, medición y error de la forma incremental del PID, con $e = m^{ref} - m$	*ticks/frame*
Δ, Δ^2	Primera y segunda diferencia entre muestras consecutivas de una señal	No aplica
M_s	Sensibilidad máxima del lazo	adim.
Identificación FOPDT		
S	Variable compleja de Laplace (en mayúscula para no confundirla con el segundo, s)	No aplica
$G_j(S)$	Función de transferencia FOPDT de la rueda $j \in \{R, L\}$	(m/s)/PWM
$K_{m,j}$	Ganancia estática del modelo de la rueda j	(m/s)/PWM
τ_j	Constante de tiempo del modelo de la rueda j	s
$L_{d,j}$	Retardo puro estimado para la rueda j	s
$n_{d,j}$	Retardo entero aplicado en la discretización de la rueda j	muestras
$u_{j,d}[k]$	Entrada PWM retardada utilizada en el modelo discreto	PWM
$\hat{v}_j[k]$	Velocidad lineal estimada por el modelo de la rueda j	m/s
$v_j^{med}[k]$	Velocidad lineal medida de la rueda j en lazo abierto	m/s
e	Número de Euler	No aplica
$v_j^{filt}[k]$	Velocidad medida tras el filtro uniforme centrado	m/s
Sintonía Lambda (comparación ilustrativa)		
λ_j	Constante de diseño de la sintonía Lambda ($\lambda_j = 3\tau_j$)	s
$L_{ef,j}$	Retardo efectivo Lambda, máx($L_{d,j}$, 2/30)	s

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
f_λ	Factor de escalamiento a la escala de *firmware*, $143\,K_o/\kappa_{sim}$	esc. *script*
Optimización por enjambre (PSO)		
$\xi_{i,d}, V_{i,d}$	Posición y velocidad de búsqueda de la partícula i, dimensión d	adim.
$\xi_{mín}$	Límite inferior aplicado al estado de partícula (10^{-4}); el estado no tiene límite superior	adim.
$P_{i,d}, G_d$	Mejor posición individual y vector de posición global almacenado	adim.
J^*	Mejor costo global almacenado, que corresponde al mismo candidato que G_d (ANEXO 4.8)	varias
ζ	Factor de constricción (≈ 0.7298)	adim.
w_{in}^g	Peso de inercia (decreciente con la generación g)	adim.
c_1, c_2	Coeficientes cognitivo y social	adim.
ψ	Suma de coeficientes cognitivo y social ($\psi = c_1 + c_2 = 4.1$)	adim.
ρ_1, ρ_2	Coeficientes aleatorios $\sim \mathcal{U}[0, 1]$	adim.
$\mathcal{U}[a, b]$	Distribución uniforme continua en el intervalo [a, b]: todos sus valores son igual de probables. El exponente indica la dimensión del vector sorteado; por ejemplo, $\mathcal{U}[0, 1]^3$ denota tres componentes muestreadas de forma independiente en [0, 1]	No aplica
$g, g_{máx}$	Generación actual y generación máxima	No aplica
N_p	Número de partículas del enjambre	No aplica
θ_i^{eval}	Vector de parámetros físicos usado para evaluar el candidato de la partícula i	varias
θ^{out}	Vector de parámetros informado al finalizar el PSO, escalado desde la posición global almacenada sin aplicar un nuevo recorte superior	varias
J	Función de costo	varias
w_{mse}, w_{OS}	Pesos de error cuadrático y de sobreimpulso	adim.
v_j^{ref}	Velocidad de consigna estacionaria obtenida como media para $t \geq 7$ s	m/s
Validación experimental y cobertura		
d_{ref}	Distancia comandada en la prueba rectilínea	m
d_{real}	Distancia física medida o verdad de terreno	m
d_{odom}	Distancia estimada mediante odometría	m
E_{seg}	Error de seguimiento respecto de la distancia comandada	%
E_{odom}	Error entre la odometría y la distancia física medida	%

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
$\Delta\theta_{360}$	Referencia nominal fija de 360° utilizada por los *scripts* de giro	grados
$\Delta\theta_{obs}$	Giro observado: acumulación de `/odom` en simulación o medición física en *hardware* real	grados
$D_{\theta,360}$	Desviación angular normalizada respecto de $\Delta\theta_{360}$; no corresponde a error odométrico angular ni a seguimiento de una consigna efectiva sin limitación	%
M	Conjunto de intensidades de celda de la imagen PGM del mapa analizado	No aplica
p	Intensidad de una celda de la imagen PGM	nivel de gris
$\mathbf{1}[\cdot]$	Función indicadora de una condición lógica	adim.
N_{libre}	Número de celdas clasificadas como espacio libre	celdas
$N_{desconocido}$	Número de celdas clasificadas como desconocidas	celdas
C_{libre}	Índice de cobertura libre del mapa	%
A_{libre}	Área libre mapeada, calculada como $N_{libre}\rho_{map}^2$	m^2
Estadística de los ensayos		
n	Número de repeticiones de un ensayo. En A^*, n denota el nodo evaluado	No aplica
μ, σ	Media y desviación estándar muestrales de un indicador, esta última con n – 1 en el denominador. En la Ec. (3.12), σ denota la variable de integración	unidad del indicador
$t_{0.975,\,n-1}$	Cuantil 0.975 de la distribución t de Student con n – 1 grados de libertad, igual a 2.776 para n = 5	adim.
Navegación y exploración		
f(n), g(n), h(n)	Evaluación, costo acumulado y heurística de A^*	No aplica
n, α	Nodo evaluado y celda sucesora en A^*	No aplica
J_{DWB}	Costo total del controlador local DWB	No aplica
C_c, S_c	Crítico c del DWB y escala configurada para ese crítico	No aplica
d_o	Distancia al obstáculo	m
$r_{inflacion}$	Radio de inflación	m
β	Factor de decaimiento del costo (`cost_scaling_factor`)	1/m
U(F)	Puntaje de costo de la frontera F (heurístico, no homogéneo)	No aplica
|F|	Contador de tamaño de la frontera; se inicializa en 1 y se incrementa por celdas vecinas incorporadas. Si no se añade ninguna, la frontera conserva tamaño 1	celdas

continúa en la página siguiente

(continuación de la Nomenclatura)

Símbolo	Descripción	Unidad
p_F^{impl}	Punto representativo de la frontera F almacenado por la implementación como `centroid`; no es el centroide geométrico de todas sus celdas (ANEXO 5.6)	m
d_{min}^{world}	Distancia mínima de la frontera F al robot, en coordenadas del mundo (`frontier.min_distance`)	m
ρ_{map}	Resolución del mapa/*costmap* (`resolution`); en este sistema 0.05, usada tanto en el área libre como en el costo de frontera	m/celda
w_{pot}, w_{gain}	Pesos de distancia (`potential_scale`) y de ganancia (`gain_scale`)	adim.
f_{exp}	Frecuencia configurada de reevaluación del explorador (`planner_frequency`)	Hz
$\varepsilon_j[k]$	Error de seguimiento del modelo para la rueda j, $v_j^{ref} - \hat{v}_j[k]$	m/s